T · · Systems · · ·

rvsEVO

Version 7.00

Operator Manual

rvsEVO
Version 7.00
Operator Manual

© 2025 by T-Systems International GmbH
Winterfeldtstr. 21
10781 Berlin

This manual is protected by copyright. All rights reserved. No part of this book may be used or reproduced in any form or by any means including photocopies, microfilm or any other means or stored in a database or retrieval system without obtaining prior permission from T-Systems. Rights are also reserved as far as lectures, radio and television is concerned.

We reserve the right to make changes to the content of this manual without giving prior notice. T-Systems is not liable for technical or printing errors or defects in this manual. Moreover, T-Systems shall not be liable for damage which is directly or indirectly caused by delivery, performance and use of this material.

Content

1 Introduction 7
1.1 Representation Means 7
2 Technical Pre-Conditions 9
2.1 System Requirement 9
2.2 Networks 9
3 Migration from rvs® portable to rvsEVO 11
3.1 Migration Procedur 11
4 Fall Back after Update Installation 13
4.1 Fall Back after Update of rvsEVO Version 5.xx to Version 6.xx 13
4.2 Fall Back after Update of rvsEVO with Derby Database 13
4.3 Fall Back after Update of rvsEVO with external Database 13
5 Server Farm of rvsEVO 15
5.1 System Requirements 15
5.2 Architecture of a Server Farm 15
5.3 Installation of an rvsEVO Server Farm 17
5.4 Setting up of a Server Farm 23
5.5 Update of rvsEVO Server Farm 24
5.6 Server Farm with rvs® OFTP Proxy Implementation 24
6 Multitenancy Functionality of rvsEVO 27
6.1 System Requirements 27
6.2 Installation of rvsRuntime 27
6.2.1 Obtaining a License 27
6.2.2 New Installation of rvsRuntime 27
6.2.3 Update Installation 29
6.2.4 rvsRuntime Installation Folder 30
6.3 Graphical User Interface of rvsRuntime 31
6.4 Managing of rvsEVO Tenants 31
6.4.1 Creation of rvsEVO Tenant 33
6.4.2 Distritution of Licensekey or Station Configuration 37
6.4.3 Deletion of rvsEVO Tenant 38
7 Distributed Update 41
7.1 Pre-Conditions 41
7.2 processDistributedUpdate Tool 42
7.3 Status Values and NERP Messages of Distributed Update 44
8 Odette Partner Data eXchange (PDX) 47
8.1 How to Exchange Partner Data 47
8.1.1 Offline Method 47
8.1.2 Online Method 48
8.2 processPdxOutbound and processPdxInbound Tools 49
8.2.1 processPdxOutbound: Export of Station Data 49
8.2.2 processPdxInbound: Managing Incoming PDX Files 51
8.3 Files and Configuration Directories 53
8.3.1 PDX Files 53
8.3.2 Properties Files 54
9 rvsEVO within Container Environment 55
9.1 User and Credentials and Mounted Directories 55
9.2 How to Deploy and Run rvsEVO into Docker Container 56

© T-Systems

9.2.1 Test of rvsEVO Server Start within the Init Container . . . 58
9.2.2 Start of Productive rvsEVO Server Container . . . 59
9.3 Update of rvsEVO in Docker Container . . . 59
9.4 Server Farm of rvsEVO within Container Environment . . . 61
9.4.1 Creating a Cluster Node of rvsEVO Server Farm . . . 61
9.4.2 Update Server Farm of rvsEVO within Container Environment . . . 63
9.5 rvs® OFTP Proxy within Container Environment . . . 64
9.5.1 Mounting the Configuration and Log Directories . . . 64
9.5.2 Setting up an RMI Communication via TLS Protocol . . . 65
9.5.3 Docker Command (Template) . . . 66
10 Customizing rvsEVO . . . 69
10.1 Customizing the RMI Parameters . . . 69
10.2 Customizing rvsEVO via rvs-system.properties File . . . 70
10.3 Customizing rvsEVO via rvsEVOClient.prefs File . . . 77
10.4 Customizing the Log Files . . . 79
10.5 Customizing rvsWeb and REST-API . . . 80
10.6 Customizing RMI Connection over TLS . . . 82
10.6.1 Setting up of an RMI Connection via TLS Protocol . . . 82
10.6.2 Customizing RMI Connection via SSH Tunneling . . . 83
10.7 Pass Parameters to Jobstart Process . . . 84
10.8 Conversion Tables . . . 87
10.8.1 Code Tables Offered by rvsEVO . . . 87
10.8.2 How to Add Your Own Conversion Table . . . 87
10.9 Connection Pooling . . . 88
10.10 Operating with OFTP Routing Rules . . . 89
11 Maintenance of rvsEVO . . . 91
11.1 rvsEVO Installation Folder . . . 91
11.2 Maintenance of Directories . . . 92
11.3 Maintenance of rvsEVO Database . . . 93
11.3.1 Deletion of Job Data from Database . . . 93
11.3.2 Deletion of Monitor Log Data from Database . . . 94
11.3.3 Deletion of Jobs and Monitor Log Data from the Database . . . 95
12 ODETTE Protocol . . . 97
12.1 RFC 5024: OFTP Commands for OFTP File Transfer . . . 97
12.1.1 Message Flow of Odette Session . . . 98
13 Monitoring of rvsEVO . . . 101
13.1 What Gets Logged . . . 101
13.2 Log Files . . . 101
13.2.1 monitor.log . . . 101
13.2.2 rvs.log . . . 105
13.2.3 rvsClient.log . . . 105
13.3 Tracing . . . 106
13.3.1 Odette Tracing . . . 106
13.3.2 TLS Tracing . . . 107
13.3.3 API Tracing . . . 107
14 Error Situations of rvsEVO . . . 109
14.1 Debugging the Installer . . . 109
14.2 Failure of Installation . . . 109
14.2.1 Wrong Permission of DB Scripts . . . 109

14.2.2 Aborting of Installation on Linux/Unix Systems 109
14.2.3 Aborting of Installation on UNIX Systems 110
14.2.4 Aborting of Installation on Windows Systems 111
14.2.5 Installation of rvsEVO or rvs® OFTP Proxy aborts with NoClassDefFoundError 111
14.3 Failure of rvsEVO Server Start 111
14.3.1 License over 111
14.3.2 Failure of rvsEVO Server Start with Derby Database 112
14.4 Failure of Listener Start 113
14.4.1 Port is already in use 113
14.5 Failure of RCP-GUI Start on Linux Systems 114
14.6 Connection Failure 114
14.6.1 Connection to Proxy Bastion Fails 114
14.6.2 Examples with outgoing call, TCP Connection 116
14.6.3 Examples with outgoing call, TLS Connection 119
14.7 Failure during Bild-up of OFTP Session to Neighbour Station 121
14.8 Failure of File Transmission 122
14.8.1 SFNA and EFNA 122
14.8.2 SFNA: REAST=15 „Cipher suite not supported“ 122
14.8.3 Error Messages via NERP 123
14.8.4 Outgoing Call, EERP is Missing 124
14.8.5 Routing Send Job Fails with ESIDREAS=06 125
14.9 File Transmission: Pre- or Post-Processing Fails 125
14.9.1 Key or Certifiacte is Missing 125
14.9.2 Jobstart After Receive Fails on Receiver Side 126
14.9.3 Failure of OFTP2 Conversion with Cluster Functionality on Linux and UNIX Systems 126
14.10 Failure of Connection from rvsEVO Client to rvsEVO Server 127
14.10.1 Failure of Connection to Server if java.rmi.server.hostname is set 127
14.10.2 Failure of Connection to Server in Case of Invalid Login Data 127
14.10.3 Failure of Connection to Server with rvsEVO Server Farm after Update 128
14.11 Failure during Launching Scripts in „tools“ Directory 129
14.12 Failure of rvsRuntime Commands 129
14.12.1 Failure of starting a tenant 129
15 rvsEVO Support 131

1 Introduction

The rvsEVO Operator Manual is an addition to the rvsEVO User Manual. It is meant for operators and administrators.

This manual contains information that 7.00goes beyond basic usage information like first installation or sending and receiving files. It provides:

– detailed technical information

– detailed description of

  – Migration from rvs® portable to rvsEVO

  – Fall back scenario

  – Application cluster functionality

  – Multitenancy funktionality

  – Customizing rvsEVO

  – Maintenance of rvsEVO

  – Monitoring

– Failure solutions.

For information of topics relating to the normal usage please have a look in the rvsEVO User Manual.

1.1 Representation Means

This chapter describes the typographic conventions used in this manual and explains the meaning of specially highlighted expressions.

Typographic Conventions

• Instructions begin with a bullet.

– Other lists begin with the en dash.

Character styles

`Courier`	Commands, menu commands, file names, path names, programs, examples, scripts, options, qualifiers, data sets, fields, modes, window names, dialog boxes and statuses
BOLD and **UPPER CASE**	Parameters, environment variables, variables
"Inverted comma"	Links to other manuals, sections and chapters, literature
Bold	Important terms, names of operating systems, proper names, buttons, function keys

Installation Directory:

$RVS_HOME As user directories are found on different locations for the different operating systems we use the variable **$RVS_HOME** in this manual. Default values are:

– `C:\Programs\`rvsEVO for **Windows**.

Substitute the variable with your correct path.

2 Technical Pre-Conditions

This chapter describes the system requirements and other information about technical pre-conditions.

2.1 System Requirement

To successfully operate rvsEVO you need the following software:

Software

- Operating system: **Windows 10 / Windows 11 / Windows Server 2016 / Windows Server 2019 / Windows Server 2022 / Windows Server 2025** or **UNIX (AIX)** as well as **Linux** and **zLinux.**
- X11-Server (Linux or UNIX operating systems): for graphics mode of rvsEVO installer and for Swing GUI.
- X11-Server and GTK-2.4 (Linux): for RCP GUI.
- Java runtime environment (JRE) version 21.

Hints:

- With Windows and Linux systems the installer distributes the required Java runtime environment version into the rvsEVO installation folder.
- With UNIX systems you can select an installed JRE version 21 during the installation.
- Initially, you need at least 400 MB free space on your hard disk. Depending on the amount of usage, the retention period for old entries, and the time between database cleanups, the space requirement may be considerably larger.

2.2 Networks

Communication line based on TCP/IP, TLS, Proxy TCP/IP und Proxy TLS.

3 Migration from rvs® portable to rvsEVO

This chapter describes the migration from rvs® portable to rvsEVO.

Please have a look at rvsEVO User Manual for information about first installation and usual update installation.

Requirements

For migration of settings from rvs® portable you have to consider the following requirements:

– an executable rvs® portable must be installed on your system

– migration should only be started if the absence of communication has been assured (no file transmission and no encryption/compression)

– The system variable **RVSENV** has to be set for logged on user. Set **RVSENV** by choosing `Start -> Control Panel -> System -> Advanced -> Environment Variables`.

3.1 Migration Procedur

The following steps need to be done for migration from rvs® portable to rvsEVO:

- Install rvsEVO on the same machine where rvsXP / rvsX is installed like described in the rvsEVO User Manual.
- The installer searches through your system for an existing rvs® portable. If you are asked whether you wish to import the settings from rvs® portable, press the **Yes** button.



- Only with encrypted transmission: import the public keys of your partners.
- Create the users which were defined in rvs® portable. They will not be created automatically.

Note: Use the following command tools in `$RVS_HOME/tools` directory for importing the settings into an existing rvsEVO:
`$RVS_HOME/bin/portable2codetable.bat`
`$RVS_HOME/bin/portable2jobstart.bat and`
`$RVS_HOME/bin/portable2stationlist.bat`

portable2codetable

Use this tool for converting a codetable from rvs® portable (rtc format) to xml format.

Syntax:

`portable2codetable <rtc file> <xml file>`

`rtc file` is the name of the codetable in rvs® portable. `xml file` is the name of the codetable in rvsEVO.

portable2jobstart

Use this tool for creating a jobstart list (xml file) from rvs® portable. rvs® portable must be launched.

Syntax:

`portable2jobstart <file>`

`file` is the xml file name. The file can be imported into the rvsEVO database via `$RVS_HOME/tools/importJobstarters` script.

portable2stationlist

Use this tool for creating a stationlist (xml file) from rvs® portable. rvs® portable must be launched.

Syntax:

`portable2stationlist <file>`

`file` is the xml file name. The file can be imported into the `rvsEVO` database via `$RVS_HOME/tools/importStationlist` script.

Important: rvsEVO supports only tools which are provided by rvsEVO and no other tools which are supported by rvsX / rvsXP. Please see rvsEVO User Manual, chapter 'Command Line Tools' and 'Database Maintenance and further Administrator Tools' for getting information about the supported tools.

Communication with rvs400:

In case of transmitting files bigger than 2 GB with AS/400 machine, EFID gets a negative value. Thus an EFNA is sent to the sender of the file and the transmission is restarted.

Set the denifinition `rvs_evo.receive_job.ignore_illegal_efid_values=true` in `$RVS_HOME/conf/rvs-system.properties` file to circumvent this problem.

(In rvs® portable this problem is fixed via RVSNOSIZECHECK environment variable.)

4 Fall Back after Update Installation

This chapter describes a fall back scenario. In case of a faulty update you can fall back to your previous rvsEVO version. The procedure depends on the rvsEVO version and the database.

Please have a look at rvsEVO User Manual for information about first installation and usual update installation.

4.1 Fall Back after Update of rvsEVO Version 5.xx to Version 6.xx

The installer is saving the installation directory in a directory named **$RVS_HOME** with added time stamp (Format: `$RVS_HOME.backupYYYY-MM-DD_hh-mm-ss`).

Fall Back:

For falling back to your origin installation delete or rename the updated rvsEVO and rename the saved installation directory to **$RVS_HOME**.

4.2 Fall Back after Update of rvsEVO with Derby Database

Preparation Prior to Update:
Save a copy of your rvsEVO installation directory.

Fall Back:

– Delete or rename your updated rvsEVO installation.

– Renname your saved rvsEVO installation directory into the origin **$RVS_HOME** directory name.

– Check wether rvsEVO is correctly installed as Windows service and the entry in your **$RVS_HOME** variable.

4.3 Fall Back after Update of rvsEVO with external Database

Preparation Prior to Update:

– Back up the rvsEVO data. Please see the rvsEVO User Manual chapter ’Backing Up and Recovering rvsEVO Data’ for more information about the back up procedure.

– Save a copy of your rvsEVO installation.

Fall Back:

– Delete or rename your updated rvsEVO installation.

– Renname your saved rvsEVO installation directory into the origin **$RVS_HOME** directory name.

– Recover the rvsEVO data from the back up file. Please see the rvsEVO User Manual chapter ’Backing Up and Recovering rvsEVO Data’ for more information about the recover procedure.

© T-Systems

– Check wether rvsEVO is correctly installed as Windows service and the entry in your **$RVS_HOME** variable.

Hints:

– During the update procedure the rvsEVO installer saves a back up of the rvsEVO data in the users `temp` directory. The directory with backup files is called 'rvsdata' plus time stamp [`rvsdata.YYYY-MM-DD-hh-mm-ss`]. Windows: `C:\Users\user_name\AppData\Local\Temp`.

– In case of a faulty update installation the rvsEVO installer saves an update log file in the users `temp` directory.

– For each rvsEVO installation/update installation a log file is also saved in **$RVS_HOME/UninstallerData** directory.

5 Server Farm of rvsEVO

rvsEVO provides the application cluster functionality with a server farm.

See the rvsEVO User Manual Chapter „**Server Farm of rvsEVO**“ for information about provided functionalities and benefits of application cluster capability.

5.1 System Requirements

System requirements

To successfully operate the server farm you need the following components:

– Operating systems: Windows, Unix or Linux. The same operation system should be installed on all nodes.

– Database: Oracle 12 and higher, MS SQL, PostgreSQL

– IP load balancer

– network file System/network drive (e.g. NFS or CIFS)

Hint: The system clock of all nodes of the server farm has to be synchronized via NTP.

5.2 Architecture of a Server Farm

System architecture

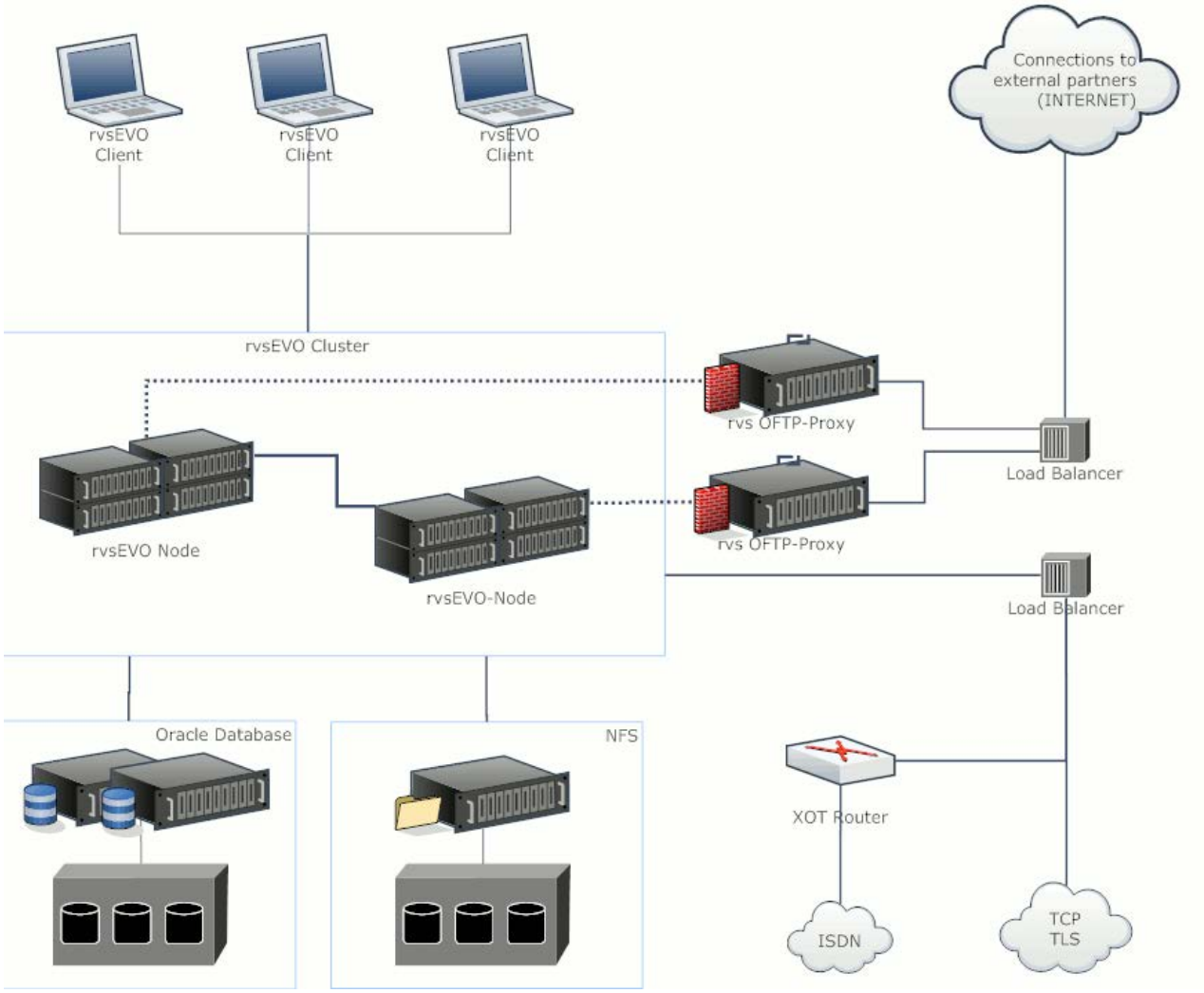
Each rvsEVO node is equipped with all functions and needs access to all resources in the cluster systems (resource sharing).

An rvsEVO cluster comprises the following components:

– several rvsEVO nodes (rvsEVO server with rvsEVO client).

– Network file system

The illustration below shows the system architecture of the operation with application cluster functionality.



Note:

If the network share for data storage is supplied by **CIFS** (Windows: SMB / Linux: SAMBA) **opportunistic locking** must be disabled on the file server.

Proceed as follows to disable **opportunistic locking**:

Unix/Linux Systems: Add in Samba Config for the appropriate share:

```
oplocks = no
```

Windows Server Systems: With Windows Server systems additional the **SMB feature level** is to set down to version 1. This will have effect on the performance.

```
https://support.microsoft.com/en-us/kb/296264
```

https://support.microsoft.com/en-us/kb/2696547)

Perform the following settings via PowerShell:

Windows Server 2012 upwards:

- `Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" EnableOplocks -Type DWORD -Value 0 -Force`
- `Set-SmbServerConfiguration -EnableSMB1Protocol $true`
- `Set-SmbServerConfiguration -EnableSMB2Protocol $false`
- `Restart-Service server -PassThru -Force`
  or restart your system.

5.3 Installation of an rvsEVO Server Farm

Preparation

Please observe the pre-conditions for using an Oracle, PostgreSQL or MS SQL database (see the „rvsEVO User Manual“ chapter „rvsEVO Database“).

Define an alias name for the local IP address of rvsEVO server. Alias name is a pseudonym for the IP address and must be the same on all nodes. Assign an IP address to the alias name in `\etc\hosts` file (on Windows systems in `C:\Windows\System32\drivers\etc` directory).

Example:

```
# rvsEVO alias
102.54.94.97     EVOCLUSTER
```

You also need an alias name for setting up of local receiver. If the IP address of the local receiver is identical to the IP address of rvsEVO server you should use the same alias name.

Hint: With rvsEVO version 6.02.02 and higher, alternatively you can use `host-alias.properties` file in `$RVS_HOME/conf` directory:

- After successful completed installation, add an entry for the alias name in `host-alias.properties` file.

  Syntax:

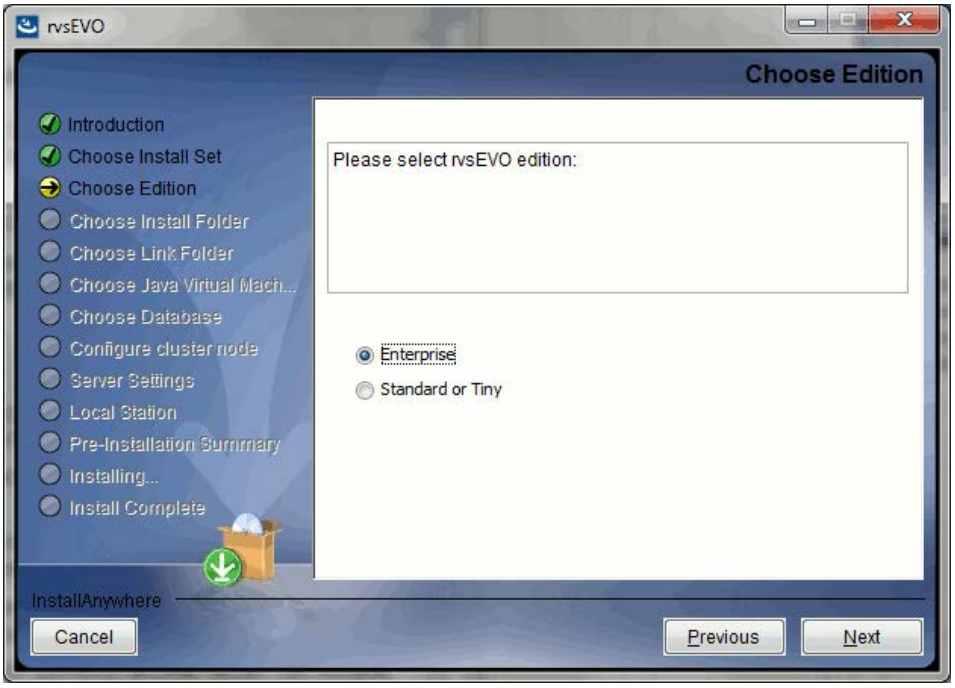
  `<alias>=<hostname>`

- Replace the value of `"client.RMIServiceHost"` in `$RVS_HOME/conf/rvsEvoClient.prefs` file with the IP address or hostname of your machine.

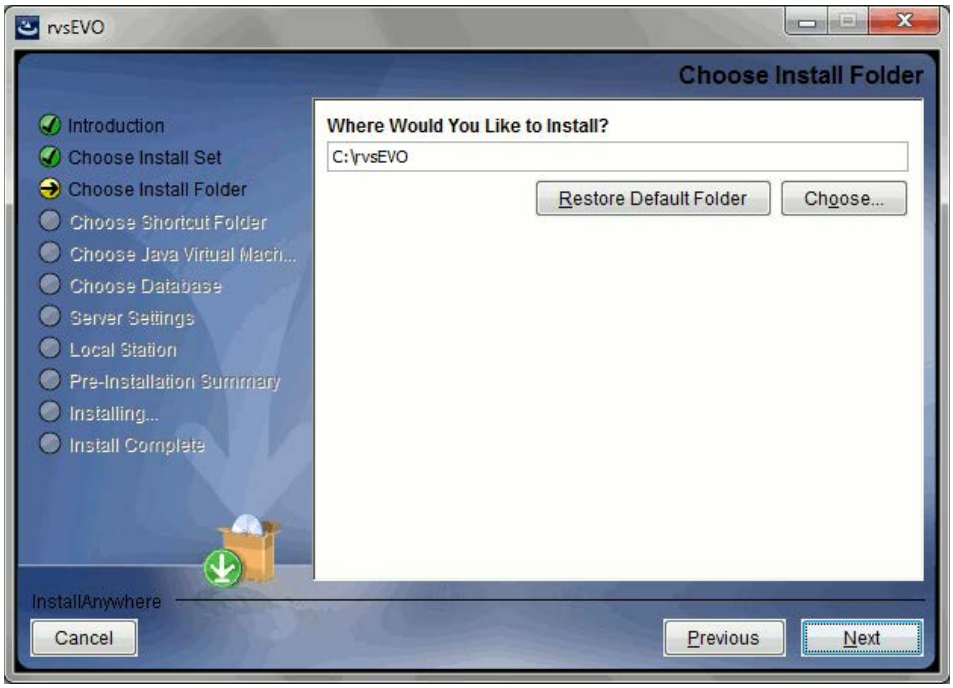
Installation

Installation of a server node runs nearly analog to an installation of a single instance (see the rvsEVO User Manuel, chapter ’New Installation of rvsEVO’). Please notice the following points:

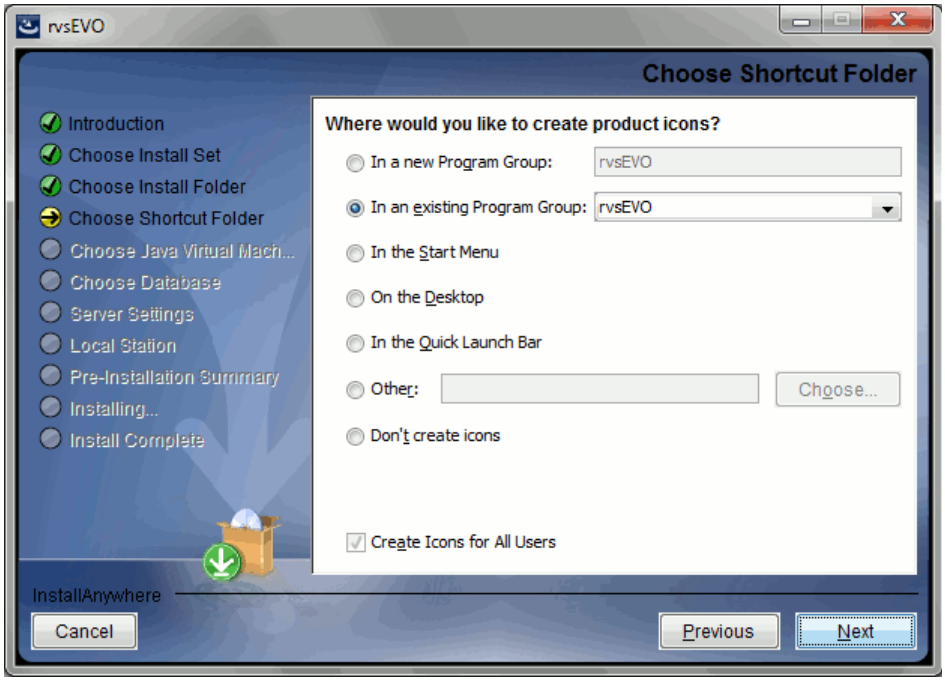
- Install the Enterprise Edition of rvsEVO. The Standard or Tiny Edition does not support application cluster functionality.



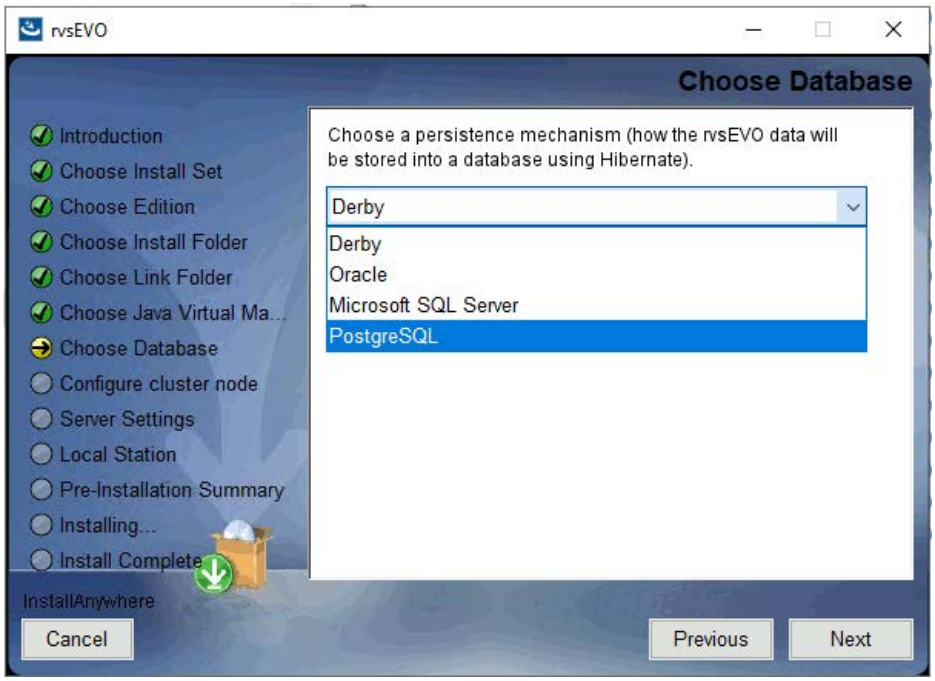
- Use the same installation path for all rvsEVO nodes. This is required because the directory pathes are defind via rvsEVO database.



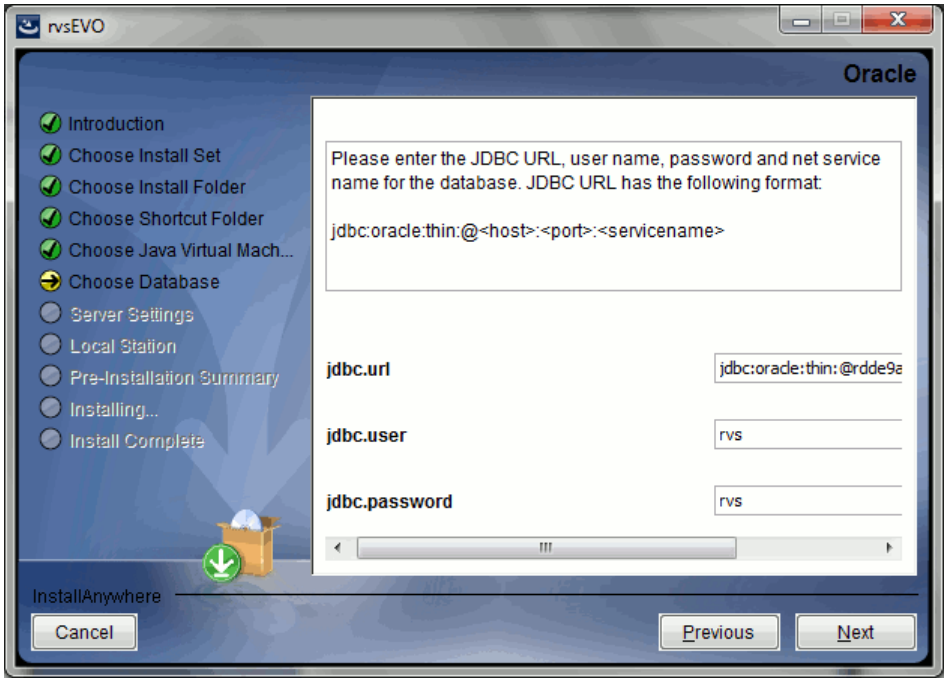
- Define the program group to install rvsEVO icons like described in the rvsEVO User Manual.



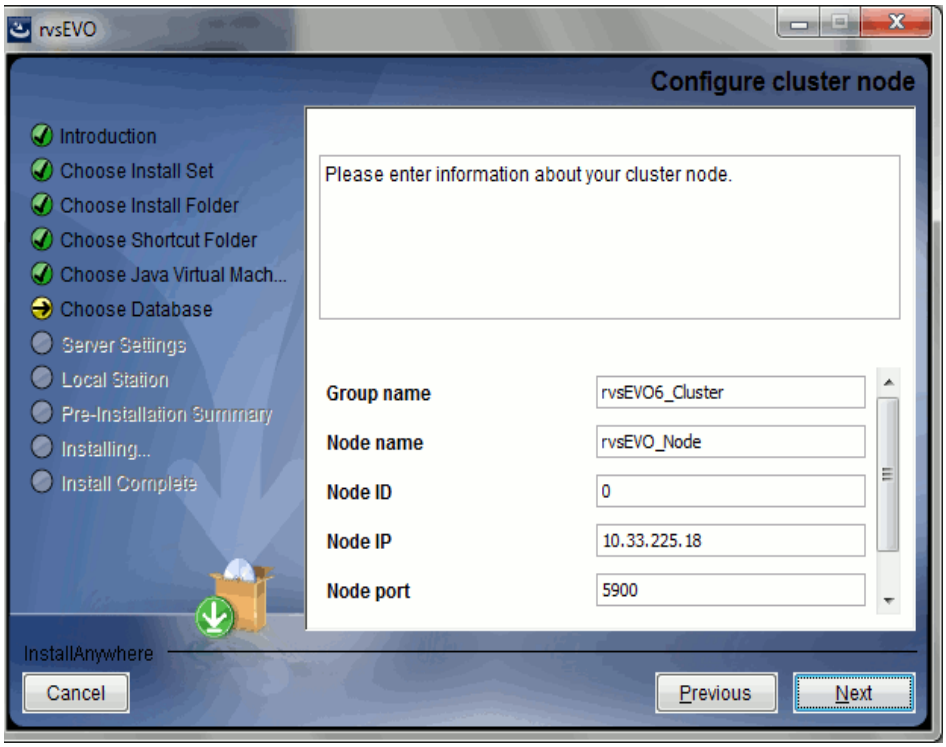
- Use an external database (Oracle, PostgreSQL or MS SQL)



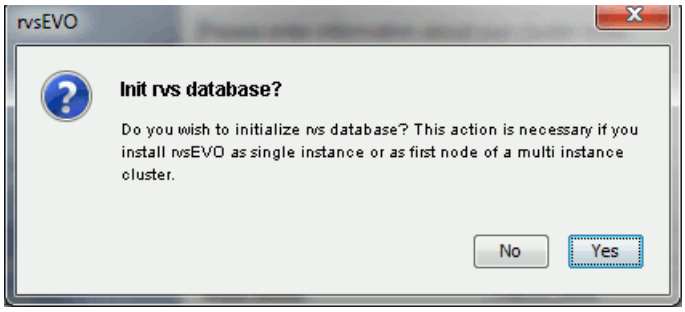
- Set the Oracle, PostgreSQL or MS SQL connection parameters like described in the rvsEVO User Manual.



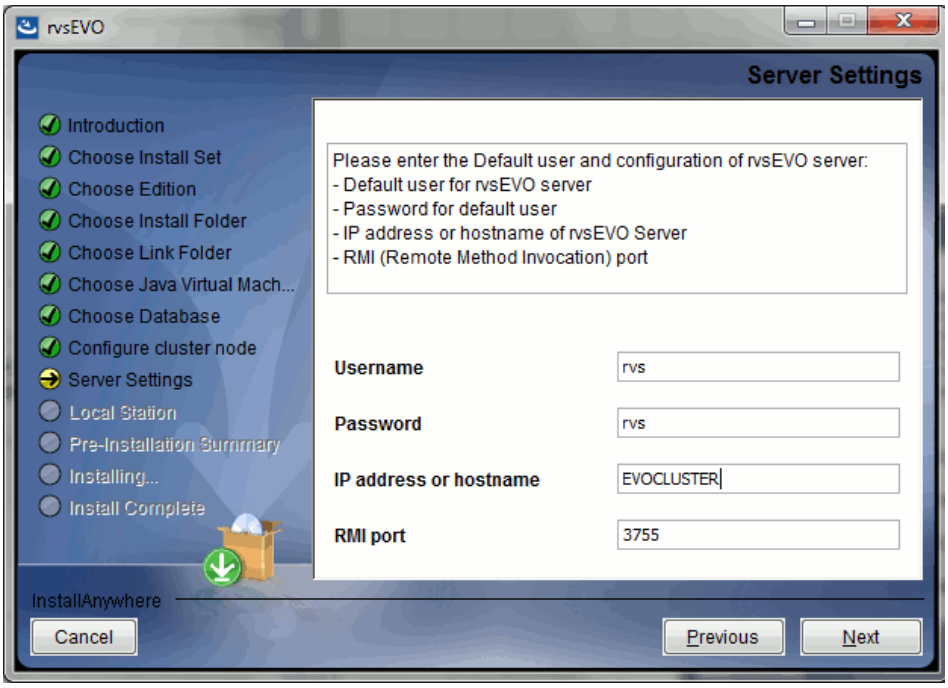
- Define the group name with installation of the first node in window **Configure cluster node**. Use the same group name for each node within the server farm.
  Node Name and **Node ID** must be unique values in your rvsEVO system.
  Node IP is the IP address of the currently installed node.
  Insert the IP addresses of the other nodes within the server farm via **Nodes** parameter. Multiple entries are separated by commas.



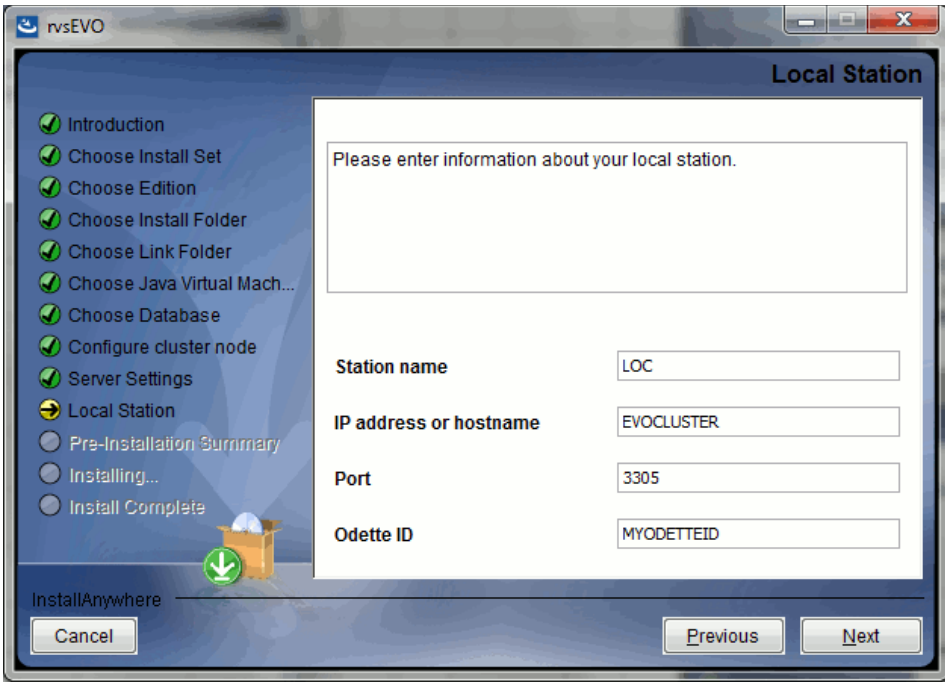
- Initialize the database only with the installation of the first node.



- Enter the server settings like described in the rvsEVO User Manual. Insert in **IP address or hostname** parameter the same alias name (see above) on all rvsEVO nodes and use the same RMI port on all nodes..



- Setting up of the local receiver is only required during installation of the first node. Insert the alias name for IP address or host name.



After installation:

- Save the directory `$RVS_HOME/files` and its subdirectories on the network file system. The path of the subdirectories `/inbox, /outbox und /temp` are to be set up in **DIRECTORY** parameter

group (see rvsEVO User Manual chapter ’Customizing the rvsEVO Parameters’).

- Create directories for jobstart scripts and Observer on the network file system and provide the read and write access to all nodes.
- In case of using `host-alias.properties` file, edit the `host-alias.properties` file and `rvsEvoClient.prefs` file like described above.

5.4 Setting up of a Server Farm

During installation the cluster node has been configured. You can set up the configuration in `$RVS_HOME/conf/cluster.properties` file. This is required after adding or removing nodes. In this case on each node of the server farm the IP addresses of the added or removed nodes has to be added or deleted in `cluster.nodes` definition.

Hint: For Kybernates cluster the DNS service names of the other nodes have to be set instead of the IP addresses.

Example of cluster.properties file:

```
cluster.group.name=rvsEVO6_190
cluster.node.name=rvsEVO_Node
cluster.node.id=0

# Tcp/Ip Cluster
cluster.node.ip=10.33.201.135
cluster.node.port=5900
# If set don't set cluster.k8s.dns.nodes
cluster.nodes=10.33.201.201,10.33.202.221

# Kubernetes Cluster.
# If set don't set cluster.nodes for Tcp/Ip
cluster.k8s.dns.nodes=
```

Definitions of Cluster.Properties

Definition	Description
cluster.group. name	Name of the server farm (has to be identical on each node)
cluster.node.name	Name of the node

Definitions of Cluster.Properties

Definition	Description
cluster.node.id	Unique node identifier. Possible values: `0-9` e.g. first node: `cluster.node.id` = `0`; second node: `cluster.node.id` = `1`; etc. You can also assign value `1` to the first node; value `2` to the second node, etc. (Default is `0`). This value is part of the job ID (fourth last position).
cluster.node.ip	IP address of the node
cluster.node.port	Port of the node (default: 5900)
cluster.nodes	IP addresses of the other nodes within the server farm, separated by commas (alternative to **cluster.k8s.dns.nodes**)
cluster.k8s.dns.nodes	Only for Kybernetes cluster: Kybernates DNS service names of the other nodes within the server farm, separated by commas (alternative to **cluster.nodes**)

5.5 Update of rvsEVO Server Farm

Update of a server node runs nearly analog to an update of a single instance (see rvsEVO User Manual chapter ’rvsEVO Update Installation’).

Please notice the following points:

– Each node of the server farm has to be updated separately.

– Before updateing you do not need to stop all servers of the farm but only the server to be updated.

– Update the database structure with updating the last node.

– During the update installation a `$RVS_HOME/conf/host-alias.properties.new` file is added. Please take over the configurations of your old `host-alias.properties` file to the new one and rename `host-alias.properties.new` in `host-alias.properties`.

– Do not start the rvsEVO server before the last node was updated.

5.6 Server Farm with rvs® OFTP Proxy Implementation

If rvs® OFTP is implemented in the rvsEVO server farm you have to use the same alias name for the IP addresses of Proxy servers and Proxy listeners on each node. Alias name is a pseudonym of IP address. The alias name of the Proxy listeners must also be assigned on the Proxy hosts in `etc/hosts`!

Hint: With version 6.02.02 and higher, alternatively you can indicate the alias name in `conf/host-alias.properties` file of rvs® OFTP Proxy installation directory and **$RVS_HOME/conf** directory. Indicate the alias name after successful completed installation of rvs® OFTP Proxy:

- After successful completed installation, add an entry for the alias name in `host-alias.properties` file.

  Syntax:

  `<alias>=<hostname>`

  `alias` is the alias name used during installation.

  `hostname` is hostname or IP address of the rvs® OFTP Proxy listener

- Subsequently, add one element each for the alias name of the Proxy bastion and the RMI Sender-IP to `conf/host-alias.properties` file of **$RVS_HOME** directory (Format: `PROXYALIAS=rvsproxy-host`).

For encrypting the line between rvs® OFTP Proxy and rvsEVO via TLS you need to use the same keystore for each rvs® OFTP Proxy installation and the same truststore for each rvsEVO node.

Hint for Update-Installation

During the update installation a `host-alias.properties.new` file is added to `conf` directory. Please take over the configurations of your old `host-alias.properties` file to the new one and rename `host-alias.properties.new` in `host-alias.properties`.

6 Multitenancy Functionality of rvsEVO

From version 6.02 upward rvsEVO provides the multitenancy functionality via rvsRuntime.

See the rvsEVO User Manual Chapter "Multitenancy Functionality of rvsEVO" for information about provided functionalities and benefits of multitenancy capability.

6.1 System Requirements

System requirements

To successfully operate rvsRuntime you need the same system requirements as for rvsEVO. Please see the rvsEVO User Manual, chapter "Installation" / "System Requirements".

6.2 Installation of rvsRuntime

Installation Directory

As user directories are found on different locations for the different operating systems we use the variable **$RVS_RUNTIME_HOME** in this manual. Default value for Windows is `C:\rvsEVO`.

Substitute the variable with your correct path.

6.2.1 Obtaining a License

The rvsEVO Server needs to be enabled with a license component to be accepted as a tenant of rvsRuntime.

rvs® After Sales Service

Please contact the rvs® Service Support Center.

From Germany for free, telephone: 0800 664 77 45

From other contries, telephone: +49 375 606 19 902

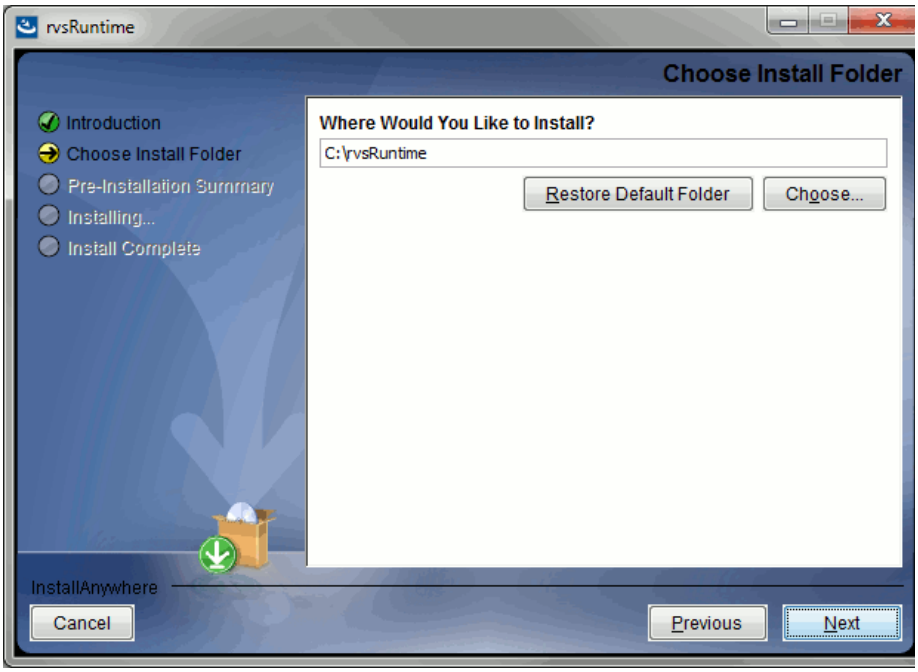
E-mail: rvs-service@t-systems.com

6.2.2 New Installation of rvsRuntime

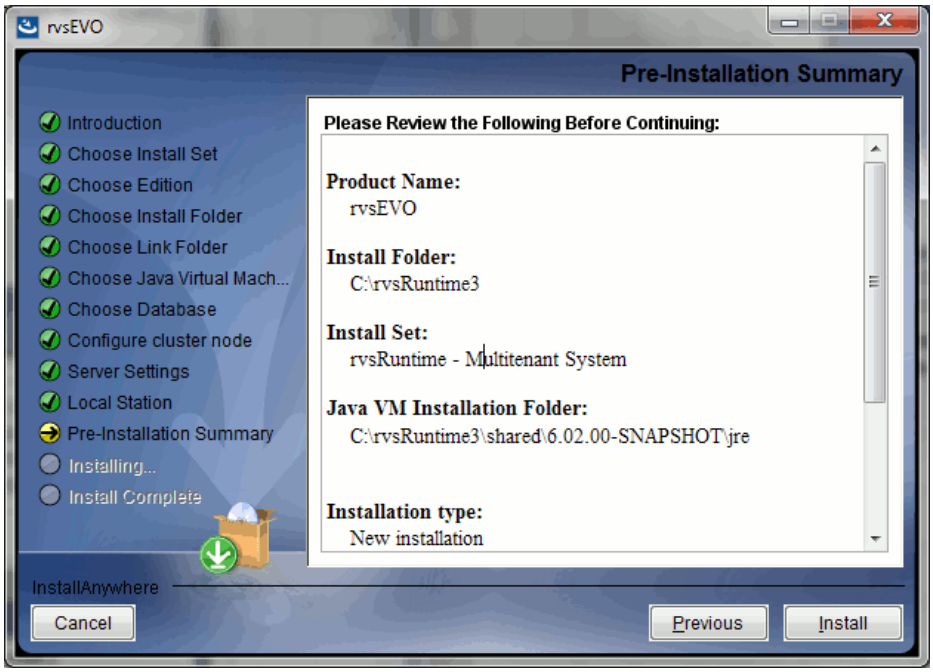
Start the installation process like described in the rvsEVO User Manual, chapter „Installation“ / „ New Installation of rvsEVO and rvsRuntime“.

After the dialog which invites you to choose the install set you will be asked to indicate the rvsRuntime destination directory (**$RVS_RUNTIME_HOME**). Alphanumeric characters without special characters, underscores, hyphens and spaces are permitted for the directory name. If you choose an existing installation folder of rvsEVO or rvsRuntime a warning is displayed and via a mouse-click on **OK** button you can indicate another directory name.

© T-Systems



- In the next dialog you are given a brief overview of selections you have made (installation directory, installation type). The required and the currently available disk space is also displayed. Press the **Install** button to start installation and to copy the installation files into the directory you specified.



- The last dialogs informs you about the successful installation of rvsRuntime.

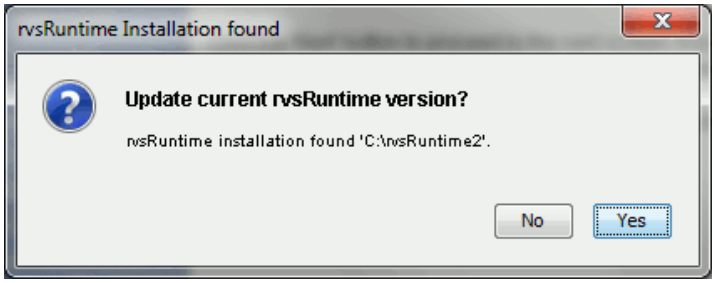
Hint: After installation process you need to save the license key file in the **$RVS_RUNTIME_HOME**\conf\ directory as license.properties.

6.2.3 Update Installation

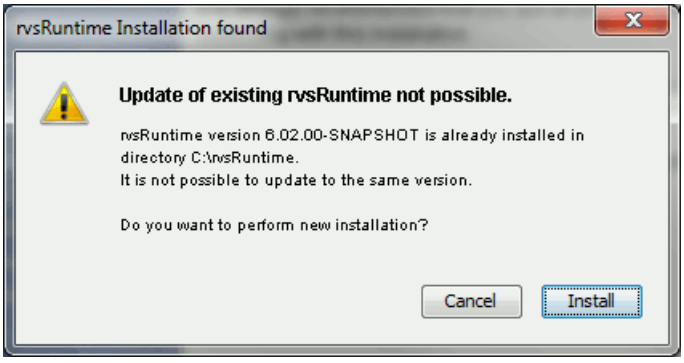
Requirement

The system variable **$RVS_RUNTIME_HOME** (please see chapter 6.2 "Installation of rvsRuntime") has to be set for the logged on user. Set **$RVS_RUNTIME_HOME** by choosing Start -> Control Panel -> System -> Advanced -> Environment Variables.

- Start the rvsEVO-setup-[version]-.exe installer by double-clicking.
- Click the **Yes** button if the installer asks you whether you would like to update your current rvsRuntime.



Hint: The version of the installer must be newer than the version of your current installation. If the versions are identical you get the following message:



- In the next dialog you are given a brief overview of selections you have made. The required and the currently available disk space is also displayed. Press the **Install** button to start the update installation.
- The last window informs you about the successfully installation of rvsRuntime. Finish the installation routine by pressing **Done** button.

6.2.4 rvsRuntime Installation Folder

During installing process the following subdirectories are installed in **$RVS_RUNTIME_HOME** directory. Please see chapter 11.1 "rvsEVO Installation Folder" for information about the folders, which are not described in this section.

- **bin** contains the `rvsrt` tool
- **conf** contains the licensekey and configuration files of rvsRuntime.
- **docs**
  - **Manuals**
  - **jobstart-examples**
  - **rvsEVOClientAPI**
- **log:** contains the installation log file
- **shared:** contains data which are identical for all tenants
  - **<rvsRuntime installation version>**
    - **clib**
    - **jre**
    - **lib**
    - **system**

      –system/conf

      –system/data
- **tenants:** contains the tenant installation directories
  - **<tenant ID>:** contains the special data for the tenant
    - **archive**
    - **bin:** contains the command line tools and `derby.log` file
    - **conf:** contains configuration files and the licensekey of the tenant

      –**conf/conversiontables:** conversiontables
    - **db**
    - **files**
    - **jobs**
    - **log**
    - **snmp-agent**

      –bin

      –conf

      –docs

–mib

- **system**

–system/db

- **tools**

– **UninstallerData** contains the files, needed for uninstalling rvsRuntime

6.3 Graphical User Interface of rvsRuntime

With version 7.00 the GUI of rvsRuntime (RCP-GUI MI) is not supported anymore. Please use the Web-GUI instead of that. Please see chapter 10.5 "Customizing rvsWeb and REST-API" for more information.

6.4 Managing of rvsEVO Tenants

A tenant works like a single rvsEVO instance. Please see the rvsEVO User Manual for more information.

The rvsServerInstanceList.xml file

The `rvsServerInstanceList.xml` file in `$RVS_RUNTIME_HOME/conf/` directory contains the information needed for server connection (tenant ID, tenant name, host name ...).

With creation of a tenant the tenant information is added to `rvsServer-InstanceList.xml` file and with deletion of a tenant it is removed from the list.

`rvsrt` Tool

The `rvsrt.bat` tool in `$RVS_RUNTIME_HOME/bin` directory and its commands provides creation, deletion and managing of rvsEVO tenants.

Syntax:

```
rvsrt  [-h ] [-p][-v] [-V] <command> [command-options]
[<tenent-id>]
```

Attention: For successful execution of commands the sequence of the parameters must be followed as given in the syntax: „`rvsrt`“ <rvsrt parameters> <command> <command options> <tenant ID>.

rvsrt Parameters:

Parameter	Description
-h **-?** **--help**	Displays the description of the command.
-p **--plain-output**	Parameter for plain output.
-V **--version**	Displays the installed rvsRuntime version.
-v **--verbose**	Disables colorised ANSI output

The table below shows the `rvsrt` commands and its functionality.

rvsrt Commands:

Command	Description
create	Creates a new rvsEVO tenant (see chapter 6.4.1 "Creation of rvsEVO Tenant")
delete	Removes an rvsEVO tenant (see chapter 6.4.3 "Deletion of rvsEVO Tenant")
list	Lists the state of all rvsEVO tenants. Syntax: `rvsrt [<rvsrt otpions>] list [-c=<column-list> | --columns=<column-list>] [--help | -h | -?]` **Hint:** You can list the columns to be shown comma-separated. The following values are allowed: `name, version, rmi, userid, runmode, directory, command, color`. Default: tenant ID, state, tenant name
rollout	For distribution of licensekey (available for local instance, only) or station configuration from one rvsEVO tenant to others (see chapter 6.4.2 "Distritution of Licensekey or Station Configuration").
status	Issues a rvsEVO tenant status Syntax: `rvsrt [<rvsrt otpions>] status [--help | -h | -?] <tenant ID>` You get the following information: status, tenant ID, tenant name, version, RMI, user ID, run mode, installation directory, start command, color, login status.
start	Starts an rvsEVO server (available for local instance, only). The start command is defined in `$RVS_RUNTIME_HOME/conf/rvsServerInstanceList.xml` file. Via XML element `startServerCommand` the command can be adjusted to your requirements by the superuser. **Hint:** On Windows systems the command prompt has to be opened as administrator. Syntax: `rvsrt [<rvsrt otpions>] start [--help | -h | -?] <tenant ID>`
stop	Stops an rvsEVO server (available for local instance, only). Syntax: `rvsrt [<rvsrt otpions>] stop [--help | -h | -?] <tenant ID>`

rvsrt Commands:

Command	Description
update	Updates an rvsEVO tenant (available for local instance, only). **Hints**: Multiple tenants should be updated successively. Syntax: `rvsrt [<rvsrt otpions>] update [--help | -h | -?] <tenant ID>`

6.4.1 Creation of rvsEVO Tenant

You can import the parameters for creating a tenant via input file or enter the parameters into the command prompt. An example of input file is given at the end of this section.

Use the `rvsrt` command `create` for creating an rvsEVO tenant and to add the tenant to the server instance list in `rvsServerInstanceList.xml`. The installation directory of the tenant is `$RVS_RUNTIME_HOME/tenants/<TENANT-ID>`

On Windows systems rvsEVO tenant will be installed as Windows service, named „rvsEVO“ <TENANT-ID>, by default.

Hints:

– Please see the „rvsEVO User Manual“ chapter „rvsEVO Database“ before creating a tenant with Oracle, PostgreSQL or MS SQL database.

– The creation process can be terminated via entering `exit` or `quit` command.

Attention:

The sequence of the parameters must be followed as given in the syntax: „`rvsrt`“ <rvsrt parameters> <command> <command options> <tenant ID>.

On Windows systems: The command prompt has to be opened as administrator.

Syntax:

```
rvsrt [<rvsrt options>] create [-r] [-h] [-i
<filename>]
```

Optional `create` Parameters:

Parameter	Description
-r **--remote**	With this parameter you can add an existing rvsEVO installation (on remote machine) to the instance list in **`$RVS_RUNTIME_HOME`**`/conf/rvsServerInstanceList.xml` file.

Optional `create` Parameters:

Parameter	Description
-h **-?** **--help**	Displays a description of the current command.

During creation process the following parameters are requested. The values in parentheses are to be used in input file (see the example below the table).

Requested create Parameters:

Parameter	Description
License file [LICENSE_FILE]	Path and name of license file that will be distributed to the tenant. Default: `$RVS_RUNTIME_HOME/conf/license.properties`
Tenant ID [TENANT_ID]	Unique ID of rvsEVO tenant, max 16 characters. Permitted characters: Alphanumeric characters (upper-case letters) without special characters, underscores, hyphens and spaces. Default: `RVS001`
Tenant name [TENANT_NAME]	Unique name of rvsEVO tenant. Default: „`RVS_`" & <specified Tenant ID>
Database type [DATABASE_TYPE]	With creation of a local tenant, only. rvsEVO Database type. Possible values: - `Derby` (default) - `Oracle` (only with Enterprise Edition) - `Microsoft SQL Server` (only with Enterprise Edition) - `PostgreSQL` (only with Enterprise Edition)
Username [DEFAULTUSER_NAME]	User name of the default user with administrator rights Default: `rvs` **Hint:** Please use unique user names.
Password [DEFAULTUSER_ PASSWORD]	Password of the default user with administrator rights Default: `rvs`
IP address or hostname [RMI_HOST]	IP address or host name for RMI connection Default: host name of your machine

Requested create Parameters:

Parameter	Description
RMI Port [RMI_PORT]	Each tenant needs a unique RMI port Default: 3755

create Parameters for Oracle, PostgreSQL or MS SQL Database:

Parameter	Description
Database host [DATABASE_HOST]	Host Name or IP address for database connection
Database port [DATABASE_PORT]	Port for database connection Default for Oracle: `1521` Default for MS SQL: `1433` Default for PostgreSQL: 5432
Database name [DATABASE_NAME]	Name of external database Default: `EVODB`
Database user [DATABASE_USER]	User setting up on external database Default: `rvs`
Database password [DATABASE_ PASSWORD]	Password of database user Default: `rvs`
Group name [CLUSTER_GROUP_ NAME]	Name of server farm Default: `rvsEVO6_Cluster`
Node name [CLUSTER_NODE_ NAME]	Unambiguous designation of the node to be installed Default: `rvsEVO6_Node`
Node ID [CLUSTER_NODE_ID]	Unambiguous ID of the node to be installed Default: `0`
Node IP [CLUSTER_NODE_IP]	IP-Adress or Host Name where the node is to be installed Default: IP address of your machine
Node port [CLUSTER_NODE_ PORT]	Port for the node Default: `0`
Nodes [CLUSTER_NODES]	IP addresses or host names of the other nodes of the server farm: Default: Host name of your machine

Below you find examples of input files for importing the `create` paramaters via input file:

Example of input file for tenants with derby Database:

```
TENANT_ID            =4011
TENANT_NAME          =RVS_4011
DATABASE_TYPE        =derby
DATABASE_HOST        =
DATABASE_PORT        =
DATABASE_USER        =rvs
DATABASE_PASSWORD    =rvs
DATABASE_NAME        =0
CLUSTER_GROUP_NAME   =rvsEVO6_Cluster
CLUSTER_NODE_NAME    =rvsEVO_Node
CLUSTER_NODE_ID      =0
CLUSTER_NODE_IP      =BE1DY430
CLUSTER_NODE_PORT    =0
CLUSTER_NODES        =
DEFAULTUSER_NAME     =rvs
DEFAULTUSER_PASSWORD =rvs
RMI_HOST             =BE1DY430
RMI_PORT             =4011
LICENSE_FILE         =c:/rvsRT/conf/license.properties
```

Example of input file for tenants with external Database:

```
TENANT_ID            =SQL4011
TENANT_NAME          =SQL4011
DATABASE_TYPE        =mssql
DATABASE_HOST        =<hostname>
DATABASE_PORT        =1433
DATABASE_USER        =dbuser
DATABASE_PASSWORD    =userpw
DATABASE_NAME        =evodb
CLUSTER_GROUP_NAME   =rvsEVO6_Cluster
CLUSTER_NODE_NAME    =rvsEVO_Node
CLUSTER_NODE_ID      =0
CLUSTER_NODE_IP      =10.177.63.220
CLUSTER_NODE_PORT    =2988
```

```
CLUSTER_NODES         =10.177.63.220
DEFAULTUSER_NAME      =rvs
DEFAULTUSER_PASSWORD =rvs
RMI_HOST              =RDDE9D84
RMI_PORT              =3755
LICENSE_FILE         =c:/rvsRT/conf/license.properties
```

6.4.2 Distritution of Licensekey or Station Configuration

Via `rollout` command you can distribute a licensekey to specified rvsEVO tenants or station configurations from one rvs EVO tenant to others.

Hints:

In case of distributing station configurations the rvsEVO servers must be started.

A licencekey can be distributed to tenants on local machine, only.

Syntax:

```
rvsrt [rvsrt options] rollout [--help | -h | -?]
[-l | --license=<license-file>]
[-u | --update-if-exitsts]
[-r | --receiver-change-enabled]
[-n | --network-change-enabled]
[-o | --origin=<tenent-id>]
[-s | --station=<station-ID>] <target-list>
```

Attention: The sequence of the parameters must be followed as given in the syntax: „`rvsrt`“ <rvsrt parameters> <command> <command options> <tenant ID>.

Example for distribution of a license:

```
rvsrt -v rollout -l c:\Daten\license.properties
4001,ora4002,mssql4003
```

Examples for distribution of station configuration:

```
rvsrt -v rollout -s PS1 -o PS4005 -u -r -n
PS4006,PS4001
```

In the example above station „PS1“ is distributed from tenant „PS4005“ to tenants PS4006 and PS4011. Existing stations shall be updated (also network type and receiver number)

```
rvsrt -v rollout -s PS1 -o PS4005 @all
```

In the example above station „PS1“ is distributed from tenant „PS4005“ to all other tenants.

Required `rollout` Parameter:

Parameter	Description
<target-list>	Destination tenant IDs. Possible values: - comma-separated list of tenant IDs - „`@all`“ for all tenants except origin

Optional `rollout` Parameters:

Parameter	Description
-l **--license**	Path and name of license file which is to be distributed
-n **--network-change-enabled**	Enables change of network type of station if station ID already exists. Only if parameter **-u** and **-s** is set.
-o **--origin**	Tenant from which the station configuration is to be read. Only if parameter **-s** is set
-r **--receiver-change-enabled**	Enables change of receiver number of station if station ID already exists. Only if parameter **-u** and **-s** is set.
-s **--station**	ID of the station which is to be distributed
-u **--update-if extists**	Will update station at target in case station already exists. Only if parameter **-s** is set
-h **-?** **--help**	Displays a description of the current command.

6.4.3 Deletion of rvsEVO Tenant

Use the `rvsrt` command `delete` for removing an rvsEVO tenant. The tenant to delete must be in STOPPED or START-FAIL status.

Hints:

A tenant on remote machine can be deleted from instance list, only. Database and filesystem cannot be removed.

On Windows systems: The command prompt has to be opened as administrator.

Syntax:

```
rvsrt [rvsrt options] delete [-c] [-d] [-s] [-h]
<tenant ID>
```

Attention:

The sequence of the parameters must be followed as given in the syntax: „`rvsrt`“ <rvsrt parameters> <command> <command options> <tenant ID>.

In case of using the `delete` command only with <tenant-id> parameter the program guides you through the deletion process step by step:

- First you are asked whether you want to delete the tenant definition. Enter „Yes“ or „y“ to delete the tenant from instance list (`rvsServerInstanceList.xml` file in `$RVS_RUNTIME_HOME/conf` directory) and to move to the next step.
- Subsequent you are asked whether you want to delete the database and filesystem of the tenant.
  Enter „Yes“ or „y“ to finish the deletion of the tenant.

  With external database you may delete filesystem and database content separately.

Required `delete` Parameter:

Parameter	Description
<tenant ID>	ID of the tenant to be removed

Optional `delete` Parameters:

Parameter	Description
-c **--clean-database**	This parameter deletes the database content of the tenant, incl. transfer jobdata
-d **--delete-files**	This parameter deletes the tenant from `rvsServerInstanceList.xml` file in `$RVS_RUNTIME_HOME/conf` directory and the file system of the tenant incl. transfer files
-s **--silent**	This parameter deletes the tenant from `rvsServerInstanceList.xml` file in `$RVS_RUNTIME_HOME/conf` directory without interaction.
-h **-?** **--help**	Displays a description of the current command.

7 Distributed Update

From version 6.05.00 the function **Distributed Update** provides automatical updating of multiple defined rvsEVO partner stations (targets).

The control of this process is handled by one selected rvsEVO station which will be the controller for the distributed update process (Distributed Update Controller). However, we recommend setting up one separate rvsEVO Station to be the controller.

Distributed Update is available for

– simple rvsEVO instances with Derby database on Linux systems

– active part of rvsEVO ACTIVE-PASSIVE backup installation

Distributed Update is not available for

– rvsEVO instances on Windows and UNIX systems

– rvsEVO instances with rvs® OFTP Proxy implementiation

– rvsEVO cluster installations

– passive part of rvsEVO ACTIVE-PASSIVE backup installation

– rvsEVO within container environment

– rvsEVO tenants within rvsRuntime

7.1 Pre-Conditions

The target stations that will be updated must already have version 6.05.00.

Requirements of Target Stations

– The target stations need a license key with included Distributed Update component.

– The Distributed Update Controller must be indicated via **DistributedUpdateController** parameter in group MAINTENANCE.

Necessay Steps of Distributed Update Controller:

– Configuration of `rvs-rest.properties` file (please see chapter 10.5 "Customizing rvsWeb and REST-API" for more information)

– Storage of installer package

– Configuration of target list

– Import of zip-file with license keys of the targets. The licenses need to be issued on Odette ID and must include ’Distributed Update’ component.

Use `processDistributedUpdate` tool in **`$RVS_HOME`**`/system/distribution` directory to prepare and execute Distributed Update.

7.2 processDistributedUpdate Tool

This script offers commands for adding, listing and deletion of target stations, licenses and installer packages and for executing Distributed Update.

Syntax:

```
processDistributedUpdate [-h] [--verbose] <sub-command> <command options>
```

The following sub-commands are available:

- `software-add`: storage of update files for later use
- `software-list`: listing of all stored update files
- `software-delete`: deletion of stored update files
- `target-add`: adding a station to the list of target stations
- `target-list`: displaying list of all stored target stations
- `target-delete`: deletion of a SID from target station list
- `reset-status`: reset the installation status of a target station locally (please find the description of status values in chapter 7.3 "Status Values and NERP Messages of Distributed Update")
- `license-add`: storage of license files for later use
- `license-list`: listing of all stored license files
- `license-display`: displaying list of all stored license files
- `license-delete`: deletion of license files
- `start-update`: starting update of one indicated station, all stations or several stations indicated via input file

Hints:

License, software (package) and target lists can be exported and imported into the rvsEVO database via scripts `exportAllConfiguration` and `importAllConfiguration` (please see rvsEVO User Manual for further information).
The xml files are called `rvsDistributionLicenses.xml`, `rvsDistributionPackages.xml` and `rvsDistributionTargets.xml`.

Update information is written into `rvsevo-distupdate.log` file which is saved in the same directory as updated rvsEVO installation.

In case of rollback log files are saved in **`$RVS_HOME`**`/log/distupdate.<timestamp>` directory.

Parameters:

Parameters	Description
-f <filename>	Path and name of the file Required for `license-add`: zip file containing license files `software-add`: zip file containing update package Optional for `start-update`: input file containing SIDs to be updated (seperated by line feed)

-c	Do not truncate output table
	Available for
	`license-list`
	`software-list`
	`target-list`

-k <hashkey>	Hashkey of update zip files
	Required for
	`software-delete`
	`start-update`
	The hashkey is displayed via `software-list` command

-m	Displays error message in target list in case of unsuccessful update process Available for `target-list`
-s <sid>	Station ID Required for `target-add`: SID to be added to target station list `target-delete`: SID to be deleted from target station list `license-display`: SID associated with license `license-delete`: SID associated with license `start-update`: SID of the target station to be updated (`@all` for all stations)
-t	Sort list by „Status Date Time“ Available for `target-list`
-v <version>	Version of the license Required for `license-delete`: Version of license to be deleted
-h, --help, -?	Requests help information (usage).
--verbose	Verbose message output.

Remote Mode Parameters:

Parameter	Description
-su <user>	Client user for remote login.
-sp <password>	Password of the client
-sh <host:port>	`host` is the IP address or hostname of rvsEVO Server; `port` is the RMI port for client server connection

7.3 Status Values and NERP Messages of Distributed Update

Distributed Update states of target stations are displayed via target list. The status attribute tells about the progress and finally if the installation was successful, or even not. Distributed Update is only executed for target stations in state SUCCESS, ROLLBACK or in case of status is not set. Some status like UPD_REQ_SND, UPD_REQ_ACC are not seen that often, as they may be overwritten by the next step.

States:

– UPD_REQ_SND:Update Request Job was created by controller.

– ERR_UPD_REQ_SND: Error occured on controller-side by creating Update Request Job. Please consider rvs.log.

– UPD_REQ_ACC: The target station has accepted the Update Request from the controller.

– UPD_REQ_REJ: The target station rejected (denied) the Update Request from the controller. Please consider Monitor-Log where description should appear with more details. The process ends here.

– PKG_REQ_REC: Target station has ordered the software package.

– PKG_NOT_FND: Controller can't find the software package to be send to the station. This will lead to stop the process.

– PKG_PROC_ERR (Exception): Controller got an error (exception) during preparation the next step sending the software to the station. Please consider the log file.

– PKG_SND: Controller has created send job with the software package for the station.

– PKG_FIL_REJ: The target station has rejected (denied) this software package due to any negative validation (e.g. wrong version, wrong signature) Please consider the monnitor log message for more details.

– PKG_FIL-ACC: Target station has accepted the software package due positive validation.

– REQ_DENIED: Sender not accepted as Controller

– UPD_REQ_REJ: Different reasons, e.g.

  - DistUp does not support external DB
  - Update version is not greater than current
  - DistUp does not provide Proxy-Station
- Negative Status responses, e.g.
  - NO_DSK_SPC
  - NV_INST
  - NV_JRE
  - JRE_FAIL
  - LIC_INV
  - UPD_FAIL
- UPD_START: Response from the station the update process has been started.
- SUCCESS: Positive response from the station that the installation process has been finished successfully.
- FAILED: Negative response from the station that the installation process cannot be started due an error by creating the backup.
- ROLLBACK: Negative response from the station that the installation process was not successfully and the rollback to the origin version was successfully.
- BLOCKED: Controller has been started again for any station, while another process is in progress for this station. This blocking mechanism should protect the target station from multiple overlapping installations. If status BLOCKED does not disappear after a while, you should analyze if target-station is still in Installation-Modus. If not, you can reset this status on the controller side and try again. Take care about overlapping installations.

Description of NERP Messages:

If you get NERP messages on the controller side from one or more stations, this happens to basic errors, where the station fundamentally denies the processing of the incoming Distributed Update Request.

You may see the NERP messages that contains the following text on the controller:

- SEND_NOT_ACC_AS_CONTR:
  Sender not accepted as controller - the controller is not registered as the controller at the target station. The target station needs to know, who is the controller who is allowed to update the target station. This parameter has to be maintained on the target station (Parameter ->Maintenance->DistributedUpdateController.)
- JSON_CRPT:
  JSON CORRUPT means that the JSON file could not be read or match to the schema of Distributed Update.

© T-Systems

- JSON_HASH_UKN:
  JSON Hash value unknown - means the hash value transferred within the json file could not be assigned to the running process data.
- SYN_ABRT:
  Synchronization abort - means the target station has detected that more than one process tries to update the target station at the same time.
- WRONG-PRE_STATE:
  Wrong predecessor status when trying to update the target station. This happens when the controller tries to send software to the station, but the station has not send a software request before. This is for security.

8 Odette Partner Data eXchange (PDX)

With PDX feature, rvsEVO offers a very simplified function for setting up a connection to a partner station or for updating an existing partner station.

Via outbound process station data and certificates of local and virtual stations, as well as security and connection settings can be exported via rvsEVO script into an XML file (ocs.xml). All the partner has to do, is to import the data of this file into the rvsEVO database (inbound process). No further configuration is required, by default.

Hints:

The content of `ocs.xml` file is described in an XML pattern (**`$RVS_HOME`**`/system/data/xsd/OftpCommunicationSetup-<version>.xsd`)

In chapter 8.3 "Files and Configuration Directories" you find the directory structure relating to PDX

Please see chapter 8.2 "processPdxOutbound and processPdxInbound Tools" for description of `processPdxOutbound` and `processPdxInbound` tools.

8.1 How to Exchange Partner Data

The inbound and outbound processes can be executed offline or online.

To establish a new connection, only the offline method is available. Updating an existing partner station can be executed both, offline and online.

8.1.1 Offline Method

To add a new partner station the following steps are neccessary:

Outbound:

- Adjust the settings in the outbound properties file, only if neccessary.
- Create an `ocs.xml` file via `processPdxOutbound` tool with `create` command:

  Syntax:
  `processPdxOutbound create [command-options]`

  Exported data (`ocs.xml`), used properties file and log file are saved in a sub-directory of `$RVS_HOME/files/pdx/outbound/temp` directory. The name of the folder consists of date, time and counter (e.g. `230404163946-000`).
- Deliver the `ocs.xml` file to your partner (e.g via e-mail).

Inbound:

- The partner receives the `ocs.xml` file
- The receiver of `ocs.xml` file adjusts the inbound properties file, only if necessary

© T-Systems

- The receiver of `ocs.xml` file parses the `ocs.xml` via `processPdxInbound` tool with `parse` command.

  Syntax:
  `processPdxInbound parse [command-options]`

- If parse has been successfully, the receiver of `ocs.xml` file imports the PDX data into the database via `processPdxInbound` tool with `accept` command.

  Syntax:
  `processPdxInbound accept [command-options]`

8.1.2 Online Method

If the connection is already estalished but some settings changed, you can transmit the data via rvsEVO PDX sendjob

Outbound:

- Adjust the settings in the outbound properties file, only if neccessary. Changed parameter values are imported into the data base after EERP is received and scheduled activation time is reached.
- Create a PDX sendjob via `processPdxOutbound` tool with `send` command. The `ocs.xml` file, properties and log files are saved in a sub-directory of `$RVS_HOME/files/pdx/outbound/queued` directory.

  Syntax:
  `processPdxOutbound send [command-options]`

  Hint:
  After EERP is received the files are copied into `$RVS_HOME/files/pdx/outbound/stored` directory.

  In case of NERP is received the files are copied into `$RVS_HOME/files/pdx/outbound/error` directory.

Inbound:

- The partner receives the `ocs.xml` file via rvsEVO PDX receive job. rvsEVO parses the `ocs.xml` automatically.
  If parse has been successfully, the job stays in catagory **ACTIVE** with status **PDX_HELD** and monitor messages **PDX_PARSE** and **PDX_NEEDS_PROCESSING** are written. The files are saved in `$RVS_HOME/files/pdx/inbound/inbox` directory.
  If parse fails the PDX file is rejected automatically, monitor message **PDX_PROCESSING_FAILED** is written and the job is deleted with NERP. The files are saved in `$RVS_HOME/files/pdx/inbound/error` directory.

  If parse has been successfully:

- The receiver of `ocs.xml` file adjusts the inbound properties file, only if neccessary.

- The receiver can parse the `ocs.xml` file via `processPdxInbound` tool with `parse` command again.

  Syntax:
  `processPdxInbound parse [command-options]`

- The receiver accepts the `ocs.xml` file via `processPdxInbound` tool with `accept` command. The data are imported into the database after EERP is sent and scheduled activation time is reached. The files are copied into `$RVS_HOME/files/pdx/inbound/stored` directory.

  Syntax:
  `processPdxInbound accept [command-options]`

- If the receiver doesn‘t except the PDX file, he rejects the file via `processPdxInbound` tool with `reject` command. The files are copied into `$RVS_HOME/files/pdx/inbound/error` directory.

  Syntax:
  `processPdxInbound reject [command-options]`

Hint:

Company infos and contact data are not imported into the database but saved in `CompanyInfoAndContacts.txt` file.

8.2 processPdxOutbound and processPdxInbound Tools

8.2.1 processPdxOutbound: Export of Station Data

Use `processPdxOutbound` tool to export data from your database. This function is implemented via `create` and `send` command.

– `create`: available for new and existing partner stations. The `ocs.xml` file has to be delivered manually to your partner.

– `send`: available for existing partner stations. The `ocs.xml` file is transmitted automatically via PDX send job of rvsEVO.

Syntax:

`processPdxOutbound [-h] <command> [parameters]`

Required Parameter:

Parameter	Description
-d <destination>	Only for existing station: SID of the receiver of the `ocs.xml` file (alternative to **--new-partner**)
--new-partner	alternative to **-d**, if so far the receiver isn‘t added to your stationlist.

Optionale Parameters:

Parameters	Description
-l <virtual station>	SID of the virtual Station (more than one stations must be seperated by comma)
-p <properties file>	Path and name of the properties file with specific settings. Please see chapter 8.3.2 "Properties Files" for further information.
-r, --report	Writes PDX report in console
-s, --sign	Available for `send` command: `ocs.xml` file is signed before transmission
--valid-from <date-time>	Only for `send` command: Defines the timepoint in format `yyMMddHHmmss` for activation of new settings. If the timepoint is exceeded and `ocs.xml` file has been accepted the data are imported immediately into the database.
-x <xsd-version>	Version of XML schema
h, --help, -?	Requests help information (usage).
--verbose	Verbose message output.

Remote Mode Parameters:

Parameter	Description
-su <user>	Client user for remote login.
-sp <password>	Password of the client
-sh <host:port>	`host` is the IP address or hostname of rvsEVO Server; `port` is the RMI port for client server connection

Examples:

– `processPdxOutbound` with command `create`:

```
processPdxOutbound create --new-partner -p
c:\rvsEVO\conf\pdx\pdx-outbound-special.properties
-r
```

In example above a `ocs.xml` file is created for a new partner station. The basic-settings are overwritten with the values defined in `c:\rvsEVO\conf\pdx\pdx-outbound-special.properties` file. The report is written into the console.

– `processPdxOutbound` with command `send`:

```
processPdxOutbound send -d PARTNER -l VIRTUAL-SID
--valid-from 250410080000 -p
c:\rvsEVO\conf\pdx\pdx-outbound-special.properties
-s
```

In example above a `ocs.xml` is created for local station and virtual station `VIRTUAL-SID`. The file is signed and transmitted to the partner with SID `PARTNER`. The basic settings are overwritten with the values defined in `c:\rvsEVO\conf\pdx\pdx-outbound-special.properties` file and scheduled activation time is 25-04-10 8:00:00.

8.2.2 processPdxInbound: Managing Incoming PDX Files

Use `processPdxInbound` tool to parse, accept or reject a received PDX file (reject is only available for online processes).

Syntax:

```
processPdxInbound [-h] <command> [parameters]
```

parse Command

During parse process the `ocs.xml` file is verified and a success or error message is returned.

Syntax:

```
processPdxInbound parse -f <filename> | -n <jobId>
[--allow-gateway-change] [--create-substations]
[-g <gateway>] [-p <preset-properties>] [-r]
[-s <sid>] [-h] [--verbose] [-su <user> -sp <password>
-sh <host:port>]
```

Example:

```
processPdxInbound parse -f
c:\rvsEVO\archive\pdx\temp\inbound\250405094158-
000\pdx.xml -s TESTSTATION --create-substations
```

In example above a new partner station is to be created with SID TEST-STATION. The `ocs.xml` file is indicated via parameter **-f** and creation of routed station is allowed.

accept Command

Use `accept` command to import the `ocs.xml` data into rvsEVO database. The data will only be imported after successful parse which is executed automatically by accept command.

Syntax:

```
processPdxInbound accept -f <filename> | -n <jobId>
[--allow-gateway-change] [--create-substations]
[-g <gateway>] [-p <preset-properties>] [-r]
[-s <sid>] [-h] [--verbose] [-su <user> -sp <password>
-sh <host:port>]
```

Example:

```
processPdxInbound accept -f
c:\rvsEVO\archive\pdx\temp\inbound\250405095958-
000\pdx.xml -g 20003 --allow-gateway-change
```

In example above existing stations should be adjusted. The `ocs.xml` file is indicated with **-f** parameter, main and substation are to be routed via gateway 20003.

reject Command

Use `reject` command if you don‘t accept the `ocs.xml` file. `reject` is only available for receive jobs in state **PDX_HELD**.

Syntax:

```
processPdxInbound reject -n <jobId> [-m <NERP
message>] [-h] [--verbose] [-su <user> -sp <password>
-sh <host:port>]
```

Example:

```
processPdxInbound reject -n 2504051010210000 -m
„Insufficient security settings“
```

In example above the `ocs.xml` file received with job ID `2504051010210000` is to be rejected. With outgoing NERP message `„Insufficient security settings“` is transmitted to the originator.

Required Parameters:

Parameter	Description
-f <filename>	Requiered for Inbound offline processes; Path and name of PDX file
-n <jobID>	Required for Inbound online processes; JobID of PDX receive job

Optional Parameters:

Parameter	Description
--allow-gateway-change	Allows change of gateway for updated station
--create-substations	Allows import of substations as routed stations
-g <gateway>	SID of existing station that acts as gateway for all stations to be imported
-m <NERP message>	only for `reject`: Message for NERP reponse
-p <properties file>	Path and filename of preset mapping properties file

Optional Parameters:

Parameter	Description
-r, --report	Writes PDX report in console
-s <sid>	Only available for creation of new stations: SID for the PDX main station to be imported into the database
-h, --help -?	Requests help information.
--verbose	Verbose message output.

Optionale Remote-Parameter:

Parameter	Description
-su <user>	Client user for remote login.
-sp <password>	Password of the client
-sh <host:port>	`host` is the IP address or hostname of rvsEVO Server; `port` is the RMI port for client server connection

8.3 Files and Configuration Directories

8.3.1 PDX Files

The files, created during PDX process are saved in sub-directories of **`$RVS_HOME`**`/files/pdx`:

– **`$RVS_HOME`**`/files/pdx/inbound` (sub-directories and files, created with inbound process):
  – `error` (PDX data of failed processes)
  – `inbox` (PDX data received online, as long as receive job is in status **PDX_HELD**)
  – `queued` (PDX data with scheduled time in future)
  – `stored` (activated PDX data)
  – `temp` (temporary files, saved via `parse` command)

  Files of inbound process:
  – `CompanyInfoAndContacts.txt`
  – `InboundActivateTaskParameter.properties` (PDX activation information, e.g. job-ID, processPdxInbound.sh parameters ...)
  – `ocs.xml` (exchange data)
  – `report.log` (mapping information of all relevant data)

- **$RVS_HOME**/files/pdx/outbound (sub-directories and files, created with outbound process):
  - error (PDX data of failed processes)
  - queued (PDX data with scheduled time in future and PDX data created via PDX online process)
  - stored (activated PDX data)
  - temp (temporary files saved via create command)

  Files of outbound process:
  - ocs.xml (exchange data)
  - OutboundActivateTaskParameter.properties (PDX activation information, e.g. job-ID, processPdxInbound.sh parameters ...)
  - pdxOutbound.properties (indicated properties file with outbound process)
  - report.log (mapping information of all relevant data)

Hint: Via MAINTENANCE parameter **DeleteTempPdxOlderDays** you can indicate the interval in days for deletion of files in temp directory.

8.3.2 Properties Files

The configuration files are saved in **$RVS_HOME**/conf/pdx directory:

- PdxOutboundDefault.properties
- PdxInboundDefault.properties

The default properties files are only applied in case of connecting a new partner and no specific properties file is indicated via parameter **-p**.

You can adjust the default properties files or add own properties files with special settings.

Hint: The files must be saved in UTF-8 format

Via properties files e.g. the file security parameters can be customized.

In case of setting outbound properties with online process, reverse mapping takes place. That means that the values, defined via properties file, are also imported into the database of the sender after the EERP is received.

In case of setting outbound properties with offline process or setting inbound properties the defined values are only imported into the database of the receiver.

9 rvsEVO within Container Environment

Since version 6.04.00 rvsEVO is also runable in container environment.

In this chapter deployment of rvsEVO into Docker container and update of rvsEVO within Docker container is described. If you are using another tool for container orchestration (e.g. Openshift or Kubernetes), please proceed analogously in your environment.

Requirements

Correctly installed Docker

Good knowledge about Docker

Good knowledge of administration of your operating system.

9.1 User and Credentials and Mounted Directories

For reasons of security rvsEVO container will not run as root user. Instead rvsEVO container comes with a pre-defined user ID (uid) = 2019 with group ID (gid) = 2019.

Since there are mounted directories outside the container the 2019 user needs at least read and write access to those directories. In Unix-Environment the best practice is to create a user with the uid = 2019 and gid = 2019 and assign this user to have access to the mounted directories.

However, in some cases it might be possible that the uid / gid = 2019 is already occupied by another user. In this case there are at least two options to handle it.

– Subuser mapping

– Changing the uid and create new image

Subuser-Mapping

Docker provides a way to map the user ID inside the container to a uid according to your wishes. In other words it is possible to map the `uid=2019` from inside the container to a uid=xxx outside the container, here called userns-remap.

Please read the documentation of Docker and other how to handle this:

`https://docs.docker.com/engine/security/userns-remap/`

Changing uid:gid and Recreate Image

To change the uid you need to run one container and step into the container with root-user (`u=0:0`).

Inside the container you need to execute these Linux commands:

```
usermod -u <uid> rvsuser

groupmod -g <gid> rvsuser

chown -R <uid>:<gid> /home/rvsuser
```

After that you need to create a new image from that container for further usage.

```
docker commit <CONTAINER-ID> <IMAGE-NAME>
```

Example:

```
docker commit 964ea42a2dba rvsevo.60400.modified-rvsuser
```

9.2 How to Deploy and Run rvsEVO into Docker Container

Hint: If rvsEVO is deployed into a Docker Container the installation is always configured with Derby database. If an external database is to be used, execute the script **`$RVS_HOME`**`/tools/switchDatabase` after deployment.

Please follow the instructions below for deployment of rvsEVO into Docker container.

Pull rvsEVO image

- Pull the public rvsEVO Image from the MTR-Registry (MTR=Magenta Trusted Registry)

  Syntax:

  ```
  docker pull [OPTIONS] NAME[:TAG|@DIGEST]
  ```

  Example for rvsEVO 6.04.00:

  ```
  docker pull mtr.devops.telekom.de/rvs-evo-docker/rvsevo.60400
  ```

- Check if image download succeeded:

  ```
  docker images
  ```

Create directories

- Create a working directory from where you like to start your work.

  Example:

  ```
  mkdir /home/dockerhost/rvsEVO
  ```

- Next step is to create some folders (also called Docker volumes) on the host to store data outside the running container so that files are persisted even when the container has been removed. We will later, when we call the execute command, associate each folder with its corresponding folder in the container.

  Create the sub-directories `rvsarchive, rvsconf, rvsdb, rvsfiles` and `rvslog` in your working-directory:

  Example:

  ```
  mkdir /home/dockerhost/rvsEVO/rvsarchive
  mkdir /home/dockerhost/rvsEVO/rvsconf
  mkdir /home/dockerhost/rvsEVO/rvsdb
  mkdir /home/dockerhost/rvsEVO/rvsfiles
  mkdir /home/dockerhost/rvsEVO/rvslog
  ```

Start rvsEVO container

- Start an rvsEVO container in interactive mode for initial configuration.

  Hints:

  Replace the values in the brackets with your values.

If you don't use "named volumes" you need to enter absolute paths for the host, as we do. So e.g. <rvsdb_folder> should be replaced by an absolute path.

Syntax:

```
docker run -it \
--hostname <EXTERNAL_VISIBLE_HOSTNAME> \
  -p <DOCKERHOST_IP>:<DOCKERHOST_OFTP_PORT>:3305 \
  -p <DOCKERHOST_IP>:<DOCKERHOST_RMI_PORT>
  :<DOCKERHOST_RMI_PORT> \
  -v <rvsarchive_FOLDER_ON_DOCKERHOST>:/home/rvsuser/
  rvsEVO/archive \
  -v <rvsconf_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/conf \
  -v <rvsdb_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/db \
  -v <rvsfiles_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/files \
   -v <rvslog_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/log \
   <IMAGENAME>:latest \
   bash
```

Example:

```
docker run -it \
--hostname q4de3esy217.gdc-bln01.t-systems.com \
-p 10.33.101.197:14305:3305 \
-p 10.33.101.197:14755:14755 \
-v /home/dockerhost/rvsEVO/rvsarchive:/home/rvsuser/rvsEVO/archive \
-v /home/dockerhost/rvsEVO/rvsconf:/home/rvsuser/rvsEVO/conf \
-v /home/dockerhost/rvsEVO/rvsdb:/home/rvsuser/rvsEVO/db \
-v /home/dockerhost/rvsEVO/rvsfiles:/home/rvsuser/rvsEVO/files \
-v /home/dockerhost/rvsEVO/rvslog:/home/rvsuser/rvsEVO/log \
mtr.devops.telekom.de/rvs-evo-docker/rvsevo.60400:latest \
bash
```

Now you have a bash shell inside the container.

- If you need to change uid / gid, please proceed as describt in section 9.1 "User and Credentials and Mounted Directories"/ "Changing uid:gid and Recreate Image". (First you have to leave the container)

Initialize configuration

- It is expected, that for the very first intallation the mounted directories `rvsdb, rvsconf` and `rvsfiles` are completely empty. Please check this before you continue.
- Execute the `configure` script to copy and fill the mounted directories with its init-data files.

  `/home/rvsuser/rvsEVO/init/configure.sh`

- Next we need to edit the `rvsEvoClient.prefs` and `rvs-api-rmi.properties` files which are placed in container folder `/home/rvsuser/rvsEVO/conf`. You may use vi-editor for that.

  In addition you may edit those files outside the container, since these volumes are mounted.

- Customize the files below as described:

  `rvsEvoClient.prefs`

  - set the value of "client.RMIServiceHost" to EXTERNAL_VISIBLE_HOSTNAME
  - set the value of "client.RMIServiceHost" to DOCKERHOST_RMI_PORT

  `rvs-api-rmi.properties`

  - set the value of "rvs.api.rmi.service.host“ EXTERNAL_VISIBLE_HOSTNAME
  - set the value of "rvs.api.rmi.service.port“ to DOCKERHOST_RMI_PORT

  Hint: If you want to use an external database you can execute **`$RVS_HOME`**`/tools/switchDatabase` now from inside the container.

9.2.1 Test of rvsEVO Server Start within the Init Container

Finally test the startup of rvsEVO server and the connection of remote clients.

Hint: Before you start the rvsEVO server you need to place the `license.properties` file to your mounted `rvsconf` directory.

- Start the rvsEVO server via `/home/rvsuser/rvsEVO/bin/startServer.sh` tool within the init container
- After the rvsEVO server was started successfully, start the rvsEVO GUI from remote computer. (Please read the rvsEVO User Manual, chapter „Graphical User Interface“/“Remote GUI“„ for information about Remote GUI)

  The following specifications are required:

  User: `rvs`

  Password: `rvs`

  Server Host: `<EXTERNAL-VISIBLE-HOSTNAME>:<DOCKERHOST_RMI_PORT>`
- After test is successfully completed stop the rvsEVO server with **CTRL-C** and leave the bash shell inside of the container via `exit` command.

- Now the init container can be removed:

  `docker rm <CONTAINER-ID>`

  Hint: The container ID is displayed via `docker ps -al` command

9.2.2 Start of Productive rvsEVO Server Container

Create and start a new container with the command below. Please replace the placeholders (bold with brackets) with the real values.

Syntax:

```
docker run -it -d \
   --hostname <EXTERNAL_VISIBLE_HOSTNAME> \
     -p <DOCKERHOST_IP>:<DOCKERHOST_OFTP_PORT>:3305 \
     -p <DOCKERHOST_IP>:<DOCKERHOST_RMI_PORT>
     :<DOCKERHOST_RMI_PORT> \
     -v <rvsarchive_FOLDER_ON_DOCKERHOST>:/home/rvsuser/
     rvsEVO/archive \
     -v <rvsconf_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/conf \
     -v <rvsdb_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/db \
     -v <rvsfiles_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/files \
      -v <rvslog_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/log \
       <IMAGENAME>:latest \
       /home/rvsuser/rvsEVO/bin/startServer.sh
```

Example for rvsEVO 6.04.00:

```
docker run -it -d \
  --hostname q4de3esy217.gdc-bln01.t-systems.com \
  -p 10.33.101.197:14305:3305 \
  -p 10.33.101.197:14755:14755 \
  -v /home/dockerhost/rvsEVO/rvsarchive:/home/rvsuser/rvsEVO/archive \
  -v /home/dockerhost/rvsEVO/rvsconf:/home/rvsuser/rvsEVO/conf \
  -v /home/dockerhost/rvsEVO/rvsdb:/home/rvsuser/rvsEVO/db \
  -v /home/dockerhost/rvsEVO/rvsfiles:/home/rvsuser/rvsEVO/files \
  -v /home/dockerhost/rvsEVO/rvslog:/home/rvsuser/rvsEVO/log \
  mtr.devops.telekom.de/rvs-evo-docker/rvsevo.60400:latest \
/home/rvsuser/rvsEVO/bin/startServer.sh
```

9.3 Update of rvsEVO in Docker Container

In this chapter the update procedure of rvsEVO within Docker container is described. All examples refer to update from rvsEVO 6.03.03 to rvsEVO 6.04.00.

The steps below are necessary for the update:

- Download the rvsEVO image
- Open the bash shell inside the container

- Stop rvsEVO inside the container
- Save the rvsEVO job and configuration data outside the container
- Remove the container
- Create a new container in interactive mode
- Execute `update --backup-data` script inside the container
- Exit and remove the container
- Create a new container with downloaded image in interactive mode
- Execute `update --restore-data` script inside the container
- Exit and remove the container
- Create your productive rvsEVO Server container

Detailed Description of Update Procedure:

Pull rvsEVO image

- Pull the public rvsEVO image as described in chapter 9.2 "How to Deploy and Run rvsEVO into Docker Container"

Save rvsEVO data

- First the rvsEVO server has to be stopped inside the container. Use the following command to go into the container:

  ```
  docker exec -it <container-id> bash
  ```

  Use the following command to stop the rvsEVO server:

  ```
  /home/rvsuser/rvsEVO/bin/stopServer.sh
  ```

  The container is stopping automatically.
- Save the rvs directories mounted on the host outside the container:
  - `rvsarchive`
  - `rvsconf`
  - `rvsdb`
  - `rvsfiles`
  - `rvslog`
- Export job and configuration data from database.

  Therefor a container has to be started in interactive mode.

  Example:

  ```
  docker run -it \
  --hostname q4de3esy217.gdc-bln01.t-systems.com \
  -p 10.33.101.197:14305:3305 \
  -p 10.33.101.197:14755:14755 \
  -v /home/dockerhost/rvsEVO/rvsarchive:/home/rvsuser/rvsEVO/archive \
  -v /home/dockerhost/rvsEVO/rvsconf:/home/rvsuser/rvsEVO/conf \
  -v /home/dockerhost/rvsEVO/rvsdb:/home/rvsuser/rvsEVO/db \
  -v /home/dockerhost/rvsEVO/rvsfiles:/home/rvsuser/rvsEVO/files \
  ```

```
-v /home/dockerhost/rvsEVO/rvslog:/home/rvsuser/rvsEVO/log \
mtr.devops.telekom.de/rvs-evo-docker/rvsevo.60303:latest \
   bash
```

Now you have a bash inside the container and can backup your rvsEVO data with the following script:

`/home/rvsuser/rvsEVO/tools/update.sh --backup-data`

The backup data are saved into mounted directory `/home/rvsuser/rvsEVO/rvsarchive/backup-for-update` on host.

Hint: If the backup fails please check the user data of `rvsarchive` folder. Read and write rights must be assigned to rvsuser.

Important: Don‘t save other documents in `/home/rvsuser/rvsEVO/rvsarchive/backup-for-update` directory, because the content is removed with next update.

Start the backup again after the user rights were changed.

Leave the container via `exit` command.

Remove the container via `docker rm <container-id>` command.

Restore configuration data

At first you have to create and start a container from the new image `mtr.devops.telekom.de/rvs-evo-docker/rvsevo.060400`

- Start the container in interactive mode as described in chapter 9.2 "How to Deploy and Run rvsEVO into Docker Container" page 56.
- Now you have a bash inside the container and can restore the configuration data from your backup via the script below: `/home/rvsuser/rvsEVO/tools/update.sh --restore-data`
- If the restore has been finished successfully you can leave the container with `exit` command and remove the container via `docker rm <container-id>` command.

Start rvsEVO server container

- Start your productive rvsEVO container as described in chapter 9.2.2 "Start of Productive rvsEVO Server Container".

You can check if the container has been started successfully via `docker ps` command.

9.4 Server Farm of rvsEVO within Container Environment

Please read chapter 5 "Server Farm of rvsEVO" as well as 9, 9.1, 9.2 before you start to set up of a cluster node into Docker Container.

9.4.1 Creating a Cluster Node of rvsEVO Server Farm

Proceed as described in chapter 9.2. and consider the following aberrations:

- Create a working directory as described in chapter 9.2.
  Hint: rvsfiles directory (and if applicable observer directory and jobstart scripts) must be located on your NFS.
- Create and start an rvsEVO container in interactive mode with addional row for mapping the cluster node.

  Syntax:

```
docker run -it \
--hostname <EXTERNAL_VISIBLE_HOSTNAME> \
  -p <DOCKERHOST_IP>:<DOCKERHOST_OFTP_PORT>:3305 \
  -p <DOCKERHOST_IP>:<DOCKERHOST_RMI_PORT>
  :<DOCKERHOST_RMI_PORT> \
  -p <DOCKERHOST_IP>:<DOCKERHOST_CKUSTER_NODE_PORT>
  -v <rvsarchive_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/archive \
  -v <rvsconf_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/conf \
  -v <rvsdb_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/db \
   -v <rvsfiles_FOLDER_ON_NFS>:/home/rvsuser/rvsEVO/files \
   -v <observer_FOLDER_ON_NFS>:/home/rvsuser/rvsEVO/observer
   -v <rvslog_FOLDER_ON_DOCKERHOST>:/home/rvsuser/rvsEVO/log \
    <IMAGENAME>:latest \
•    bash
```

  Example:

```
docker run -it -d \
--hostname q4de3esy217.gdc-bln01.t-systems.com \
 -p 10.33.101.197:14305:3305 \
 -p 10.33.101.197:14755:14755 \
 -p 10.33.101.197:5900:5900 \
-v /home/dockerhost/rvsEVO/rvsarchive:/home/rvsuser/rvsEVO/archive \
 -v /home/dockerhost/rvsEVO/rvsconf:/home/rvsuser/rvsEVO/conf \
 -v /home/dockerhost/rvsEVO/rvsdb:/home/rvsuser/rvsEVO/db \
 -v /tbshare/rvsEVO/rvsfiles:/home/rvsuser/rvsEVO/files \
 -v /tbshare/rvsEVO/observer:/home/rvsuser/rvsEVO/observer \
 -v /home/dockerhost/rvsEVO/rvslog:/home/rvsuser/rvsEVO/log \
 mtr.devops.telekom.de/rvs-evo-docker/rvsevo.60400:latest \
 bash
```

- Change uid / gid if required and create a new image from the container
- Set user rights for `/home/rvsuser/rvsEVO/init/configure.sh` script and execute the script.
- Export the configuration data
- Edit the configuration files
- Import the modified files
- Start rvsEVO inside the container

- Now some configurations for the cluster node must be done:
  - Set the alias name of the IP address of rvsEVO server in `$RVS_HOME/conf/host-alias.properties` file.
  - Configure the remote API-RMI access by setting **rvs.api.rmi.service.host** parameter in **`$RVS_HOME`**`/conf/rvs-api-rmi.properties` file to the alias name.
  - Set the IP Address of the rvsEVO listeners to the alias name (e.g. via GUI or rvscli script).
  - Configure the `cluster.properties` file as described in chapter 5.4 "Setting up of a Server Farm".
  - Stop rvsEVO Server inside the container and switch your database to Oracal or MSSQL via `$RVS_HOME/tools/switchDatabase` script.

    Important: Execute `switchDatabase` tool with `-switchonly` parameter, if this is not the first node of your cluster.
  - Replace "`standalone`" by "`cluster`" in element "**spring.profiles.active**" of `$RVS_HOME/conf/rvs.properties` file.
- Stop and remove the container.
- Create and start a new container from the image which was created from your container.

  Example:

```
docker run -it -d \
--hostname q4de3esy217.gdc-bln01.t-systems.com \
-p 10.33.101.197:14305:3305 \
-p 10.33.101.197:14755:14755 \
-p 10.33.101.197:5900:5900 \
-v /tbshare/rvsEVO/rvsarchive:/home/rvsuser/rvsEVO/archive \
-v /home/dockerhost/rvsEVO/rvsconf:/home/rvsuser/rvsEVO/conf \
-v /home/dockerhost/rvsEVO/rvsdb:/home/rvsuser/rvsEVO/db \
-v /tbshare/rvsEVO/rvsfiles:/home/rvsuser/rvsEVO/files \
-v /tbshare/rvsEVO/observer:/home/rvsuser/rvsEVO/observer \
-v /home/dockerhost/rvsEVO/rvslog:/home/rvsuser/rvsEVO/log \
new.image/rvs-evo-docker/rvsevo.60400:latest \
/home/rvsuser/rvsEVO/bin/startServer.sh
```

9.4.2 Update Server Farm of rvsEVO within Container Environment

Please go on as described in chapter 9.3 "Update of rvsEVO in Docker Container" and consider the information given in chapter 5.5 "Update of rvsEVO Server Farm".

Important:

Update the database structure with updating the last node. By default the database structure is updated via `$RVS_HOME/tools/update` script. The script must be executed with parameter **skip-db,** if not the last node is updated.

Syntax:

```
update --backup-data | --restore-data [-c --skip-db]
```

Required Parameters:

Parameter	Description
--backup-data	Create backup data. The data are saved in `/home/rvsuser/rvsEVO/rvsarchive/backup-for-update` directory. Don't save other documents in this directory, because the content is removed with next update. (Use only **backup-data** or **restore-data**)
--restore-data	Restore backup data (use only **restore-data** or **backup-data**)

Optional Parameters:

Parameter	Description
-c, --clean	Delete existing `$RVS_HOME/backup-for-update` directory before backup data.
--skip-db	Skips export and import of database tables.
-h, --help, -?	Requests help information.

9.5 rvs® OFTP Proxy within Container Environment

In this chapter necessary preparations and the structure of the Docker command for starting rvs® OFTP Proxy in a Docker container are described.

Preparations

9.5.1 Mounting the Configuration and Log Directories

To store the configuration and log files outside the container, you should provide one directory for the configuration files and one for the log files. The files created by rvs® OFTP Proxy in these directories during starts-up have user ID 2019 as their owner and group ID 2019 as their group.

Changing the UID

For security reasons, the root user is not used in the container by default. Instead, a user with the UID 2019 is used.

If this user ID does not meet your requirements and name space mapping is also not an option, you can customize the UID via the usual Docker methods and create a new image.

To do this, proceed as follows:

- Create a container based on the existing image
- Use root privileges to change the owners of the relevant files (chown).
- Transfer the modified container to a new image, which can then be used as a new base image.

Hint: When committing the new image, it is important to set the entry point correctly so that the rvs® OFTP Proxy starts automatically.

Syntax:

```
docker commit --change='ENTRYPOINT ["/bin/bash",
"/home/rvsuser/rvsProxy/bin/entryscript.sh"]'
<Container-ID> <NEW_IMAGE_NAME>
```

9.5.2 Setting up an RMI Communication via TLS Protocol

If you want to set up TLS encryption between rvs® OFTP Proxy and your connected rvsEVO station, you must first generate the necessary key artifacts.

Hint: Please see the rvs® OFTP Proxy User Manual, chapter „Setting up of a TLS Connection“ for detailed information about encryption between rvs® OFTP Proxy and rvsEVO.

The key artifacts must be generated with the `keytool` script provided by rvs® OFTP Proxy. The `keytool` script is located in the rvs® OFTP Proxy container.

You can temporarily start the container to execute the `keytool` script with the required parameters. The proxy process is not started yet, as the **entrypoint** parameter (Start of rvs® OFTP Proxy) is deactivated.

The key artifacts must be generated in a mounted directory so that they are backed up outside the container. This can already be the conf directory provided for mounting.

The Docker command for generating the key artifacts has the following format:

```
docker run \
  -v <CONF_VOLUME>:/home/rvsuser/rvsProxy/conf \
  --entrypoint "" \
  mtr.devops.telekom.de/rvs-evo-docker/rvsproxy.70000.beta2:latest \
  bash /home/rvsuser/rvsProxy/bin/keytool.sh \
  -g \
  -k /home/rvsuser/rvsProxy/conf/keystorecustom.p12 \
  -t /home/rvsuser/rvsProxy/conf/truststorcustom.p12 \
  -p <PASSWORD_KEYSTORE> \
```

```
  -vt <YYYY-MM-DD>
```

Description:

`-v <CONF_VOLUME>:/home/rvsuser/rvsProxy/conf:` Mounts the configuration directory from the host into the container.

`--entrypoint ""`: Disables the default start script.

Thereafter the `keytool` script is called with the following parameters:

keytool Parameters:

Parameter	Description
-g	generate
-k <path>	Name and path of the keystore (p12 file) to the mounted directory
-t <path>	Name and path of the truststore (p12 file) to the mounted directory
-p <password>	Password of the keystore
-vt <date>	Expiration date of the keystore. Format: YYYY-MM-DD

After the key artifacts have been successfully generated you can delete the container.

In case of problems you can analyse the logs using command `docker logs <CONTAINER_ID>`

Starting rvs® OFTP Proxy

9.5.3 Docker Command (Template)

The Docker command for starting rvs® OFTP Proxy has the following format:

```
docker run -d --name evoproxy --hostname rvsproxy \
-p <HOST_IP>:<RMI_PORT>:3890 \
-p <HOST_IP>:<OFTP_PORT>:6619 \
-e EXT_HOSTNAME=<HOST_IP> \
-e RMI_PORT=<RMI_PORT> \
-e SERVICE_NAME=<SERVICE_NAME> \
-e TLS=<TRUE|FALSE> \
-e KN=<PATH-OF-KEYFILE> \
-e KP=<ENCRYPTED-PASSWORD-OF-KEYFILE > \
-v <CONF_VOLUME>:/home/rvsuser/rvsProxy/conf \
-v <LOG_VOLUME>:/home/rvsuser/rvsProxy/log \
mtr.devops.telekom.de/rvs-evo-docker/rvsproxy.70000:latest
```

Docker Command Parameters:

Parameters	Description
-p <HOST_IP>:<PORT>: <DESTINATION_PORT>	Defines the IP address, external port, and internal destination port of the container through which it is accessible for OFTP communication with other rvsEVO stations
-p <HOST_IP>:<PORT>: <DESTINATION_PORT>	Defines the address of the container and port-forwarding for RMI communication via Docker. Incoming network traffic on <HOST_IP>:<PORT> is forwarded to the internal port <DESTINATION_PORT> within the container. **Important:** For RMI, the external port (<PORT>) and the internal destination port (<DESTINATION_PORT>) must be identical.
-e EXT_HOSTNAME	Hostname / IP address under which the container is accessible. (For Kubernetes, this corresponds to the DNS name of the service.)
-e RMI_PORT	Port for the RMI connection. Must be identical in the port mapping. Identical to the RMI destination port
-e SERVICE_NAME	Freely selectable name for this communication connection. Serves as namespace encapsulation for this communication between rvsEVO and rvs® OFTP Proxy
-e TLS	(optional) If set to `TRUE`, TLS is enabled. Then **KN** and **KP** must also be specified
-e KN	Path and file name of the key management file
-e KP	Encrypted password of the key management file
-v <CONF_VOLUME>	Defines how the directory `/home/rvsuser/rvsProxy/conf` is mounted externally

Placeholder and Examples:

Placeholder	Example	Description
<HOST_IP>	`10.33.101.135`	IPadress / hostname
<RMI_PORT>	`3890`	Port for RMI communication
<OFTP_PORT>	`6619`	Port for OFTP communication

Placeholder	Example	Description
<SERVICE_NAME>	`docker_proxy_service`	Proxy service name (must be unique)
<TLS>	`TRUE`	Possible values: - `TRUE`: enables TLS - `FALSE`: disables TLS
<KN>	`/home/rvsuser/rvsProxy/conf/proxy-ssl.p12`	Name and path of the key-store within the container
<KP>	`VWUv3Un7tVQIwk-KIsoy646q73x9ElxVG`	Encrypted keystore password
<CONF_VOLUME>	`/home/mounted/volumes/hostvolume/conf`	Configuration folder on host
<LOG_VOLUME>	`/home/mounted/volumes/hostvolume/log`	log folder on host

Additional Hints on the Start Command:

– As already mentioned in the parameter description: For RMI communication, make sure that the source and destination ports are identical when port mapping or port forwarding (e.g., port 3890 to port 3890).

– The key management file defined by **-e KN** may also be located in `/home/rvsuser/rvsProxy/conf` directory at startup. However, it is also possible to mount a directory that does not yet exist in the container, e.g., `/home/rvsuser/rvsProxy/keyfile`.

– Port mapping for OFTP communication allows different ports for external and internal use.

– The entry point for executing the start script is defined in the image and does not need to be specified here.

10 Customizing rvsEVO

In this chapter you find information about configuration beyond standard configuration via rvsEVO GUI.

10.1 Customizing the RMI Parameters

RMI (Remote Method Invocation) is a protocol for the internal process communication in Java. Since rvsEVO 6.05.07 the RMI parameters are saved in `$RVS_HOME/conf/rvs-api-rmi.properties` file.

The **RMI** parameters corresponds to the **BASIC** parameters, which were described in rvsEVO User Manual, chapter „Customizing the rvsEVO Parameters“ in previous versions.

Hint: Path and names of `$RVS_HOME/conf/host.allow` and `host.deny` file are fix now.

Below you find the description of **RMI** parameters:

- `rvs.api.rmi.service.name`

  Name of the rvsEVO service in the RMI registry. Default: rvsEVO

- `rvs.api.rmi.service.host`

  Host Name in the RMI registry. Default: `localhost`.

- `rvs.api.rmi.service.port`

  Port number for the communication in the RMI registry. Default: `3755`

  Hint: In case of changing the Port please adjust the **RMIServicePort** in `$RVS_HOME/conf/rvsEvoClient.prefs` file as well.

- `rvs.api.rmi.tls.enabled`

  Default: `false`
  If set to `true` secure RMI communication over TLS is enabled on server side. Please see also parameters `rvs.api.rmi.tls.keystore.filename` and `rvs.api.rmi.tls.keystore.password`.
  Please read chapter 10.6 "Customizing RMI Connection over TLS" for more information about RMI over TLS.

- `rvs.api.rmi.tls.keystore.filename`
  Default: `/home/evotest/kde/rvsevorout/conf/api-rmi-tls-keystore.p12`

  Name of the keystore file used for RMI over TLS communication. Please see also parameters `rvs.api.rmi.tls.enabled` and `rvs.api.rmi.tls.keystore.password`

- `rvs.api.rmi.tls.keystore.password`

  Encrypted password of the keystore used for RMI over TLS communication. Please see also parameters `rvs.api.rmi.tls.enabled` and `rvs.api.rmi.tls.keystore.filename`.

10.2 Customizing rvsEVO via rvs-system.properties File

Below you can find the description of the of defintions of `$RVS_HOME/conf/rvs-system.properties` file.

Please enable the settings with re-starting the rvsEVO server.

- `rvs_evo.jobstarter.keep_temp_files=true`

  This definition refers to the jobstarts. If the value of **Parameter Handling** parameter is REPLACE the jobstarter creates temporary files including the job data which substitutes the placeholders of the script. You can define whether the temporary files are to be deleted or not. Possible values:

  `true`: the files are kept

  `false`: the files are deleted (default)

- `rvs_evo.serviceprovider.keep_temp_files=true`

  Via this definition you can define whether temporary files created by the service provider are to be deleted or not. Possible values:

  `true`: the files are kept

  `false`: the files are deleted (default)

- `rvs_evo.protocol.restart=false`

  Via this definition you can define whether a restart flag is to be set up in SSID and the job is restarted.

  `true`: restart flag is set up (default)

  `false`: no restart flag

- `rvs_evo.routed_job.fsp.dont_keep.receive_file=true`

  By default, files received via `FileServiceModul` feature are kept until receiving the EERP. If `rvs_evo.routed_job.fsp.dont_keep.receive_file=true`, this function is deactivated and the files are deleted after successful routing.

- `observer.sfiddesc.usefilename=true`

  If the file transmission takes place via Observer you can define whether the original filename is to be used as file description (SFID-DESC).

- `rvs_evo.controller.underscore_in_vdsn_allowed=true`

  Use this definition to allow underscores in VDSN. Possible values:

`true`: underscores are admitted in VDSN

`false`: underscores are not admitted in VDSN (default)

- `useKeyUsage=false`

  Via this definition you can define whether the certificate usage of X.509 certificate is to be considered or not.

  `true`: the key usages for each function has to be set in the X.509 certificate (default)

  `false`: the settings of X.509 certificate will not be checked.

- `javax.net.debug=ssl`

  Via this definition you can activate the debugging for TLS connections. By default, this is disabled.

- `java.rmi.server.hostname=<IP>`

  The value of this property represents the host name string that should be associated with remote stubs for locally created remote objects, in order to allow clients to invoke methods on the remote object. The default value of this property is the IP address of the local host, in "dotted-quad" format.

- `rvs_evo.network.remote_callback_handler.verification_initial_delay=30000`

  Use the definition above to define the wait time in milli seconds after server start before end-to-end check between rvsEVO and rvs® OFTP Proxy will start.

- `rvs_evo.network.remote_callback_handler.verification_interval=60000`

  Use the definition above to define the wait time in milli seconds between two end-to-end checks between rvsEVO and rvs® OFTP Proxy.

- `rvs_evo.tracing.api.enabled = true`

  Use this definition to enable tracing function for API.

- `rvs_evo.tracing.api.subjects = *.*`

  Use this definition to configure tracing function for API. Enter a comma separated list of classes and methods.

`*.*` : enables tracing of all classes with all of its public methods.

`Rvs.login` : enables tracing of method "`login`" in class "`Rvs`".

`StationList.*` : enables tracing of all methods in class "`StationList`".

- `rvs_evo.receive_job.keep_temp_files_on_failure = true`

  Enable this defininition for keeping temporary files, generated during receiving a job, in case of failure in `$RVS_HOME/files/temp` directory and display the file name in the detailed view of the receive job.

- `rvs_evo.networkaddress.cache.ttl=30`

  Enable this property to set the validation (ttl) in seconds for caching DNS entries.

  Default value: `30`

- `rvs_evo.receive_job.ignore_jobstart_and_delivery_error=false`

  With this property is possible to force the application to ignore error while delivering files or executing jobstart after receive.

  Default value: `false`

- `rvs_evo.receive_job.ignore_illegal_efid_-values=false`

  Only relevant in case of transmitting files with AS/400 machine:
  With this property the application can be forced to ignore illegal values in EFID.

  Default value: `false`

- `rvs_evo.serviceprovider.status_update.interval=1`

  With this property it is possible to adjust time service provider will write progress information to Monitor Log.

  Default value: 1 minute

- `rvs_evo.network.tls.handshake_timeout=15000`

  Timeout in milliseconds for SSL handshake.

Sets SoTimeOut when a SSLSocket is created before handshake is initiated.

IMPORTANT: timeout must be less than OFTP_TIMEOUT

- `rvs-evo.controller.sendqueue.polling.interval=200`

  This property allows to change the length of the interval the send queue uses to poll for new jobs that needs sending.

  Default value: `200` milliseconds

- `rvs-evo.controller.activejob.status.interval=600`

  This property allows to change the interval used to write actual job workload to Monitor Log.

  Default value: `600` milliseconds

- `rvs-evo.controller.archiveJob.txtLevel=2`

  This property provides additional writing of RevisionLog.txt. (TAB separated values; in values: TAB is replaced by SPACE, leading and closing whitespaces are removed; every 100 rows a headline is printed). Possible values:

  – `0` : do not write RevisionLog.txt

  – `1` : basic

  – `2` : extended (default)

  – `3` : same as revisionLog.xml but TAB separated values

  – `4` : customized by the user via element `rvs-evo.controller.archiveJob.txtHeader` (see below)

- `rvs-evo.controller.archiveJob.txtHeader=`

  This property is mandatory in case of `rvs-evo.controller.archiveJob.txtLevel=4`

  `txtHeader` is a comma separated list of valid column headers. First column header is set implicit to `JobID`, so `JobID` is not to be set.

  Valid column headers are:

  ```
  FileName FileNameSrc SourceFileLength VDSN SidOrigi-
  nator SidNeighbour SidDestination OIDOriginator
  OIDNeighbor OIDDestination Direction NoCopy Routing
  SFIDDateTime RecordLength RecordFormat Conver-
  sionTable InputCharacterSet OutputCharacterSet
  Serialize Disposition SecurityFeatureSet Compres-
  sion Encryption SignFile SignEERP EnvFormat SFIDDes-
  ```

criptionHex SFIDDescriptionText EncryptionAlgorithm EncryptionCertificateIssuerName EncryptionCertificateSerialNumber SignCertificateIssuerName SignCertificateSerialNumber ExternalJobID ScheduleDateTime RecordMode Label RetryCount TimeStartFile TimeEndFile RestartPos CurrentRecordLength MarkedForComplete RecordCount FilePos TransmittedBytes LastByteRead LastByteSend SendAttempts LastChange WaitTime Category State ResponseCreatorSID ResponseCreatorOID NerpReasonCode NerpReasonText AssignERP UniformatFilename CompressedFilename EncryptedFilename SpCompleted SpCmsFormatCompleted SpCmsSignCompleted SpCmsCompressCompleted SpCmsEnvelopeCompleted SpCmsTempFileName ErpSignature ErpType ErpVDSN ErpDate ErpTime ErpDestination ErpOriginator ErpHash SyncSendAttempts SyncTimeout ErrorType ErrorID ErrorText

- `rvs-evo.controller.log_to_console=true`

  This property allows to enable logging to console after server start.

  Default value: `false`

- `rvs_evo.protocol.deactivate_sfid_datetime_check=false`

  This property deactivates check of SFIDDATE and SFIDTIME.

  For example: invalid is DATE 20180199 (2018/01/99) or TIME 080070 (8:00:70)

  Default value: `false`

- `rvs_evo.serviceprovider.compression_level=1`

  This property allows to adjust the compression level that the service provider will use for compression.

  Available values from `-1 to 9`.

  Default value: `1`

- `rvs_evo.serviceprovider.maximum_buffer_size=128`

  This property allows to change the buffer size used for pre- and post-processing.

  Unit: `kilo bytes`

  Default value: `128`

- `rvs_evo.controller.pre_and_post_processing.number_of_used_threads=40`

  This property controls the number of threads which are used for pre- and postprocessing. We do not recommend to increase the value.

  Default value: `40`

- `rvs_evo.controller.disable_erp_routing=false`

  This property allows to disable termination of routing jobs after receiving EFPA for the routing send jobs and to wait for EERP/NERP to terminate (default behaviour up to version 6.02.01).

  Default value: `false`

- `rvs_evo.controller.sfna_retry_no.automatic_fail=true`

  This property allows to switch back to automatic FAILED_WITH_NERP behaviour, which was implemented up to version 6.02.03, on reception of SFNA RETRY=NO for routing send jobs.

  Possible values:

  - `true`: NERP will be sent to originator and routing send and receive jobs will terminate with FAILED_WITH_NERP (category FAILED)

  - `false`: routing send and receive job will not terminate to allow operator restart or deletion of routing jobs with or without NERP

  Default value: `false`

- `rvs-evo.dispatcher=HazelcastDispatcher`

  This property allows to change the dispatcher used for internal rvsEVO communication.

  Possible values:

  - `SolitaireDispatcher` (default on Windows, Java 32-Bit and HP-UX)

  - `HazelcastDispatcher` (default on other plattforms)

- `rvs_evo.network.cluster.shared_proxy=true`

  This property allows to change behaviour for starting proxy listeners when more cluster nodes use the same proxy. If true is set then node initiating the command cares about remote listeners and other nodes ignore them.

  Default value: `false`

- `rvs_evo.crypto.validation.weak_sids=`

  This property allows to ignore certificate validation for the indicated station ID. Several SIDs are separated by comma.

- `rvs_evo.crypto.filesignature.verification.weak_-sids=`

  This property allows to ignore file signature for the indicated station ID. Several SIDs are separated by comma.

- `rvs_evo.protocol.format_t.maximum_line_length=32768`

  This property allows to change the maximum line length for format T files in case of routing send jobs.
  OFTP specification defines the maximum line length as 2048, but this is extended by rvsEVO as operating systems Windows, Linux, UNIX allow higher values.
  Line length usually is only a problem for sending files to z/OS systems.
  Default value: `32768` (bytes)

- `rvs_evo.controller.pdx.online.receive.disabled=false`

  This property allows to deactivate the possibility to receive PDX files (incomming Jobs with VDSN: "OFTP_COMMUNICATION_SETUP").
  In case this property is set to `true`, the reception will be rejected with SFNA (REAS=99).
  Default value: `false`

- `rvs_evo.controller.restrict_files_to_outbox=false`

  In case this property is set to true transmitted files must be within the outbox directory. Files may still be uploaded into the outbox, which allows them to be sent.

  Default value: `false`

Hint: During the update installation the `rvs-system.properties` file is not upgraded but an `rvs-systems.properties.new` file is added to `conf` directory. For making settings via an new `rvs-system.properties` definition, please take over the needed element from the new properties file to your old one.
If your `rvs-system.properties` file was not edited up to now, you can rename `rvs-system.properties.new` to `rvs-systems.properties`.

10.3 Customizing rvsEVO via rvsEVOClient.prefs File

Below you can find the description of the defintions of rvsEVO`Client.prefs` file in `$RVS_HOME/conf` directory.

- `<entry key="client.RMIServiceName" value="rvsEVO"/>`

  This definition sets client RMIServiceName which is used in local mode only and must be equal to the database entry for the related server.

- `<entry key="client.RMIServiceHost" value="RDDE9D84"/>`

  This definition sets client RMIServiceHost which is used in local mode only and must be equal to the database entry for the related server.

- `<entry key="client.RMIServicePort" value="3755"/>`

  This definition sets client RMIServicePort which is used in local mode only and must be equal to the database entry for the related server. `client.RMIServicePort` is a integer between `0` and `65535` with default `3755`.

- `<entry key="client.account" value="rvs"/>`

  This definition sets client account which is used in local mode only. Is set usually during installation. Should be changed by specialists only.

  `<entry key="client.passwd" value="ZOKWCALCZGKW-WICB"/>`

  This definition sets client password which is used in local mode only. Is set usually during installation. Should be changed by specialists only

- `<entry key="client.MaxLoginHistory" value="10"/>`

  This definition sets the number of maximum login history details which were shown in drop down list of **Login History Details** field during starting remote GUI.

- `<entry key="client.FirstLanguage" value="en"/>`

  Via this definition you can define the GUI language. This setting does not concern the log messages. Changes only take effect after restarting rvsEVO GUI. Possible values:

  `en`: english

  `de`: german

- `<entry key="client.RemoteAliasMode" value="no"/>`

  Via this definition you can allow remote login with the alias name defined in remote-login-alias.properties file (for remote GUI only).

- `<entry key="client.remotefileloader.port" value="3756" />`

  Port for updoading files in remote GUI

- `<entry key="client.sendjobs.allow.OID.addressing" value="yes"/>`

  With this definition you can activate parameter **Addressing Mode** in the wizard for creating a send job via rvsEVO GUI.

- `<entry key="client.transfer.sendJobWizard.external-JobId" value="mandatory"/>`

  This definition sets up **External Job ID** to mandatory parameter in the wizard for creating send jobs via rvsEVO GUI.

- `<entry key="client.monitorlog.livelog.pagesize" value="1000"/>`

  This definition sets up the number of messages that should be displayed in Live Monitor Log

- `<entry key="client.monitorlog.[INF|WRN|ERR].colour.[r|g|b]" value="[255|0]"/>`

  This definition highlights the display of information (`INF`), warnings (`WRN`) and error messages (`ERR`).

  Example:

  ```
  <entry key="client.monitorlog.WRN.colour.g" value="255"/>
  ```

  In the example above warnings are displayed in green (colour.g). Changes only take effect after restarting rvsEVO GUI.

- `<entry key="client.startjob" value=""/>`
  `<!-- used in swing GUI only -->`
- `<entry key="client.home.dir" value=""/>`
  `<!-- used in swing GUI only -->`

- `<entry key="client.conf.dir" value=""/>`
  `<!-- used in swing GUI only -->`
- `<entry key="client.engdat.conf.dir" value=""/>`
  `<!-- used in swing GUI only -->`
- `<entry key="client.arc.dir" value=""/>`
  `<!-- used in swing GUI only -->`
- `<entry key="client.help.file" value=""/>`
  `<!-- used in swing GUI only -->`

10.4 Customizing the Log Files

The maximum file size and maximum number of log files can be defined via `$RVS_HOME/conf/rvs-logger.properties` file.

By default the `monitor.log`, `rvs.log`, `sysinfo.log` and `rvsClient.log` files are saved in directory `$RVS_HOME/log`.

This setting as well as maximum size and maximum number of log files can be changed in `rvs-logger.properties`.

monitor.log

A new `monitor.log` file is generated daily and the old `monitor.log` file is renamed in „monitor.log“ plus date pattern plus counter. Also a new file is generated if the maximum file size is reached.

- Via element **rvs.evo.server.log.monitor.file.size** you can set the maximum size (default 10 MB).
- Via element **rvs.evo.server.log.monitor.file.count** you can set the maximum number of log files (default 50).

rvs.log

A new `rvs.log` file is generated if the maximum file size is reached.

- Via element **rvs.evo.server.log.rvs.file.size** you can set set the maximum size (default 2 MB).
- Via element **rvs.evo.server.log.rvs.file.count** you can set the maximum number of log files (default 10).

sysinfo.log

A new `sysinfo.log` file is generated if the maximum file size is reached.

- Via element **rvs.evo.server.log.sysinfo.file.size** you can set set the maximum size (default 10 MB).
- Via element **rvs.evo.server.log.sysinfo.file.count** you can set the maximum number of log files (default 50).

rvsClient.log

A new `rvsClient.log` file is generated if the maximum file size is reached.

- Via element **rvs.evo.client.log.rvs.file.size** you can set set the maximum size (default 2 MB).
- Via element **rvs.evo.client.log.rvs.file.count** you can set the maximum number of log files (default 10).

10.5 Customizing rvsWeb and REST-API

Via `$RVS_HOME/conf/rvs-rest.properties` file you can customize rvsWeb, REST-API and Distributed Update. Please see the User Manual, chapter „Start and Display the rvsWeb-GUI“ or „REST-API“ or chapter 7 "Distributed Update" in this manual for further information.

Below you find a description of the definitions:

rvs-rest.properties Parameters:

Parameters	Description
rvs.web.server.activate	Activates the functionalities rvsWeb, REST-API and Distributed Update. Possible values: - `true` (default) - `false`
rvs.web.server. hostname	Hostname or IP address of rvs web server
rvs.web.server.port	Port for access to rvs web server (default: 8433)
rvs.web.server.https. active	Connection to rvs web server via HTTPS Possible values: - `true` (default) - `false`
rvs.web.server.https. keystore.filename	Path and name of HTTPS keystore
rvs.web.server.https. keystore.password. encrypted	Encrypted password for HTTPS keystore. You can encrypt the password via `keytool` script (please see rvsEVO User Manual, chapter ’rvsEVO Database’/’Database Maintenance and further Administrator Tools’/’keytool: Key Management for RMI Communication over TLS’ for more information)
rvs.web.gui.contact. email	Displayed contect information in rvsWeb Default: `rvs-support@t-systems.com`
rvs.web.gui.contact. phone	Displayed phone number for contact in rvsWeb Default: `+49-375-606-19-902`
rvs.web.gui.public. access	Allows access without login Possible values: - `true` - `false` (default)
rvs.web.gui.public.query .result.limit	Maximum number of entries for transmission queries. Default: 10000

rvs-rest.properties Parameters:

Parameters	Description
rvs.web.gui.view. branding.type	Branding: Possible values: - `rvsweb-theme.json` - `rvsevo-theme.json` (default)
rvs.web.gui.view.layout	Layout Possible values: - `rvsweb-layout.json` - `rvsevo-layout.json` (default)

Hint:

HTTPS Keystore

For connection via HTTPS a keystore (p12 file) is needed (parameter **rvs.web.server.https.keystore.filename** in `rvs-rest.properties` file). If **rvs.web.server.https.active** is activated and no keystore is deposited, a p12 file will be generated during first start of rvsEVO server. For secure connection, this keystore must be replaced with a keystore with CA certificate. Path, name and encrypted password of the keystore must be configured in `rvs-rest.properties` file. You can encrypt the password via `keytool` script (please see rvsEVO User Manual, chapter 'rvsEVO Database'/'Database Maintenance and further Administrator Tools'/'keytool: Key Management for RMI Communication over TLS'for more information).

Customizing starting page of rvsWEB

rvsWEB is available for rvsEVO and rvsMVS. Via `$RVS_HOME/conf/rvs-rest.properties` file you can configure the instances which shall be available via starting page.

Syntax for an rvsEVO instance element:

```
rvs.web.instances.evo.<serial no>.<parameter>=<value>
```

Syntax for an rvsMVS instance element:

```
rvs.web.instances.mvs.<serial no>.<parameter>=<value>
```

Example:

In the following example one rvsMVS instance and two rvsEVO instances are configured for access via starting page:

```
rvs.web.instances.evo.0.id=Q2
rvs.web.instances.evo.0.description=CAD-Gateway
rvs.web.instances.evo.0.access=login
rvs.web.instances.evo.0.url=https://q2.gdc-bln01.t-systems.com:3999/rvs/web


rvs.web.instances.evo.1.id=SFS4.GATEWAY
rvs.web.instances.evo.1.description=Local Instance
```

```
rvs.web.instances.evo.1.access=both
rvs.web.instances.evo.1.url=https://q3.gdc-bln01.t-
systems.com:8084/rvs/web/

rvs.web.instances.mvs.0.id=Q8
rvs.web.instances.mvs.0.description=B-Verbund
rvs.web.instances.mvs.0.access=public
rvs.web.instances.mvs.0.url=http://q8.gdc-bln01.t-
systems:8080/rvs/web
```

rvsWEB starting page parameters:

Parameter	Description
id	Name of rvs instance
description	Short description
access	Access type for rvs web server. Possible values: - `public` (connection without login) - `login` - `both`
url	URL of rvs web server

10.6 Customizing RMI Connection over TLS

From rvsEVO version 6.02.05 upwards the communication between rvsEVO client and server can be encrypted via TLS protocol. Please see below how to customize rvsEVO for secure RMI communication.

10.6.1 Setting up of an RMI Connection via TLS Protocol

Necessary steps for line encryption via TLS protocol between client and server:

- Generate keystore file via `keytool` script. In rvsEVO User Manual chapter „Database Maintenance and further Administrator Tools“ you find a description of `keytool` script. The X.509 certificate is exported into the truststore automatically.
- Keystore and truststore must be saved in **`$RVS_HOME`**`/conf` directory. At all clients the truststore should be saved in **`$RVS_HOME`**`/conf` directory too. For connecting rvsRuntime GUI via TLS truststore is to be saved in **`$RVS_RUNTIME_HOME`**`/conf` directory.
- Set up the rvsEVO server for RMI over TLS via
  - **rvs.api.rmi.tls.enabled**

  - **rvs.api.rmi.tls.keystore.filename**
  - **rvs.api.rmi.tls.keystore.password** elements

  in `$RVS_HOME/conf/rvs-api-rmi.properties` file (the parameters are described in chapter 10.1 "Customizing the RMI Parameters").

- Set up the rvsEVO clients via setting the TLS definition in `$RVS_HOME/conf/api-rmi-connection.properties` file:

  Syntax:
  `<remote-hostname>,<remote-port>`
  `=@TLS,<trust-store-path>`

10.6.2 Customizing RMI Connection via SSH Tunneling

Connection failure due to firewall restrictions between remote client and rvsEVO server can be sidestepped by SSH tunneling. In **`$RVS_HOME`**`/conf/api-rmi-connection.properties` file you can customize static and dynamic SSH tunneling.

Static SSH tunneling

In case of static SSH tunneling the SSH tunnel needs to be created by user (e. g. with Putty) before starting the rvsEVO Client.

Syntax of definition in `api-rmi-connection.properties` file:

```
<remote-hostname>,<remote-port>=
'@EXISTING_SSH_TUNNEL',<ip-connection-port>
```

Dynamic SSH tunneling

In case of dynamic SSH tunneling the SSH tunnel is created by rvsEVO client. You can establish the connection with your login data for the server host or with a key.

Syntax of definition in `api-rmi-connection.properties` file:

```
<remote-hostname>,<remote-port>=
'@DYNAMIC_SSH_TUNNEL',<tunnel-base-port>,<auth-
type>,<user-id>,<pass>
```

Parameter for SSH Tunneling:

Parameter	Description
auth-type	Type of authentication with dynamic SSH tunneling. Possible values: - @AUTH_PASSWD for using login data on rvsEVO server machine - @AUTH_PUBKEY for using key created with ssh-keygen program
ip-connection-hostname	Hostname or IP address for connection via tunnel
ip-connection-port	Port for connection via tunnel

Parameter for SSH Tunneling:

Parameter	Description
remote-hostname	IP address or hostname of rvsEVO server machine
remote-port	RMI port of rvsEVO server
tunnel-base-port	First port number dynamic SSH tunnel will try to create tunnel. If the port is already in use next free port will be used.
user-id	User ID on rvsEVO server machine
pass	Password on rvsEVO server machine. The password has to be encrypted.

10.7 Pass Parameters to Jobstart Process

In „rvsEVO User Manual“ chapter „Management of Jobstarts“ you get information about jobstarts after transmission and jobstarts after failure.

This chapter provides a detailed discription of Jobstart parameter **Parameter Handling** with settings **Parameter Handling**=`ENV` and **Parameter Handling**=`REPLACE`.

Parameter Handling=ENV

Parameter Handling=`ENV`: job data are set as environment variables. The following environment variables are specified (in angle brackets: name in XML file, in brackets: GUI name):

- **RVS_COMPRESSION** <compression> (Compression)
- **RVS_CONVERSION_TABLE** <conversionTable> (Code Table)
- **RVS_CREATED_AT** <creationDate> (SFID Date/Time)
- **RVS_DATE** Date of <creationDate>
- **RVS_DESCRIPTION** <fileDescription> (File Description)
- **RVS_DIRECTION** <direction> (Direction)
- **RVS_DISPOSITION** <disposition> (Disposition)
- **RVS_ENCRYPTION** <encryption> (Encryption)
- **RVS_ENCRYPTION_ALGORITHM** <encryptionAlgorithm> Encryption Algorithm)
- **RVS_ERROR_ID** <errorID> (Error Number)
- **RVS_ERROR_TEXT** <errorText> (Description)
- **RVS_EXTERNAL_JOB_ID** <externalJobId> (External Job ID)
- **RVS_FILE_BASENAME** (File name)

- **RVS_FILE_NAME** <filename> (Path and name of the file in outbox directory)
- **RVS_FILE_NAME_SRC** <filenameSrc> (Source File Name)
- **RVS_JOB_ID** <jobNumber> (ID)
- **RVS_LABEL** <label> (Label)
- **RVS_LAST_CHANGE** <lastStateChange> (Last Change)
- **RVS_NERP_CREATOR_ODETTE_ID** <nerpCreatorOdetteID> (Nerp Creator OdetteID)
- **RVS_NERP_REASON_CODE** <nerpReasonCode> (Nerp Reason Code)
- **RVS_NERP_REASON_TEXT** <nerpReasonText> (Nerp Reason Text)
- **RVS_PRIORITY** <priority> (Priority)
- **RVS_RECORD_FORMAT** <recordFormat> (Format)
- **RVS_RECORD_LENGTH** <recordLength> (Record Length)
- **RVS_RESTART_POSITION** <restartPos> (Restart Position)
- **RVS_SECURITY_FEATURE_SET** <securityFeatureSet> (Security Feature Set)
- **RVS_SEND_ATTEMPTS** <sendAttempts> (Send Attempts)
- **RVS_SERIALIZE** <serialization> (Serialization)
- **RVS_SID** <SID>
- **RVS_SID_DESTINATION** <sidDestination> (SID Destination)
- **RVS_SID_ORIGINATOR** <sidOriginator> (SID Originatior)
- **RVS_SIGNATURE** <sign> (File Signature)
- **RVS_SIGNATURE_EERP** <signEERP> (Request Signed EERP/ NERP)
- **RVS_STATUS** <status> (Status)
- **RVS_TIME** Time of <creationDate>
- **RVS_TIME_START_FILE** <timeStartFile> (Time Start File)
- **RVS_TRANSMITTED_BYTES** <transmittedBytes> (Bytes Transmitted)
- **RVS_VDSN** <VDSN> (VDSN)

Please see the rvsEVO User Manual, table “Explanation of the job attributes” in chapter ’Command Line Tools’ for description of the environment variables .

Parameter Handling=REPLACE

Parameter Handling=`REPLACE`: A copy of the denoted file is created and discarded into `temp` directory. The placeholders of the scripts are substituted for job data. The placeholders are encased by ? (e.g. "`move ?DSN? /home/`rvsEVO`/incomming" becomes "move /home/`

rvsevo/rvsEVO/files/inbox/TEST.TXT /home/rvsevo/incomming").

With MAINTENANCE parameter **PermissionTempScript** you can set up the file permissions of the temporary file. (Default: read/write/execute-permission for owner and group).

Placeholder of all jobstarts:

- **?DSN?:** DataSetName (name of local data set, where received information has been stored)
- **?VDSN?:** Virtual data set name under which the data set was transmitted
- **?DTAVAIL?:** Date, when the data set was available for sending; Format TT/MM/JJ HH:MM:SS
- **?FORMAT?:** Record format of data set
- **?BYTES?:** Number of transmitted bytes
- **?RECORDS?:** Number of transmitted records for F and V format data sets; always zero for **T** und **U** format data sets.
- **?DTRCV?:** Date, when data set was delivered to local user in format TT.MM.JJ HH:MM:SS
- **?UID?:** User ID
- **?SID?:** Station ID
- **?DSNTEMP?:** Name of temp. data set

Placeholder of jobstart type SND:

- **?MAXRECL?:** The meaning of this field depends upon the record format of the transmitted data set:
- - F format: length of each record
- - V format: maximum length a record may have
- - T und U format: always 0 (zero)
- **?LABEL?**: string if the send command contained a LABEL parameter. Can be used to identify the send command.
- **?SECN?**: rvsEVO job ID (number of send command SE in rvs® portable)
- **?SKCN?**: rvsEVO job ID (number of send command SK in rvs® portable)
- **?SIDORIG?**: Station ID of originatior
- **?SENDATT?**: number of unsuccessful attempts after which the program is to be started

Placeholder of jobstart type RCV:

- **?MAXRECL?**: The meaning of this field depends upon the record format of the received data set:

– - `F` format: length of each record
– - `V` format: maximum length a record may have
– - `T` und `U` format: always 0 (zero)
– **?CNQS?:** rvsEVO job ID (command number of EERP (End-to- End-Response) for received file in rvs® portable).
– **?CNIE?:** rvsEVO job ID (command number of IE for received file in rvs® portable).
– **?CNIZ?:** rvsEVO job ID (command number of IZ for received file in rvs® portable)

10.8 Conversion Tables

The file transfer provides the possibility of code conversion. This chapter gives a generally explanation of the conversion tables and describes the process of creating own conversion tabels.

10.8.1 Code Tables Offered by rvsEVO

Text files are stored on most systems in one of two computer codes, namely ASCII (American National Standard Code for Information Interchange) or EBCDIC (Extended Binary Coded Decimal Interchange Code).

ASCII is the standard code for UNIX and DOS/Windows systems. EBCDIC was developed for IBM Mainframe computers.

ASCII: US-ASCII ISO 646; the ASCII character set defines 128 characters (0 to 127 decimal). This character set is a subset of many other character sets with 256 characters, including the ANSI character set of MS Windows.

ANSI: Windows ANSI, Values 0 to 127 are the same as in the ASCII character set, values 128 to 255 are similar to the ISO Latin-1 character set.

EBCDIC 037: support characters, which are used in the following countries: Australia, Brazil, Canada, New Zealand, Portugal, South Africa, USA.

EBCDIC 273: supports characters (especially umlauts), which are used in the following countries: Germany, Austria and Switzerland.

10.8.2 How to Add Your Own Conversion Table

In order to use your own conversion table, please follow the procedure given below:

• Create a new conversion table. The content of this file is described in the following XML schema:

```
<conversiontable codein=Original-Characterset
codeout=Ziel-Characterset>
```

```
    <description>Table to converts signs from ... to
...</description>
     <char value="0">0</char>
     <char value="1">1</char>
            .
            .
</conversiontable>
```

Note: `char value` is equivalent to the source code and the value in angle brackets is equivalent to the target code.

- Store the XML file in the directory `$RVS_HOME/conf/conversiontables`
- To add your table to the list of conversion tables, edit the XML file `$RVS_HOME/conf/conversiontables/characterSetConverters.xml`.

  Next you find an abridged version of the file

  `characterSetConverters.xml`:

```
<characterSetConverters coding="">
    <!-- -->
    <converter cid="ANSIIBM037"
        >conversion_ANSI_IBM037.xml</converter>
    <converter cid="IBM273-ASCII"
        >conversion_IBM273_ASCII.xml</converter>
        .
        .
</characterSetConverters>
```

  In the example above `cid="ANSI-IBM037" (cid="IBM237-ASCII")` is the used name in rvsEVO and `conversion_ANSI_IBM037.xml (conversion_IBM237_ASCII.xml)` is the name of your XML file.

- Please restart rvsEVO after your complement.

10.9 Connection Pooling

When using MS SQL, PostgreSQL- or Oracle database rvsEVO provides the function **connection pooling**.

A connection pool is a cache with existing data base connections. If a connection is needed, one of the pool can be used and it is not necessary to establish a connection for every access. rvsEVO uses `C3P0` (Hibernate) for connection pooling.

The settings for Connection Pooling can be done via `$RVS_HOME/conf/jdbc.properties` file.

Example of configuration with default values:

```
c3p0.minPoolSize=5
c3p0.maxPoolSize=25
c3p0.checkoutTimeout=0
c3p0.maxIdleTime=0
c3p0.idleConnectionTestPeriod=60
c3p0.automaticTestTable=POOLING_TEST
c3p0.acquireRetryAttempts=30
c3p0.acquireRetryDelay=1000
c3p0.breakAfterAcquireFailure=false
```

The most important parameter is `c3p0.idleConnectionTestPeriod`. With this parameter you define the time interval (in seconds) for considering the connections of the pool. The parameter `maxPoolSize` defines the maximum number of connections to the database and `minPoolSize` defines the initial number of connections.

For further information please read the `C3P0` documentation:

`http://community.jboss.org/wiki/HowToconfiguretheC3P0connectionpool`

10.10 Operating with OFTP Routing Rules

From version 6.02.05 upwards rvsEVO provides the possibility of operating with internal routing rules on router side. That means you can route incoming files to another destination or convert the transmitted stations data of originator in routing send jobs for outgoing transmissions.

Application

You decide to perform your rvsEVO processes from an new machine. All your partners must take over your new data into their systems. You can circumvent this procedure by applying OFTP routing rules. OFTP routing rules can be created, updated, displayed, deleted and listed via `rvscli` feature. (See rvsEVO User Manual, chapter 'Command Line Tools' for more information about rvscli)

Requirements

– All affected stations are reachable via router (gateway)

– Feature File Service Module is linked to your rvsEVO system which functions as router.

  – OFTP parameter **File Service Module** of internal stations is set to `INTERNAL`

  – OFTP parameter **File Service Module** of external stations is set to `EXTERNAL`.

Hint:

© T-Systems

De- and encryption for stations set up as INTERNAL is processed by routing station. Please see the rvsEVO User Manual, chapter „File Servcie Module“ for further information about File Service Module.

Please see the rvsEVO User Manual, chapter „Management of Stations“ --> „Routed Stations“ for information about setting up of a routed station.

Examples:

– Internal station INT1 (destination) receives files from external station EXT (originator) via station ROUT (gateway). Station INT1 is replaced with internal station INT2.

  With rvscli command `createOftpRoutingRule` the files can be routed to new station INT2:

  ```
  createOftpRoutingRule --type INBOUND --newSid INT2
  --originator EXT --destination INT1 --vdsn *
  ```

– Internal station INT2 (originator) transmits files to external station EXT (destination) via gateway ROUT. Partner station INT1 is configured with EXT. The data of INT2 are unknown with EXT. Create the following routing rule to convert the transmitted stations data of routing send job from INT2 to INT1:

  ```
  createOftpRoutingRule --type OUTBOUND --newSid INT1
  --originator INT2 --destination EXT --vdsn *
  ```

– All incoming jobs with VDSN=“TEST“ shall be routed to INT2. This can be obtained with the following command:

  ```
  createOftpRoutingRule --type INBOUND --newSid INT2
  --destination INT1 --vdsn TEST
  ```

Hint:

Routing rules are to be created by the gateway station.

In case of using an OFTP routing rule no particular ERP job is generated.

No changes of settings are necessary at the external station.

Further rvscli commands concerning OFTP routing rules:

- `updateOftpRoutingRule`
- `displayOftpRoutingRule`
- `listOftpRoutingRule`
- `deleteOftpRoutingRule`

The parameters are described in the usages.

11 Maintenance of rvsEVO

In this chapter you find a description of the rvsEVO installation folder and information about required maintenance of directories and your stored data.

Via parameter group **MAINTENANCE** in Parameter perspective of rvsEVO GUI settings for automatic backup and cleanup can be configured. (see rvsEVO User Manual, chapter „Customizing the rvsEVO Parameters“ for more information).

11.1 rvsEVO Installation Folder

During installing process the following subdirectories are installed in **$RVS_HOME** (see chaper 1.1 for further information) directory:

- **archive:** directory for archive and backup files

  `RevisionLog.xml` and
  `RevisionLog.txt` files contain the entries of processed send or receive jobs

  `jar files` contain the backup data.
- **bin:** contains scripts (see chapter ’Command Line Tools’ in rvsEVO User Manual for more information)
- **clib:** contains C-Libraries
- **conf:** contains configuration files
  - **conf/conversiontables:** convertion tables
- **db:** contains derby data base files (only existing with derby embedded data base)
- **docs:** contains readme file, unser manual, operator manual, ClientAPI documentation
  - **conf/jobstart-examples:** examples of jobstart scripts
  - **docs/rvsEVOClientAPI:** ClientAPI libraries (javadoc, examples)
- **files:** contains the subdirectories used for transmission
  - **files/inbox**: default folder of received files
  - **files/outbox**: default folder of files to be transmitted
  - **files/temp**: default folder of temporary files for post- and preprocessing of send jobs
- **jobs**: after installing the folder is empty. Recommended for own scripts (e.g. for automating or logging processes) because the folder is saved during back up process
- **jre and subdirectories:** default folder of JRE, provided by rvsEVO installer (not available for genericUNIX)
- **lib:** contains rvsEVO libraries

- **log:** contains log files

  `monitor.log`: current rvsEVO messages

  `rvs.log`: StackTraces of rvsEVO server, error log

  `rvsClient.log`: StackTraces of rvsEVO clients, error log

  `rvsEVO_Installieren_MM_DD_YYYY_hh_mm_ss.log`: installer log

  `rvscli.log`: messages from command line tool `rvscli`

  - **log/trace/odette:** .Odette trace files
  - **log/trace/tls:** .TLS trace files
  - **log/trace/api:** .API trace files
- **snmp-agent:**
  - **bin:** contains scripts for starting rvsSNMP-Agent
  - **conf:** contains configuration files for rvsSNMP-Agent
  - **docs:** contains readme and user documentation for rvsSNMP-Agent
  - **mib:** contains the configuration files for NMS
- **system:** recommended directiory for PKCS#12 files of OFTP2-Proxies
  - **system/conf:** configuration files that does not belong to EVO business logic. The files that are independent from a certain tenent/ mandant. The files should not be edited.
  - **system/data:** contains subdirectory `/system/data/xsd` with XML shema definitions
  - **system/db:** contains generic scripts for creating, initialazing and removing of database tables.

    This scripts use the SQL scripts of `/system/db/derby`, `/system/db/mssql`, `/system/db/postgresql` and `/system/db/oracle` subdirectory
  - **system/distribution:** contains the `processDistributedUpdate` script for setting up and executing Distributed Update.
- **tools and subdirectories:** contains scripts for maintenance of rvsEVO database.

  Have a look at the rvsEVO User Manual, chapter „rvsEVO Database“ for detailed information of the scripts.
- **UninstallerData:** contains files needed for uninstalling rvsEVO.

11.2 Maintenance of Directories

The following directories are not cleared by rvsEVO and have to be maintained by the rvsEVO operator to prevent filling up of the directories. In

parameter group RESOURCE_CHECK you can set the options for warnings and error messages in case of low disk space (see rvsEVO User Manual, chapter „Customizing the rvsEVO Parameters“ - Group RECOURCES CHECK).

– **archive:** .jar files of backup and RevisionLog files
– **files:** inbox, outbox, temp.
– Directory for Observer functionality.

  Hint: In rvs-system.properties file and during creating send jobs you can set up whether the files in outbox and Observer directory should be kept after transmission or not.

– **log:** log files and subdirectories with trace files.

11.3 Maintenance of rvsEVO Database

The job and MonitoLog data must be maintained by the rvsEVO operator to prevent filling up of the database.

11.3.1 Deletion of Job Data from Database

There are several possibilities for deletion of job data:

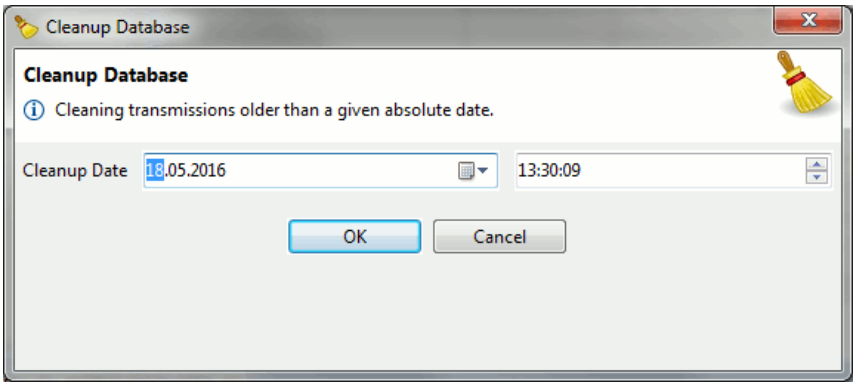
– via Transfer perspective of rvsEVO GUI
– via `cleanup` cmd tool (see chapter 11.3.3 "Deletion of Jobs and Monitor Log Data from the Database")
– via MAINTENANCE parameter **CleanupDays, CleanupInterval, CleanupTime** (see rvsEVO User Manual, chapter „Customizing the rvsEVO Parameters“ - Group MAINTENANCE).
– via `rvscli` tool (see rvsEVO User Manual, chapter „Command Line Tools“).

rvsEVO GUI

Hint: This function is only available for user with administrator or operator rights.

Click the **Cleanup** icon in the toolbar for deletion of ended and failed jobs from the database. The jobs which are older than the entered date and time will be deleted.

© T-Systems



11.3.2 Deletion of Monitor Log Data from Database

Monitor Log data are only saved in rvsEVO database if MAINTENANCE parameter **MonlogInDatabase** = `Y`

There are several possibilities for deletion of Monitor Log data:

– via Monitoring perspective of rvsRVO GUI

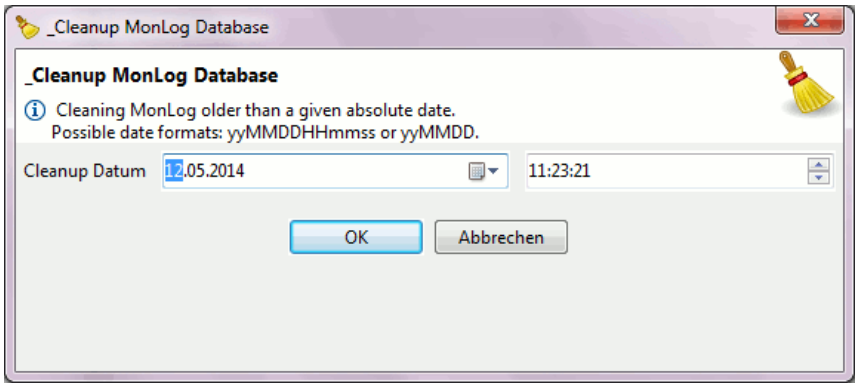
– via `cleanup` cmd tool (see chapter 11.3.3 "Deletion of Jobs and Monitor Log Data from the Database")

– via MAINTENANCE parameter **CleanupDays**, **CleanupInterval**, **CleanupTime**, **CleanupInterval**, **MonlogInDatabase**, **MonlogInDatabaseCleanup** (see rvsEVO User Manual, chapter „Customizing the rvsEVO Parameters“ - Group MAINTENANCE)

– via `rvscli` tool (see rvsEVO User Manual, chapter „Command Line Tools“)

rvsEVO GUI

Single-click the `Cleanup Monlog` icon in the toolbar to open the dialog window for deletion of Monitor Log data. All entries are deleted which are older than defined date.



11.3.3 Deletion of Jobs and Monitor Log Data from the Database

Use the `cleanup` program to delete ended and failed transmission jobs or/and Monitor Log data which are older than the defined date.

Syntax:

```
cleanup -d <date> [-r][-t <type>][-help] [-?]
[-verbose|-v]
```

Required Parameter:

Parameter	Description
-d <date>	This option needs a date in the following format: `yyMMddHHmmss` or `yyMMdd`. The jobs or Monitor Log data, which are older than the entered date will be deleted from the database.

Optional Parameters:

Parameter	Description
-r	Remote, availables deletion during rvsEVO is running.
-t **-type**	Type of entry to be deleted. Possible values: - `JOBS` (default): data of ended or failed jobs - `MONLOG`: Monitor Log data - `ALL`: jobs and Monitor Log data
-help	Requests help information.
-?	Displays a description of the current command.
-verbose (-v)	Verbose message output.

12 ODETTE Protocol

It is the purpose of the Odette File Transfer Protocol (OFTP) to ensure the reliable transfer of a data set. The OFTP enters a protocol session with the OFTP on the remote rvsEVO station which logically runs on top of the linedriver connection.

After the OFTP session has started, both sides exchange their Odette IDs and passwords, negotiate some parameters, like Odette exchange buffer size, Odette credit value (the number of buffers the sending side can send without waiting for a response), and exchange information about name, approximate size and format of the data set to be transferred.

During transfer, a compression and decompression of data is performed. After the data have been transferred, the byte count is checked between both sides. After the data set has successfully been stored, a receipt is sent to the sending station. If the transfer has been disrupted, for example by a link failure, the OFTP protocol provides a mechanism that allows to restart the transfer at the point of rupture.

'Change direction' feature allows the receiver to become sender and to send acknowledgments and data sets.

For protocol details kindly refer to the publications of the ODETTE and VDA groups: "ODETTE Specifications for File Transfer".

12.1 RFC 5024: OFTP Commands for OFTP File Transfer

Below you find the most important OFTP commands:

Session Establishment

SSRM: Start **S**ession **R**eady **M**essage
This is the first command to be sent after a connection has been requested, indicating that the application is communicating with another OFTP.

SSID: Start **S**ession **I**n**D**ication
This is a request for initiate a session. It contains information such user and password of the requester.

File Transfe**r**

SFID: Start **F**ile **I**n**D**ication
This is a request for permission to send a file. It contains information such as the origin and destination of the file, its name and physical size.

SFPA: Start **F**ile **P**ositive **A**nswer

This is a positive acknowledgement for SFID request. Transfer is accepted.

SFNA: Start **F**ile **N**egative **A**nswer
This is a negative acknowledgement for SFID request. Transfer is rejected.

DATA: The actual file data sent to the trading partner.

© T-Systems

CDT: Set **C**re**D**i**T**
Service command for flow control.

EFID: End of **F**ile **I**n**D**ication
This command is sent immediately after the file has been transferred containing control totals to ensure the integrity of the sent file.

EFPA: End **F**ile **P**ositive **A**nswer
This is a positive acknowledgement for EFID request. Transfer is successfully completed.

EFNA: End **F**ile **N**egative **A**nswer
This is a negative acknowledgement for EFID request. Transfer is cancelled.

CD: Change **D**irection
The CD command gives token to partner.

Confirm File Receiption

EERP: End to **E**nd Res**P**onse
The EERP is generated when a file reaches its ultimate destination. It is sent to the originator of the file to inform them that the file has been received.

NERP: Negative **E**nd **R**es**P**onse
The NERP is generated when a file cannot be transmitted to its ultimate destination. It is sent to the originator of the file to inform them that the file cannot be transmitted

Terminate Session

ESID: End **S**ession **I**n**D**ication
The ESID requests that the communication session be terminated.

Please sea chapter 14.7 "Failure during Bild-up of OFTP Session to Neighbour Station" and following for information on ReasonCodes in case of failure.

12.1.1 Message Flow of Odette Session

Shown below is the general but simplified message flow within an Odette session. The sending side acts as initiator, the receiving side as responder.

Initiator (sender)		Responder (receiver)
Connect	----- network connect ----->	
	<----- SSRM -----	„ODETTE FTP READY“
Odette ID, password. buffersize, credit	----- SSID ----->	
	<----- SSID -----	Odette ID, password etc.

Start**F**ile **ID** (name, size, format)	----- SFID ----->	
	<----- SFPA -----	**S**tart**F**ile **P**ositive **A**nswer
'n' data records	----- DATA -----> ----- DATA -----> ----- DATA -----> ...	
	<----- CREDIT -----	send credit value 'n'
send 'n' data records	----- DATA -----> ----- DATA -----> . .	
End**F**ile **ID** (byte count	----- EFID ----->	
	<----- EFPA ----- or <----- EFNA -----	**E**nd**F**ile **P**ositive **A**nswer (if store successful and byte count correct)/ **E**nd**F**ile **N**egative **A**nswer
Change **D**irection	------ CD ------>	
	<----- EERP ----- or <----- NERP -----	**E**nd-to-**E**nd **R**es**p**onse/ **N**egative **E**nd-to-End **R**es**p**onse (acknowledge)
	<----- ESID -----	**E**nd-**S**ession **ID**
Network disconnect		Network disconnect

© T-Systems

13 Monitoring of rvsEVO

In this chapter you find out how to display rvsEVO messages and warnings.

13.1 What Gets Logged

In rvsEVO, messages from rvsEVO server, rvsEVO Swing GUI and rvsEVO clients are logged by default and saved in log files.

Additionally the transmitted Odette data, TLS and Proxy TLS connection data and used API methods are traced, if configured.

13.2 Log Files

The log files contain the messages from rvsEVO server and rvsEVO clients. By default, they are saved in `$RVS_HOME/log` directory.

Please see chapter 10.4 "Customizing the Log Files" for information about setting up the maximum file size and maximum number of log files.

There are the follwing log files:

– `monitor.log` (monitor messages)
– `rvs.log` (error messages and warnings from `rvsEVO` server)
– `rvsClient.log` (messages from the command prompt)
– `rvscli.log` (messages from command line tool `rvscli`)

13.2.1 monitor.log

The `monitor.log` file contains all messages which are displayed in rvEVO GUI: Errors (ERR), warnings (WRN) and information (INF).

A new `monitor.log` file is generated daily and the old `monitor.log` file is renamed in „monitor.log“ plus date pattern plus counter. Also a new file is generated if the maximum file size is reached.

The list beolw shows the message sending rvsEVO modules:

rvsEVO Modules and its Kind of Information:

Module	Kind of Message
ACX	Automatic exchange of certificates
CTRL	Controller, e.g. cleanup, Proxy connection, start-up listeners, stopping receive channel
NET	Network, e.g. network connection information, expire of certificates
OBSR	Observer

rvsEVO Modules and its Kind of Information:

Module	Kind of Message
OFTP	Odette protocol, e.g. Odette session information
PROXY	Proxy
PDX	Odette Partner Data eXchange
SP	Service provider, e.g. pre- and post processing

The list below shows examples of short messages that are saved in `monitor.log` file. All short messages are explaned in Text column. For informtion about error messages see chapter 14 "Error Situations of rvsEVO".

Messages from ACX module:

– ACX_CERT_FOR_REF_FOUND
– ACX_CERT_FOR_REF_NOT_FOUND
– ACX_CERTIFICATE_IMPORT_DENIED
– ACX_CERTIFICATE_NOT_IMPORTED
– ACX_CERTIFICATE_REPLACED
– ACX_CERTIFICATE_USAGE_ADOPTED
– ACX_COMMAND_FAILED
– ACX_DELIVER
– ACX_DELIVER_FAILED_MISSING_CERT
– ACX_DELIVER_WITH_REF
– ACX_IMPORT_SUCCESSFUL
– ACX_RECEIVE_DELIVER
– ACX_RECEIVE_DELIVER_DENIED
– ACX_RECEIVE_DELIVER_FAILED
– ACX_RECEIVE_DELIVER_REF_DENIED
– ACX_RECEIVE_DELIVER_WITH_REF
– ACX_RECEIVE_REPLACE
– ACX_RECEIVE_REPLACE_DENIED
– ACX_RECEIVE_REPLACE_FAILED
– ACX_RECEIVE_REPLACE_REF_DENIED
– ACX_RECEIVE_REPLACE_WITH_REF
– ACX_RECEIVE_REQUEST
– ACX_REPLACE

– ACX_REPLACE_FAILED_INVALID_CERT
– ACX_REPLACE_WITH_REF
– ACX_REQUEST
– ACX_REQUEST_FAILED_MISSING_CERT

Messages from CTRL module:

– CREATE_SENDJOB
– JOB_SET_TO_SEND_NEXT
– JOB_SEND_NEXT_NOT_DONE
– PREPROCESSING_ENDED
– OUTGOING_CALL_INITIATED
– EERP_ASSIGNED
– TERMINATE_SENDJOB
– CLEANUP_STARTED
– REINIT_JOBLIST
– CLEANUP_COMPLETED
– SERVER_STARTED
– CERTIFICATE_EXPIRATION
– RES_NOT_AVAIL_DIR

Messages from NET module

– LISTENER_START_SUCCESS
– CONNECTION_CREATE_SUCCESS_1
– CONNECTION_CLOSE_SUCCESS
– INCOMING_CALL_RECEIVED

Messages from OBSR module:

– OBSERVER_STOPPED
– OBSERVER_STARTED
– OBSERVER_SE_CREATED

Messages from OFTP module:

– SENT_F_PDU
– RECEIVED_F_PDU
– SESSION_CREATED
– OFTP_TIMEOUT

Messages from PDX module:

– PDX_PROCESSING_FAILED
– PDX_PARSE

- PDX_NEEDS_PROCESSING
- PDX_INBOUND_ACTIVATED
- PDX_INBOUND_ACTIVATION_FAILED
- PDX_INBOUND_SCHEDULED
- PDX_INBOUND_SCHEDULED_FAILED
- PDX_INBOUND_HANDLE_EERP_FAILED
- PDX_INBOUND_REJECTED
- PDX_INBOUND_ACCEPTED
- PDX_OUTBOUND_ACTIVATED
- PDX_OUTBOUND_ACTIVATION_FAILED
- PDX_OUTBOUND_CREATED
- PDX_OUTBOUND_SEND
- PDX_OUTBOUND_SCHEDULED
- PDX_OUTBOUND_SCHEDULED_FAILED
- PDX_OUTBOUND_HANDLE_EERP_FAILED
- PDX_OUTBOUND_CHANGE_NETWORK_TYPE
- PDX_COMMAND_FAILED
- AUTOMATIC_PDX_DIR_DELETE_STARTED
- AUTOMATIC_PDX_DIR_DELETE_COMPLETED

Messages from PROXY module:

- LISTENER_START_SUCCESS
- CONNECTION_CREATE_SUCCESS_1
- CONNECTION_CLOSE_SUCCESS
- RMI_TLS_CERT_EXPIRING

Messages from SP module:

- SERVICE_PROVIDER_CHAIN_STARTED
- START_CONVERSION2_OFTP2FORMAT
- FINISH_CONVERSION2_OFTP2FORMAT
- CMS_SIGNATURE_START
- CMS_SIGNATURE_FINISHED
- CMS_COMPRESSION_START
- CMS_COMPRESSION_FINISHED
- CMS_ENVELOPING_START
- CMS_ENVELOPING_FINISHED
- MESSAGE_DIGEST_BUILD_START

– MESSAGE_DIGEST_BUILD_FINISHED
– SERVICE_PROVIDER_CHAIN_FINISHED
– CMS_ERP_VERIFY_SIGNATURE_START
– CMS_ERP_VERIFY_SIGNATURE_FINISHED

Messages of server start:

– CLEANUP_STARTED
– OBSERVER_STARTED
– REINIT_JOBLIST
– CLEANUP_COMPLETED
– OUTGOING_CALL_INITIATED
– LISTENER_START_SUCCESS
– SERVER_STARTED

Messages of resource check:

– CERTIFICATE_EXPIRATION
– RES_NOT_AVAIL_DIR

Hint: Please see the rvsEVO User Manual, chapter „Monitoring“ for further information about Monitor Log.

13.2.2 rvs.log

The `rvs.log` file contains error messages and warnings from rvsEVO server. In detail the following information are saved ...

– ... with each start-up of rvsEVO server:
  – java.properties
  – rvsEVO properties
  – license validation data
– ... in on-going operation processes:
  – error messages from rvsEVO server, by default
  – warnings from rvsEVO Server, if configured

The messages in `rvs.log` are also displayed via console.

Please see chapter 14 "Error Situations of rvsEVO" for more information about error messages and warnings.

13.2.3 rvsClient.log

The warnings and error messages of rvsEVO SWING-GUI and of command line tools are saved in `$RVS_HOME/log/rvsClient.log` file.

13.3 Tracing

The trace files are saved in `$RVS_HOME/log/trace` directory by default. You can change this setting via **LOGDIR** parameter in parameter group DIRECTORIES.

13.3.1 Odette Tracing

Hint: Please see the rvsEVO User Manual, chapter „Log and Trace Files“ for information about enabling Odette tracing.

The Odette trace files contain the transmitted Odette data packages. Below you find a description of the saved Odette parameters:

Start Session
- SSIDLEV: OFTP Release
- SSIDCODE: Initiator / Responder Odette ID
- SSIDPSWD: Receive / Send password
- SSIDSDEB: Exchange Buffer Size
- SSIDSR: Transmission capability (**S**end, **R**eceive, **B**oth)
- SSIDCMPR: Compression
- SSIDREST: Restart
- SSIDSPEC: Special Logic
- SSIDCRED: Excgange Buffer Credit
- SSIDAUTH: Authentication
- SSIDRSV1: No function
- SSIDUSER: Special user information
- SSIDCR: Carriage return

Start File
- SFIDDSN: VDSN
- SFIDRSV1: Without function
- SFIDDATE: SFID Date
- SFIDTIME: SFID Time
- SFIDUSER: Special user information
- SFIDDEST: Destination Odette ID
- SFIDORIG: Originator Odette ID
- SFIDFMT: Format
- SFIDLRECL: Record Length
- SFIDFSIZ: File Size
- SFIDOSIZ: Original File Size
- SFIDREST: Restart Position
- SFIDSEC: Security Feature Set

– SFIDCIPH: Encryption Algorithm
– SFIDCOMP: Compression
– SFIDENV: Enveloping format
– SFIDSIGN: Signed EERP/NERP requested
– SFIDDESCL: Length of File Description
– SFIDDESC File Description

Start File Positive Answer
– ACNT: Answer Count (for Restart Position)
– EFPACD: Change directory

End File
– EFIDRCNT: Record count
– EFIDUCNT: Transmitted Bytes

End to End Response
– EERPDSN: VDSN
– EERPRSV1: No function
– EERPDATE: Date (YYYMMDD) for file date stamp
– EERPTIME: Time counter (hhmmsscccc) for file time stamp
– EERPUSER: Special user information
– EERPDEST: Odette ID of destination
– EERPORIG: Odette ID of originator
– EERPHSHL: File hash length with signed EERP
– EERPHSH: File hash
– EERPSIGL: Length of EERP signature
– EERPSIG: EERP signatur

End Session
– ESIDREAS: Reason code
– ESIDREASL: Length of reason text
– ESIDREAST: Reason text
– ESIDCR: Carriage return

13.3.2 TLS Tracing

Please see the rvsEVO User Manual, chapter „Log and Trace Files“ for information about enabling TSL tracing.

13.3.3 API Tracing

API tracing function allows tracing of methods of the following API classes:

– Configuration
– JobList
– KeyList

- RVS
- StationList
- UserList
- UserManager
- UserProfile

See the javadoc in `$RVS_HOME/docs/rvsEVOClientAPI/rvs-evo-api-6.01.07-javadoc.jar` for more information about the API classes and its methods.

Enable API tracing function via defintion `rvs_evo.tracing.api.enabled = true` in `$RVS_HOME/conf/rvs-system.properties` file.

Configure API tracing function via definition `rvs_evo.tracing.api.subjects = *.*` in rvs-system.properties file. (See also chapter 10.2 "Customizing rvsEVO via rvs-system.properties File").

14 Error Situations of rvsEVO

This chapter presents error situations with related entries in `monitor.log`, `rvsClient.log` and `rvs.log` and proposes possible solutions. The `monitor.log` entries are also displayed via rvsEVO GUI.

14.1 Debugging the Installer

Before starting the installer, set the environment variable **LAX_DEBUG** to record the error messages to the console.

Linux / Unix:

```
export LAX_DEBUG=true
```

Windows:

```
set LAX_DEBUG=true
```

14.2 Failure of Installation

14.2.1 Wrong Permission of DB Scripts

Rights for DB scripts were set wrong by the installer.

Error:

```
/tmp/174067.tmp/executeScriptTmp0[2]: /home/qsevo/
rvsEVO/system/db/create.sh:

0403-006 Execute permission denied.
```

Reason:

The user temp directory respectively **$IATEMPDIR** (see rvsEVO User Manual, chapter „New Installation of rvsEVO and rvs Runtime“ for more information) does not have enough free space.

Solution:

New installation: Clean up your temp directory, uninstall / delete your rvsEVO installation and start the installer again.

Update installation: Clean up your temp directory, recover your rvsEVO backup which was created before update installation and start the installer again.

14.2.2 Aborting of Installation on Linux/Unix Systems

During launching process:

Occasionally the installer aborts during the launching process after the messages below:

```
[qsevo@q4de3esy426:~/installer]$ ./rvsEVO-setup-
6.01.08-GenericUnix.bin

Preparing to install...
```

```
Extracting the installation resources from the
installer archive...

Configuring the installer for this system's environ-
ment...

Launching installer...

[qsevo@q4de3esy426:~/installer]$
```

Solution:

Make the following Install Anywhere setting via environment variable **DO_NOT_FORK**:

```
export DO_NOT_FORK=true
```

Thereafter the installation should be executed completely.

With error 'JRE libraries are missing or not compatible':

The installer aborts with following message:

```
Preparing to install
Extracting the JRE from the installer archive...
Unpacking the JRE...
Extracting the installation resources from the
installer archive...
Configuring the installer for this system's environ-
ment...
Launching installer...
JRE libraries are missing or not compatible....
```

Reason:

Either user temp directory respectively **$IATEMPDIR** (see rvsEVO User Manual, chapter „New Installation of rvsEVO and rvs Runtime“ for more information) does not have enough free space or user rights for **$IATEMPDIR** are insufficient .

Solution:

Set value of **$IATEMPDIR** to a directory with enough space and execute rights before the installation is started:

```
export IATEMPDIR=<dir>
```

If the error occurs again set environment variable **DO_NOT_FORK**:

```
export DO_NOT_FORK=true
```

14.2.3 Aborting of Installation on UNIX Systems

UnsupportedClass-VersionError

The installer aborts with following message:

```
Error: A JNI error has occurred, please check your
installation and try again

Exception in thread "main" java.lang.UnsupportedClas-
sVersionError: JVMCFRE199E bad major version 55.0 of
```

```
class=com/zerog/lax/LAX, the maximum supported major
version is 52.0; offset=6
```

Reason:

The expected JRE version, needed by the installer, is not found on your system.

Solution:

Include the needed JRE version in your **$PATH** environment variable and execute the installation again.

Example:

```
PATH="/home/rvsuser/JRE/semeru/jdk-17.0.12+7-jre/
bin":$PATH
```

14.2.4 Aborting of Installation on Windows Systems

Sometimes the installation is broken after choosing install set during checking the administrator rights.

Solution:

Set the environment variable **SKIP_ADMIN_TEST** to „`true`“ in the same command line where the installer will be started before (update) installation is started:

```
set SKIP_ADMIN_TEST=true
```

14.2.5 Installation of rvsEVO or rvs® OFTP Proxy aborts with NoClassDefFoundError

The installer aborts with following message:

```
java.lang.NoClassDefFoundError: Could not initialize
class java.awt.Toolkit
```

Solution:

Perform the following setting in the same shell where the installer will be started before (update) installation is started:

```
export _JAVA_OPTIONS=-Djava.awt.headless=true
```

14.3 Failure of rvsEVO Server Start

14.3.1 License over

monitor.log:

```
2016-11-16 08:19:32,462 | ERR | CTRL | LICEN-
SE_OVER                    |            |
|                 | The license [{0}] has been expired.

 … [ERROR ] [main ]
```

rvs.log:

```
2016-11-16 08:19:32,459 [DEBUG ] [main      ]
com.tsystems.rvs.evo.controller.licenses.LicenseCon-
troller.?(): Validation Result         :  [NOT VALID]

2016-11-16 08:19:32,459 [ERROR ] [main      ]
com.tsystems.rvs.evo.controller.licenses.LicenseCon-
troller.?(): License over!

2016-11-16 08:19:32,459 [DEBUG ] [main      ]
com.tsystems.rvs.evo.controller.licenses.LicenseCon-
troller.?(): <--------------LICENSE---------VALIDA-
TION---------------

2016-11-16 08:19:32,665 [ERROR ] [main      ]
com.tsystems.rvs.evo.EVO.?(): Error validating
license. Server startup interrupted.
```

Reason:

License is expired

Solution:

Save a valid license file `license.properties` in `$RVS_HOME\conf` directory.

14.3.2 Failure of rvsEVO Server Start with Derby Database

Every process accessing the derby database locks the derby data base exclusive: no other processes can access to the database.

Server is already running:

rvs.log:

```
* initialising application ...

[ERROR ] org.hibernate.tool.hbm2ddl.SchemaUpdate.?():
HHH000319: Could not get database metadata

java.sql.SQLException: Failed to start database '/
home/qsevo/rvsEVO/db'

with class loader sun.misc.Launcher$AppClas-
sLoader@5c647e05, see the next

exception for details.

Caused by: ERROR XSDB6: Another instance of Derby may
have already booted the database /home/qsevo/
rvsEVO.6.01.03.RC2/db.
```

Reason:

rvsEVO server is already running.

Solution:

Stop the running rvsEVO server and start the rvsEVO server again.

RMI port in use

rvs.log:

```
java.rmi.server.ExportException: Port already in use:
3755; nested exception is:

java.net.BindException:  Address already in use:
JVM_Bind
```

Reason:

RMI Port is in use.

Solution:

If the port is not used by another process: restart your system.

If the port is used by another process: Change the RMI port in the rvsEVO configuration files in `$RVS_HOME\conf` directory:

– `rvsEvoClient.prefs`: element `client.RMIServicePort`

– `rvs-api-rmi.properties`: element `rvs.api.rmi.service.port`

Warning without effect on rvsEVO functions

During the server start the warning below can be output. This does not have any effect on the rvsEVO functions.

```
* checking database connection...

2017-11-14 16:30:16,028 [WARN  ]
com.hazelcast.nio.UnsafeHelper.?(): sun.misc.Unsafe
has been disabled because your platform does not
support unaligned access to memory, some features
might be not available.
```

14.4 Failure of Listener Start

14.4.1 Port is already in use

external database

rvs.log:

```
2016-11-21 13:52:02,090 [ERROR ] [10.33.67.220]
com.gedas.rvs.tiny.network.tls.TLSListener.?(): run:
Exception java.net.BindException: Cannot assign
requested address: JVM_Bind
```

monitor.log:

```
2016-11-21 13:52:02,180 | WRN | NET  | LISTENER_-
START_FAILURE           |            |
|                  | Failed to start TLS Listener,
system tried the following parameters: ip=/
10.33.67.220, port=6619. Reason: Message: Cannot
assign requested address: JVM_Bind. Cause: BindExcep-
tion
```

```
2016-11-21 16:54:57,862 | WRN | NET  | LISTENER_-
START_FAILURE           |             |
|                  | Failed to start TCPIP Listener,
system tried the following parameters: ip=RDDE9D84/
10.177.191.220, port=3307. Reason: Message: Address
already in use: JVM_Bind. Cause: BindException
```

Reason:

More than one TLS-listerners are generated with the same port number

Solution:

Change the port number of the added listener

14.5 Failure of RCP-GUI Start on Linux Systems

Error in log file saved in $`RVS_HOME/rcpgui/configuration` folder:

```
!ENTRY org.eclipse.osgi 4 0 2023-01-31 14:16:55.241
!MESSAGE Bei der automatischen Aktivierung des Produktpa-
kets com.tsystems.rvs.client.gui.framework.core (17) ist
ein Fehler aufgetreten.

...

Caused by: org.eclipse.swt.SWTError: No more handles
[gtk_init_check() failed]

...
```

Solution:

Please check the `X11 forwarding` setting in Putty.

`X11 forwarding` must be enabled.

14.6 Connection Failure

14.6.1 Connection to Proxy Bastion Fails

Bastion not active

rvs.log:

```
2016-11-22 15:25:31,744 [ERROR ] [RemoteCallback-
Handler] com.tsystems.rvs.evo.network.remote.call-
back.CallbackHandlerDelegate.?(): Receiving of
requests got interrupted. - Unable to connect either
the primary nor the backup proxy.

2016-11-22 15:25:31,745 [ERROR ] [main      ]
com.gedas.rvs.tiny.network.impl.ResetBas-
tionRule.?(): Unable to connect to proxy. (RemoteNet-
workHost [hostName=RDDE9D84, port=3762,
serviceName=service, command=, backupHostName=,
backupPort=0, rmiSenderIp=RDDE9D84, remoteClas-
sName=com.gedas.rvs.evo.network.remote.rmi.server.Rem
```

```
oteNetworkManagerImpl, tlsEnabled=false, trustStore-
FileName=])
```

monitor.log:

```
2016-11-22 15:25:58,486 | ERR | CTRL | PROXY_CHECK_-
FAILED                |              |
|                | End-to-end check of proxy 1 failed.
```

Reason:

Proxy Bastion was not started.

Solution:

Start the Proxy Bastion first and restart the rvsEVO server

No valid certificate

rvs.log:

```
2017-08-08 16:09:50,444 [ERROR ] [taskScheduler-1]
com.tsystems.rvs.evo.network.remote.callback.Callbac-
kHandlerDelegate.?(): Unable to check proxy connec-
tion. Unable to connect either the primary nor the
backup proxy.
```

```
2017-08-08 16:09:53,994 [ERROR ] [RemoteCallback-
Handler]
com.tsystems.rvs.evo.controller.networks.RemoteNet-
workHostExceptionHandler.?(): Unable to connect to
RemoteNetworkHost [hostName=RDDE9D84, port=2600,
serviceName=service, command=, backupHostName=,
backupPort=0, rmiSenderIp=RDDE9D84, remoteClas-
sName=com.gedas.rvs.evo.network.remote.rmi.server.Rem
oteNetworkManagerImpl, tlsEnabled=true, trustStoreFi-
leName=C:\rvsRT\tenants\4001\system\rmiTlsTrust-
Store.p12]. NotAfter: Mon Aug 07 16:02:05 CEST 2017
```

monitor.log:

```
2017-08-08 16:09:50,446 | ERR | CTRL | PROXY_CHECK_-
FAILED                |              |
|                | End-to-end check of proxy 1 failed.
```

Reason:

Certificate is expired.

Soloution:

Import a valid keypair into rvs® OFTP Proxy keystore and exchange the certificate in rvsEVO used for rvs® OFTP Proxy connection (Please see rvs® OFTP Proxy User Manual, chapter „Setting up of a TLS Connection“ for more information.)

Wrong bastion host/ port

rvs.log:

```
2016-11-22 15:42:18,530 [ERROR ] [main      ]
com.gedas.rvs.tiny.network.impl.ResetBas-
tionRule.?(): Unable to connect to proxy. (RemoteNet-
```

```
workHost [hostName=RDDE9D84, port=3765,
serviceName=service, command=, backupHostName=,
backupPort=0, rmiSenderIp=RDDE9D84, remoteClas-
sName=com.gedas.rvs.evo.network.remote.rmi.server.Rem
oteNetworkManagerImpl, tlsEnabled=false, trustStore-
FileName=])
```

monitor.log:

```
2016-11-22 15:42:45,270 | ERR | CTRL | PROXY_CHECK_-
FAILED                |              |
|                | End-to-end check of proxy 2 failed.
```

Reason:

The Proxy Bastion what was set up in rvsEVO is not avavilable.

Solution:

Check and correct the Proxy Bastion Parameters **Server** and **Host** in Stations perspective.

14.6.2 Examples with outgoing call, TCP Connection

monitor.log:

Wrong hostname

Message box (Activate Station):

```
Unable to activate station 1.
Message: Unable to activate station 1.
Details:
Cause: Connect Error: Hostname 'be1dy430-123' can not
be resolved.
```

monitor.log:

```
2019-04-04 18:31:37,575ERRrvsEVO_NodeNETCONNECTION_-
CREATE_FAILURE_104183135300001Failed to create TCPIP
Connection, receiver: ip=Host: be1dy430-123 [lookup
result => unknown host], port=3321, sender:
ip=be1dy430/10.181.183.111, port=3305.  Reason:
UnknownHostException/be1dy430-123
```

Solution:

Check and correct the hostname of your partner station; contact your partner

Host not available

Message box (Activate Station):

```
Unable to activate station MAIN. Message: Unable to
activate station MAIN.
```

Primary connection fails:

```
Details:
Cause: Connect Error: Primary Connection: IP connec-
tion failed for '10.33.156.111' 10.33.156.111:3305 ;
```

Alternative connection fails:

```
Alternative Connection: Hostname '11.33.222.333' can
not be resolved.
```

monitor.log:

Primary connection fails:

```
2019-04-29 13:35:29,066  |  WRN  |  rvsEVO_Node  |
NET  |  CONNECTION_CREATE_PRIMARY  |
2913350830000  |  MAIN  |     |  Failed to create
primary TCPIP Connection, receiver: ip=/10.33.156.111,
port=3305, sender: ip=be1dy430/10.180.68.154,
port=3331.  Reason: CONNECTION_FAILED Connection timed
out: connect.
```

Alternative connection fails:

```
2019-04-29 13:35:31,319  |  ERR  |  rvsEVO_Node  |
NET  |  CONNECTION_CREATE_FAILURE_1  |
2913350830000  |  MAIN  |     |  TCPIP Connection
could not be created. Receiver: ip=Host: 11.33.222.333
[lookup result => unknown host], port=3305, Sender:
ip=be1dy430/10.180.68.154, port=3331, Reason: CONNEC-
TION_FAILED Connection timed out: connect.
```

Test connection (example for Linux systems):

```
-bash-3.2$ telnet 10.33.156.111 3305

Trying 10.33.156.111...

telnet: connect to address 10.33.156.111: Connection
timed out

telnet: Unable to connect to remote host: Connection
timed out
```

Reason:

Host is not available; neither primary IP address/hostname nor alternative connection

Solution:

Check and correct the hostname of your partner station; contact your partner.

Port not available

Message box (Activate Station):

```
Unable to activate station 1.
Message: Unable to activate station 1.
Details:
Cause: Connect Error: IP connection failed for
'10.33.108.246' 10.33.108.246:3323
```

monitor.log:

Primary and alternative connection fail:

```
2016-11-23 10:42:38,757 | WRN | NET  | CONNECTION_-
CREATE_PRIMARY        | 2310423730000  | PS1
|                  | Failed to create primary TCPIP
Connection, receiver: ip=/10.33.146.75, port=3309,
sender: ip=RDDE9D84/10.33.231.43, port=3307.  Reason:
ConnectException/Connection refused: connect
```

```
2016-11-23 10:42:39,788 | ERR | NET  | CONNECTION_-
CREATE_FAILURE_1       | 2310423730000  | PS1
|                | Failed to create TCPIP Connection,
receiver: ip=/10.33.146.75, port=3308, sender:
ip=RDDE9D84/10.33.231.43, port=3307.  Reason:
ConnectException/Connection refused: connect
```

Reason:

Neither primary indicated port nor alternative one is available

Solution:

Check and correct the port of your partner station; contact your partner.

Wrong Odette ID

Wrong entry of Odette ID of your partner station

monitor.log:

```
2016-11-24 08:41:58,270 | ERR | OFTP | SENT_F_PDU
| 2408415730000  | PS1            |                |
ESID - End Session ID sent. REAS=03 (User code not
known), REAST=""
```

```
2016-11-24 08:41:58,391 | INF | NET  | CONNECTION_CLO-
SE_SUCCESS          | 2408415730000  | PS1
|                 | TCPIP Connection was successfully
closed, parameters: ip=10.33.146.75, port=3306, local-
port=60324.
```

Reason:

The created OFTP session is closed because of wrong entry of Odette ID in **Odette ID** parameter of partner station.

Solution:

Check and correct the Odette ID of your partner station. Contact your partner.

Own Odette ID is unknown with the Partner station

monitor.log:

```
2019-04-04 11:41:31,226 INF rvsEVO_Node OFTP
SENT_F_PDU 0411413130000 1 SSID - Start Session ID
sent. OdetteID="OMAIN", Auth="N"
```

```
2019-04-04 11:41:31,233 ERR rvsEVO_Node OFTP RECEIVE-
D_F_PDU 0411413130000 1 ESID - End Session ID
received. REAS=03 (User code not known), REAST=""
```

Reason:

The created OFTP session is closed because of wrong entry of our Odette ID in **Odette ID** parameter with the partner.

Solution:

Comunicate your correct Odette ID to your partner.

14.6.3 Examples with outgoing call, TLS Connection

Wrong IP address / port

monitor.log:

```
… | ERR | PROX | CONNECTION_CREATE_FAILURE_1 | … |
Failed to create TLS Connection,

receiver: ip=/10.33.101.122, port=6700, sender: ip=/
10.33.137.38, port=6720. Reason: ConnectionException/
Exception during creation of TLS socket
```

Reason:

Connection to partner station fails

Solution:

check and correct IP address and port of your partner

Missing partner certificate on receiver side

monitor.log:

```
2016-11-30 11:20:53,758 | INF | CTRL | OUTGO-
ING_CALL_INITIATED       | 3011205330000 | TLSN
|                  | Outgoing TLS call to TLSN
initiated,  parameters: ip=10.33.146.75, port=6621,
Alternative: ip=10.33.146.75, port=6621

2016-11-30 11:20:53,897 | INF | SP   | CERTIFICATE_VA-
LIDATION_RESULT      | 3011205330000  |
|                  | [VALID] [TLS_SERVER] CERT_PATH
validation of certificate [Subject:CN=neups Serial-
Number:5835ad75]

2016-11-30 11:20:53,941 | ERR | NET  | CONNECTION_-
CREATE_FAILURE_1       | 3011205330000  | TLSN
|                  | Failed to create TLS Connection,
receiver: ip=/10.33.146.75, port=6621, sender:
ip=RDDE9D84/10.33.231.65, port=6622.  Reason: Connec-
tionException/Exception during handshake[TLS]
```

rvs.log:

```
2016-11-30 11:20:53,940 [ERROR ] [StartSessionTask:
TLSN] com.gedas.rvs.tiny.network.tls.TLSConnec-
tion.?(): Handshake failed

javax.net.ssl.SSLHandshakeException: Received fatal
alert: bad_certificate
```

Reason:

Connection is build-up but TLS handshake fails, because of missing partner certificate on receiver side.

Solution:

Contact your partner; deliver your certificate to your partner

Receiver does not exist

monitor.log:

```
2016-11-23 13:32:40,676 | ERR | NET  | CONNECTION_-
CREATE_FAILURE        | 2313324030000  | TLS
|                  | Failed to create TLS Connection,
system tried the following parameters: ip=/
10.33.146.75, port=6619. Reason: ConnectionException/
Receiver number 1 does not exists.
```

Reason:

Number of Associated Receiver Parameter specified with the partner station is not correct.

Solution:

Check and correct the **Number of Associated Receiver** Parameter

Different TLS versions

monitor.log:

```
2022-02-03 16:36:48,322 | ERR | NET  | CONNECTION_-
CREATE_FAILURE_1      | 0316364830000  | MAINT
|                  | TLS Connection could not be
created. Receiver: ip=BE1CL150/10.176.21.117,
port=6619, Sender: ip=BE1CL150/10.176.21.117,
port=6629, Reason: HANDSHAKE_FAILED Received fatal
alert: protocol_version.
```

rvs.log:

```
2022-02-03 16:36:48,321 [ERROR] [RMI TCP Connec-
tion(38)-10.176.21.117]
com.gedas.rvs.tiny.network.impl.tls.TLSConnec-
tion.performHandshake(): Handshake failed

javax.net.ssl.SSLHandshakeException: Received fatal
alert: protocol_version

...
```

Reason:

TLS handshake fails because your partner uses another TLS version. From rvsEVO version 6.03.00 TLSv1 and TLSv1.1 are disabled.

Solution:

If you need to use TLSv1 or TLSv1.1 contrary to the safety recommendations activate TLSv1 and TLSv1.1 in `$RVS_HOME\jre\conf\security\java.security` file by deleting TLSv1 and TLSv1.1 from `jdk.tls.disabledAlgorithms` element. In this case please note that TLSv1 and TLSv1.1 is not considered secure anymore.

Contact your partner.

14.7 Failure during Bild-up of OFTP Session to Neighbour Station

During build-up, transfer and end of OFTP session several indications are exchanged with Odette protocol.

If the building-up failes frequently the ReasonCode of **ESID** (End-Session-Indication) shows the cause of failure. rvsEVO dispays the **ESID** ReasonCode in the Monitor Log:

Examples **Wrong receive password:**

Message box (Activate Station):

```
Unable to activate station 1.
Message: Unable to activate station 1.
Details:
Cause: Wrong password received.
```

monitor.log:

```
2019-04-04 17:58:39,246ERRrvsEVO_NodeOFTPSENT_F_PDU
04175839300001ESID - End Session ID sent. REAS=04
(Invalid password), REAST=""
```

Wrong send password:

Message box (Activate Station):

```
Unable to activate station 1.
Message: Unable to activate station 1.
Details:
Cause: OFTP Session aborted with ESID received:  04
(Invalid password)
```

monitor.log:

```
2019-04-04 18:04:53,818ERRrvsEVO_NodeOFTPRECEIVE-
D_F_PDU04180453300001ESID - End Session ID received.
REAS=04 (Invalid password), REAST=""
```

Solution: contact your partner, square the passwords and adjust the password settings in OFTP parameter group of your partner station.

Codes **Below you find a list of the most importend ESID ReasonCodes (rfc5024):**

- 00: Normal session termination
- 01: Command not recognised
- 02: Protocol violation
- 03: User code not known SSIDCODE (odette ID) unbekannt
- 04: Invalid password SSIDPSWD (SEND Passwort) stimmt nicht mit RECEIVE Passwort des responders überein
- 11: Invalid challenge response

– 12: Secure authentication requirements incompatible
– 99: Unspecified Abort code

14.8 Failure of File Transmission

14.8.1 SFNA and EFNA

In case of failure of file transmission the Start File Negative Answer (**SFNA**) from sender or End File Negative Answer (**EFNA**) from receiver is displayed in the Monitor Log.

Below you find a list of SFID and EFID ReasonCodes (rfc5024) :

– 01: Invalid filename. (invalid symbol in VDSN)
– 02: Invalid destination
– 03: Invalid origin.
– 04: Storage record format not supported.
– 05: Maximum record length not supported.
– 06: File size is too big.
– 10: Invalid record count.
– 11: Invalid byte count.
– 12: Access method failure.
– 13: Duplicate file.
– 14: File direction refused.
– 15: Cipher suite not supported. (Please see also chapter 14.8.2)
– 16: Encrypted file not allowed.
– 17: Unencrypted file not allowed.
– 18: Compression not allowed.
– 19: Signed file not allowed.
– 20: Unsigned file not allowed.
– 99: Unspecified reason.

The codes below concern only EFID:

– 21: Invalid file signature.
– 22: File decryption failure.
– 23: File decompression failure.

14.8.2 SFNA: REAST=15 „Cipher suite not supported“

Reason:

You get this error in case of incompatible settings.

With encrypted file transfer the settings of File Security parameter C**ipher Suite** must be compatible with the settings of your partner.

Solution:

Enable Odette tracing and send a test file to your partner. You can check the required settings in the trace file, SFID parameter **SFIDCIPH** (Cipher Suite). Please see chapter 13.3.1 "Odette Tracing" and rvsEVO User Manual, chapter "Log and Trace Files" for more information about Odette tracing.

The following tabel shows the required settings for **SFIDCIPH** (only for Cipher Suites, supported by rvsEVO):

Cipher Suite	Encryption Algorithm	Hash Algorithm
02	AES256_CBC	SHA-1
04	AES256_CBC	SHA-256
06	AES256_CBC	SHA-512
07	AES_256_CBC	SHA3-512

Hint: Encryption Algorithm DES_EDE3_CBC is not supported anymore. If DES_EDE3_CBC is indicated for any process, rvsEVO switchs to AES automatically.

14.8.3 Error Messages via NERP

The end of the transmission via OFTP is confirmed with End to End Response (**EERP**) or Negative End Respone (**NERP**). NERP means that the transmission is ended with error.

Below you find a list of NERP ReasonCodes (rfc5024) :

– 03: ESID received with reason code '03' (user code not known)

– 04: ESID received with reason code '04' (invalid password)

– 09: ESID received with reason code '99' (unspecified reason)

– 11: SFNA(RETRY=N) received with reason code '01' (invalid file name)

– 12: SFNA(RETRY=N) received with reason code '02' (invalid destination)

– 13: SFNA(RETRY=N) received with reason code '03' (invalid origin)

– 14: SFNA(RETRY=N) received with reason code '04' (invalid storage record

– format)

- 15: SFNA(RETRY=N) received with reason code '05' (maximum record length
- not supported)
- 16: SFNA(RETRY=N) received with reason code '06' (file size too big)
- 20: SFNA(RETRY=N) received with reason code '10' (invalid record count)
- 21: SFNA(RETRY=N) received with reason code '11' (invalid byte count)
- 22: SFNA(RETRY=N) received with reason code '12' (access method failure)
- 23: SFNA(RETRY=N) received with reason code '13' (duplicate file)
- 24: SFNA(RETRY=N) received with reason code '14' (file direction refused)
- 25: SFNA(RETRY=N) received with reason code '15' (cipher suite not supported)
- 26: SFNA(RETRY=N) received with reason code '16' (encrypted file not allowed)
- 27: SFNA(RETRY=N) received with reason code '17' (unencrypted file not allowed)
- 28: SFNA(RETRY=N) received with reason code '18' (compression not allowed)
- 29: SFNA(RETRY=N) received with reason code '19' (signed file not allowed)
- 30: SFNA(RETRY=N) received with reason code '20' (unsigned file not allowed)
- 31: File signature not valid.
- 32: File decompression failed.
- 33: File decryption failed.
- 34: File processing failed.
- 35: Not delivered to recipient.
- 36: Not acknowledged by recipient.
- 50: Transmission stopped by the operator.
- 90: File size incompatible with recipient's protocol version.
- 99: Unspecified reason.

14.8.4 Outgoing Call, EERP is Missing

The file is delivered to the partner; the send job goes in state WF_EERP

Reason:

The setting of **EERP Out** parameter of partner station on receiver side is `NEVER` or `HOLD`

Solution:

Contact the responsible person of your partner station

14.8.5 Routing Send Job Fails with ESIDREAS=06

Transmission of files in format **T** to routed z/os systems could lead to failure with ESIDREAS=06 (bad data).

Reason:

OFTP specification defines the maximum line length as 2048, but this is extended by rvsEVO as operating systems Windows, Linux and UNIX allow higher values.

Solution:

Activate element `rvs_evo.protocol.format_t.maximum_line_length` in **`$RVS_HOME`**`/conf/rvs-system.properties` file and reduce the value of maximum line length (e.g. to 2048)

14.9 File Transmission: Pre- or Post-Processing Fails

14.9.1 Key or Certifiacte is Missing

In case of failure with pre- or post-processing the rvsEVO module ServiceProvieder (SP) writes the reason in `moitor.log` file. The job stays in **ACTIVE** catagory and the transfer can be finished after removing the problem's cause (e.g. missing or expired certificate).

Example:

monitor.log:

```
2016-11-15 14:53:53,717 | WRN | SP   | CMS_UNWRAP-
PING_ENVELOPE_FAILED     |             | PS1
| 1611151453530000 | Failed decryption of jobId
[1611151453530000] file
C:\rvsEVO\files\temp\1611151453530000: Key not found
for received serial number 58087EFE (alias=LOC).

2016-11-15 14:53:53,723 | WRN | SP   | SERVICE_PROVI-
DER_CHAIN_FINISHED_WIT |             | PS1
| 1611151453530000 | Finished service CMS_RCV provider
chain with error.

2016-11-15 14:31:06,810 | ERR | CTRL | POSTPROCES-
SING_FAILED            |             | PS1
| 1611151431060000 | Postprocessing failed
(JobID=1611151431060000).
```

rvs.log:

```
2016-11-15 14:31:06,822 [ERROR ] [PostProcessingTask]
com.tsystems.rvs.evo.controller.jobs.JobCon-
troller.?(): Postprocessing of ReceiveJob
1611151431060000 failed. JobDataError
```

Reason:

Missing keypair of local station of destination

Solution:

Associate the corresponding key pair to local station and activate **Restart SP**.

Hint: In case of problems on sender side the job must be restarted by the sender.

14.9.2 Jobstart After Receive Fails on Receiver Side

Outgoing call: file transmission is finished but the send job terminates with state FAILED_WITH_NERP:

```
2016-11-30 15:08:23,514 | ERR | OFTP | RECEIVE-
D_F_PDU                | 3015082230000  | PS1
|                | NERP - Negative End to End Response
received. DEST="MYOIDPS", ORIG="OPT2",
VDSN="INPUT12.TXT", DATE=20161130, TIME=1508220001,
CREA="OPT2", REAS=34, REAST="Jobstart after receive
failed. File was delivered."
```

Reason:

Jobstart process on receiver side (partner) fails.

Solution:

Contact your partner

14.9.3 Failure of OFTP2 Conversion with Cluster Functionality on Linux and UNIX Systems

Pre-processing fails on files in Observer directory with the below messages:

monitor.log:

```
../log/monitor.log:2016-06-12 12:25:54,669 | INF | SP
| START_CONVERSION2_OFTP2FORMAT      |              |
SFS4.EXTERN      | 1606111126200001 | Starting conver-
sion of file /tbshare/lxcluster/2observe/SFS4.EXTERN/
T13.s4yd.251.1465637180395.gen to oftp2format.
```

```
../log/monitor.log:2016-06-12 12:25:54,672 | WRN | SP
| OFTP2_CONVERSION_FAILED           |              |
SFS4.EXTERN      | 1606111126200001 | !OFTP2_CONVER-
SION_FAILED!
```

rvs.log:

```
2016-06-12 12:25:54,673 [ERROR ] [PreProcessingTask]
com.gedas.rvs.tiny.serviceprovider.command.conver-
sion.Convert2OFTP2FormatCommand.?(): MONITORLOG:
Formating for OFTP2 failed

java.io.FileNotFoundException: /tbshare/lxcluster/
2observe/SFS4.EXTERN/T13.s4yd.251.1465637180395.gen
(Keine Berechtigung)
```

Reason:

The instance owner has no permission on `.gen` file in Observer directory.

Solution:

Set the permissions for the `.gen` file via `chmod` command (`chmod 666 <file>`) and restart the send job.

14.10 Failure of Connection from rvsEVO Client to rvsEVO Server

14.10.1 Failure of Connection to Server if java.rmi.server.hostname is set

Error message:

```
RMI Exception 'java.rmi.ConnectException' caught.

Details: First access to rvsEVO server (RCRvs.getRan-
domString) ==> take care about serverside binding of
rmi objects (remote-reference: RSRvs_Stub[UnicastRef
[liveRef:
[endpoint:[127.0.1.1:3755](remote),objID:[2c693759:16
2b4aba17a:-7fff, -38549985142335106591]]]])

Cause: Connection refused: connect

  com.tsystems.rvs.client.gui.common.exceptions.RVSU-
IException: Message: RMI Exception
'java.rmi.ConnectException' caught.

[...]
```

Solution:

Deactivate element `java.rmi.server.hostname` in `rvs-system.properties` file.

`java.rmi.server.hostname` is set by rvsEVO with value of element `rvs.api.rmi.service.host` in `$RVS_HOME/conf/rvs-api-rmi.properties` file.

14.10.2 Failure of Connection to Server in Case of Invalid Login Data

Error messages:

- `ERROR: Login failed for user <userID>`
  `Message: INVALID_USERNAME_OR_PASSWORD`
  `Details: Login failed`
  `Cause: Unable to build full UserProfile.`
- `Error in connection to Server. Please login again.`

Solution for default user:

Create a backup of `$RVS_HOME/conf/rvsEVOClient.prefs` file, first.
You can find user name and encrypted password in export file of `$RVS_HOME/tools/exportUsers` script. Replace value of element **client.account** of `rvsEVOClient.prefs` file with value of **login name** from export file and value of element **client.passwd** with value of **password** from export file.
If the decrypted password was lost, a new password can be set with `updateUser` command of `rvscli` script.

Solution for other users:

The rvsEVO administrator can set a new password via User Management of rvsEVO GUI or via `updateUser` command of `rvscli` script.

Please see rvsEVO User Manual, chapter 'Command Line Tools' for further information about `rvscli` script.

14.10.3 Failure of Connection to Server with rvsEVO Server Farm after Update

Error Message during server start:

```
2020-11-02 16:30:26,473 [ERROR ] [main      ]
com.tsystems.rvs.evo.EVOClassicServer.?(): Failed to
resolve 'EVOCLUSTER', will use LoopbackAddress :
java.net.UnknownHostException: EVOCLUSTER

[...]
```

Reason:

After update from version 6.02.04 or lower API call fails because alias name for **RMIHostname** cannot be resolved if **`$RVS_HOME`**`/conf/host-alias.properties` file is used for defining the alias name.

Solution:

Use `etc/host` file for defining the alias name of rvsEVO server or set **RMIHostname** to „`0.0.0.0`“:

- stop rvsEVO server
- export configuration into xml file via `exportConfiguration` tool
- change value of **RMIHostname** parameter in xml file to „0.0.0.0“
- import changed xml file via `importConfiguration` tool

14.11 Failure during Launching Scripts in „tools“ Directory

With derby database you can get the following error message during starting a script in `$RVS_HOME/tools` directory:

```
# Another instance of Derby may have already booted
the database C:\rvsRT1\tenants\4011\db.

# ...

Error creating bean with name 'config':  Invocation of
init method failed; nested exception is org.springfra-
mework.transaction.CannotCreateTransactionException:
Could not open JPA EntityManager for transaction;
nested exception is org.hibernate.exception.Generic-
JDBCException: Could not open connection
```

Reason:

Connection to database fails because the rvsEVO server is running.

Solution:

Some scripts can be started with `--api` component though the server is running (please see the rvsEVO User Manual, chapter „rvsEVO Database“).

Stop the rvsEVO server if `--api` is not supported.

14.12 Failure of rvsRuntime Commands

14.12.1 Failure of starting a tenant

Message:

```
Execution of StartCommand failed.
Command is not allowed in license file
'C:\rvsrt1\tenants\4015\conf\license.properties'.
Missing 'V' in licensed components: FAISCJMYKOBRT.
```

Reason:

The license key of the tenant is not valid for multitenancy functionality.

Solution:

Contact the rvs® Service Support Center

Only Windows OS: **Message:**

```
c:\rvsRT1\bin>rvsrt start 4011
Systemfehler 5 aufgetreten.
Zugriff verweigert
Start script for tenant '4011' returned with error.
Executed script : "net start "rvsEVO 4011""
Return code     : 2
```

Reason:

The command box was not opened as administrator.

Solution:

Open the command box as administrator

15 rvsEVO Support

Download

You can download manuals, instructions and software via the following links:

German:

https://system-communication-products.t-systems.net/?redirect=false

English:

https://system-communication-products.t-systems.net/en-DE/?redirect=false

rvsEVO support

Please make the following data available to the rvsEVO support:

- contact person
- license number
- rvsEVO version
- clear description of the problem
- log files as zip or tar file

Contact

Service support contact data:

- from Germany for free, telephone: 0800 664 77 45
- from other countries, telephone: +49 375 606 19 902
- e-mail: rvs-service@t-systems.com

© T-Systems